

湖北理工学院

毕 业 设 计（论文）

题 目： 基于 Python 与 opencv 的

游戏辅玩工具的研究与实现

学 院： 计算机学院

专业名称： 计算机科学与技术

学 号： 202320491312

学生姓名： 邓 烨

指导教师： 伍 红 华

年 月 日

原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品或成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律结果由本人承担。

论文作者签名：

日期： 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权湖北理工学院可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

学位论文作者签名：

指导教师签名：

日期： 年 月 日

日期： 年 月 日

摘要

随着游戏行业的快速发展，自动化游戏辅助工具在提升游戏体验和降低玩家重复性操作方面发挥着重要作用。本论文围绕基于 Python-OpenCV 的游戏辅助工具展开研究，设计并实现一个能够自动完成游戏中的日常任务、战斗及奖励领取的工具。论文说明有需求分析、系统架构设计、核心模块实现及测试验证等关键环节，重点分析图像识别与 ADB (Android Debug Bridge) 技术在游戏自动化中的应用。分析采用 OpenCV 进行游戏界面元素识别，和 ADB 实现自动化操作，让工具能够适配不同游戏场景。系统主要由任务管理模块、图像处理模块、操作执行模块和用户界面模块组成，各模块协同工作，实现任务的智能化执行。论文对任务识别与执行流程进行研究，探讨了图像匹配算法、输入指令优化及多任务并行处理的技术难点，对系统的稳定性和准确性进行了测试评估。经过测试，该工具一定程度上能够有效执行游戏自动化任务，具有一定的应用价值。后续还可以根据实际情况进一步完善与优化。

关键词：游戏自动化；Python；OpenCV；ADB；计算机视觉

Abstract

With the rapid development of the gaming industry, automated game assistance tools play an important role in enhancing the gaming experience and reducing repetitive operations of players. This thesis focuses on the research of game playability tools based on Python-OpenCV, designs and implements a tool that can automatically complete daily tasks, battles and reward claims in the game. The paper indicates that there are key links such as requirement analysis, system architecture design, implementation of core modules and test verification, with a focus on analyzing the application of image recognition and ADB (Android Debug Bridge) technology in game automation. The analysis adopts OpenCV for the recognition of game interface elements and ADB to achieve automated operations, enabling the tool to adapt to different game scenarios. The system is mainly composed of the task management module, the image processing module, the operation execution module and the user interface module. Each module works collaboratively to achieve the intelligent execution of tasks. This paper studies the task recognition and execution process, explores the technical difficulties of image matching algorithms, input instruction optimization and multi-task parallel processing, and tests and evaluates the stability and accuracy of the system. After testing, this tool can effectively perform game automation tasks to a certain extent and has certain application value. It can be further improved and optimized based on the actual situation in the future.

Keywords: Game Automation; Python; OpenCV; ADB; Computer Vision

目 录

前言	1
第一章 绪论	1
1.1 系统的研究背景	1
1.2 国内外研究状况	4
1.2.1. 国内研究现状.....	4
1.2.2. 国外研究现状.....	6
1.2.3. 研究趋势与挑战.....	7
1.3 本文的主要研究工作	8
1.3.1 市场背景分析:	8
1.3.2 研究目标和任务:	10
第二章 系统开发环境介绍.....	11
2.1 Python 编程语言	11
2.2 OpenCV 计算机视觉库	11
2.3 ADB（Android Debug Bridge）	11
2.4 雷电模拟器	12
2.5 scrcpy	12
2.6 NiceGUI.....	13
2.7 json.....	13
2.8 YOLO	13
第三章 需求分析与系统设计.....	14
3.1 可行性分析	14
3.1.1 技术可行性分析.....	14
3.1.2 经济可行性分析.....	14
3.1.3 操作可行性分析.....	15
3.2 系统主要功能模块	15
3.2.1 用户需求分析.....	15
3.2.2 辅助需求分析.....	16
3.3 系统主要功能模块	18
3.3.1 系统架构设计.....	18

3.3.2 主要功能模块设计	20
3.3.3 性能优化设计.....	22
3.3.4 用户界面与交互设计	23
第四章 系统详细设计与实现.....	25
4.1 配置管理模块设计	25
4.1.1 单例配置中心.....	25
4.1.2 智能填充机制.....	26
4.2 自动化任务引擎设计	31
4.2.1 任务抽象基类.....	31
4.2.2 智能重试机制.....	33
4.2.3 YOLO 驱动自动副本战斗	37
4.3 设备交互与图像识别设计	39
4.3.1 ADB 命令封装	39
4.3.2 智能截图系统.....	40
4.3.3 透明图像旋转匹配算法	42
4.4 可视化配置平台设计	44
4.4.1 表单生成.....	44
4.4.2 实时日志系统.....	47
4.5 智能通知系统设计	49
4.5.1 消息发送抽象层.....	49
4.5.2 邮件通知实现.....	50
第五章 系统测试	55
5.1 测试环境与测试方法	55
5.1.1 测试环境.....	55
5.1.2 测试方法.....	56
5.2 功能测试	57
5.2.1 核心功能测试.....	57
5.2.2 GUI 功能测试	68
5.3. 性能测试	78
5.3.1 核心性能指标.....	78

5.3.2 稳定性测试	81
5.4. 安全测试	83
5.4.1 数据安全	83
5.4.2 日志安全	84
第六章 总结与展望	86
参考文献	87
致 谢	90

前言

越来越多的玩家在游戏中感到疲惫，甚至产生厌烦心理。原本是轻松与愉悦的游戏体验，却被每日重复的任务而演变为机械的劳作。这不禁引发思考：当今快节奏的社会环境与游戏设计之间是否存在矛盾？玩家因为工作、学业等现实压力无法保证游戏在线时长时，完成这些重复且耗时的任务就成为了一种负担，甚至催生出“代练”这一特殊现象。玩家希望在有限的时间内获得最大收益，而游戏厂商却用任务系统来延长用户黏性，这种供需失衡推动游戏自动化工具兴起。

本项目对这类需求开发一个能够自动识别并完成日常任务的辅玩工具，不光可以减轻玩家负担、提升娱乐效率，还可以为代练行为的规范化、技术化提供了可行路径。本文以 Python 结合 OpenCV 和 ADB 技术为基础，研究一种高效、实用的游戏自动化解决方案。

第一章 绪论

1.1 系统的研究背景

随着互联网技术和移动设备的普及，全球游戏产业（特别是在中国）爆炸式增长^[1]。游戏从一种娱乐方式变成了文化现象，吸引了大量玩家的同时在社会层面产生了广泛影响^[2]。根据最新的统计数据，全球游戏市场已经突破 1800 亿美元大关，中国市场的份额也日益增加^[3]。同时游戏的种类和玩法越来越丰富，移动游戏作为其中的一大主力，特别在智能手机普及后成了大多数玩家的首选^[4]。随着游戏内容的不断增多，许多玩家在游戏中投入了大量时间和精力去完成更高难度的任务和挑战。

但随着生活节奏的加快，很多玩家面临时间和精力上的困境，越来越多的玩家既要工作又要兼顾家庭，平衡游戏与生活变成了一道难题。很多人发现游戏中有些任务和活动充满乐趣，但重复性的任务占了大多数。每日签到、资源采集、角色升级等需要投入大量时间^[5]。有了这种情况就产生了游戏代练的需求。玩家可以将这些重复或占据大量无意义时间的任务交给专业的代练人员或自动化工具来完成，这样就会节省出更多时间去做自己重要事务^[6]。

游戏代练行业兴起并形成新的经济链条，吸引了大量年轻人参与^[7]。代练为玩家提供了便利，节省时间，也为不少人提供了就业机会^[8]。代练者凭借自己在游戏中的技术从事这项服务^[9]。在一些需要高强度投入的游戏里，代练者通过接单帮别人完成任务就能获得一定经济回报，这样也推动了游戏产业中“时间与技能”的交换。但在市场需求扩大时，代练行业也暴露出行业竞争激烈、工作强度大、社会认同度低等问题，这些问题都使他们生活充满挑战^[10]。例如之前就有新闻报道某大学大学生实习期间与某公司签订游戏直播协议，工作内容就是负责帮别人打游戏，然后某天下播后猝死^[11]。

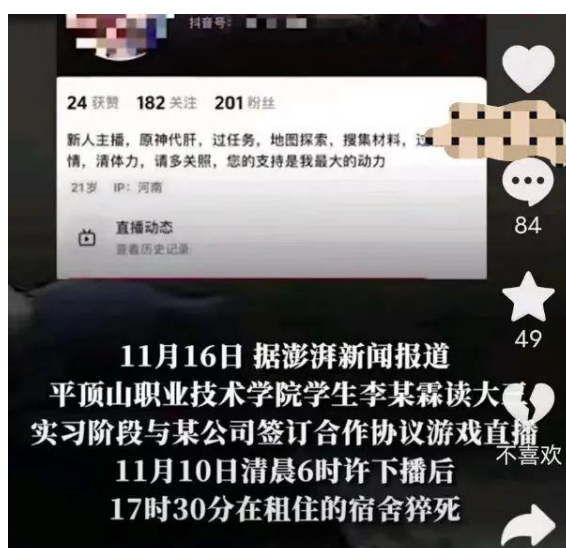


图 0-1 代肝过劳

这样的背景下，游戏自动化辅助工具的需求愈加迫切，因为这些工具能自动完成游戏里的一些重复、繁琐的任务，帮忙节省时间，减轻负担。特别是一些需要长期投入的任务，自动化工具的效率就会变得很高，它能模拟玩家操作且在不影响游戏体验的前提下完成任务。例如借助 Python 与 OpenCV 技术研究的自动化工具能通过图像识别和图像处理来获取游戏里的信息进而模拟玩家的点击和操作。这一技术的应用致使游戏代练不光只靠人工操作来提升效率，满足市场对快速高效服务的需求。



图 0-2 游戏搬砖

随着技术的不断发展，特别是计算机视觉与人工智能领域的提升，游戏自动化工具的功能也日益进步。传统的代练工作在游戏中持续进行高强度的人工操作，而

AI 兴起后，自动化工具能通过学习来优化任务执行甚至在复杂环境下自己作出决策。未来的游戏辅助工具基于深度学习和强化学习等技术的不断学习能自主识别任务目标并调整策略，以更高的智能水平模拟人类玩家的行为会极大提高代练的效率和准确性。

游戏自动化工具的研究和开发除了提升代练效率外事实上也对相关技术的进步领域起到推动作用。自动化工具的实现，计算机视觉、图像识别、机器学习等技术被广泛应用且也在其他行业从中得到重要的启示并进行应用。例如，自动化测试和图像识别技术的发展，不仅给游戏行业提供了技术支持，也为摄像头监控网络等其他领域的自动化操作和人脸识别技术提供了实践经验和技术解决方案。



图 0-3 代肝工作

然而，随着游戏自动化工具的普及后游戏行业有了一些新的问题和挑战。游戏厂商对于代练和自动化工具持比较慎重的态度，他们都可能影响游戏的公平性和竞争性。一些游戏厂商已经对自动化工具采取了反作弊措施加以限制并加强了对代练行业的监管。在技术与行业规则间寻求平衡成为当前游戏自动化工具面临的问题^[11]，开发者必须面对这个重要课题，怎样设计一个既能提升效率又不违背游戏规则的工具。

这样的环境下，游戏辅助工具的需求不仅来自玩家，那些想靠代练赚钱的从业

者更是，为了解决代练过程中可能出现的效率和公平性问题，开发出更高效智能的游戏自动化工具成了技术研究的关键所在。未来随着人工智能技术的不断走向成熟，这些自动化工具不光只能完成传统的任务，还会在更复杂的游戏场景里提供个性化服务，进一步提升玩家的游戏体验。

综上，游戏产业的发展且玩家对生活 and 游戏平衡需求，游戏自动化辅助工具已经成为一个重要的研究方向，不但改变玩家的游戏体验，还能推动游戏代练行业的发展。未来的游戏自动化工具更是因为技术不断的进步更加智能化、高效化，为玩家和代练行业从业者提供更好的服务，同时也为游戏行业的创新与变革提供更多可能性。



图 0-4 代肝工作

1.2 国内外研究状况

过去的几年里，游戏产业迅速发展和 18-22 年受环境、疫情等多方面的因素不断影响着国内外关于游戏辅助工具的相关情况。、在工具和代练服务的研究有了显著进展。尤其是在自动化控制和人工智能（AI）应用方面做得很好，研究者们不断尝试通过技术手段优化游戏体验，提高游戏代练的效率，解决玩家的各种游戏需求。以下是国内外在该领域的研究现状和发展趋势。

1.2.1. 国内研究现状

在中国，随着移动互联网的普及和智能手机游戏的崛起，游戏产业进入了一个快速发展的阶段。尤其是随着电竞产业的兴起，代练行业也逐步形成，并成为了一

个庞大的市场^[13]。随着对游戏自动化工具的需求逐渐增加，国内不少学者和开发者开始着手研究游戏辅助工具的开发和优化。

国内的研究大多集中在以下几个方面：

游戏自动化与代练工具的开发：一开始，游戏的自动化应用在游戏公司对游戏产品的测试上，使游戏产品能够快速通过测试，一定程度上考虑代替人工成本，同时为了使自动化测试顺利进行，游戏公司生产的游戏逐渐同质化，国内许多学者也使用他们的工具来进行自动辅助，较为有名的是网易的 `airtest` 和腾讯的腾讯云自动化测试平台。不使用企业自动化测试技术(工具)的技术团队则自己开发或者模仿别的技术，例如按键精灵，大漠插件等等。到了近几年，越来越多的人成为了独立开发者，用各种语言和技术开发游戏自动化工具，发布在淘宝或者 `github`。现在手机游戏盛行，结合 `OpenCV` 和 `ADB` 等技术来处理重复任务成了开发自动化游戏工具的主流方案。通过图像识别、路径规划、自动点击等技术模拟玩家在游戏操作，以此完成每日任务、资源收集、角色升级等重复任务。这些技术工具不仅为玩家节省了时间，也给代练行业给予技术支持。

随着人工智能技术的快速发展，越来越多的国内研究开始将深度学习和强化学习等技术应用于游戏自动化领域。通过训练 `AI` 模型来使程序智能识别游戏中的复杂场景并根据游戏的反馈做出合理决策。这些技术提升工具的适应性和稳定性，还给玩家带来了个性化的体验。例如：训练 `yolo` 模型自动玩 `dnf`。^[14]

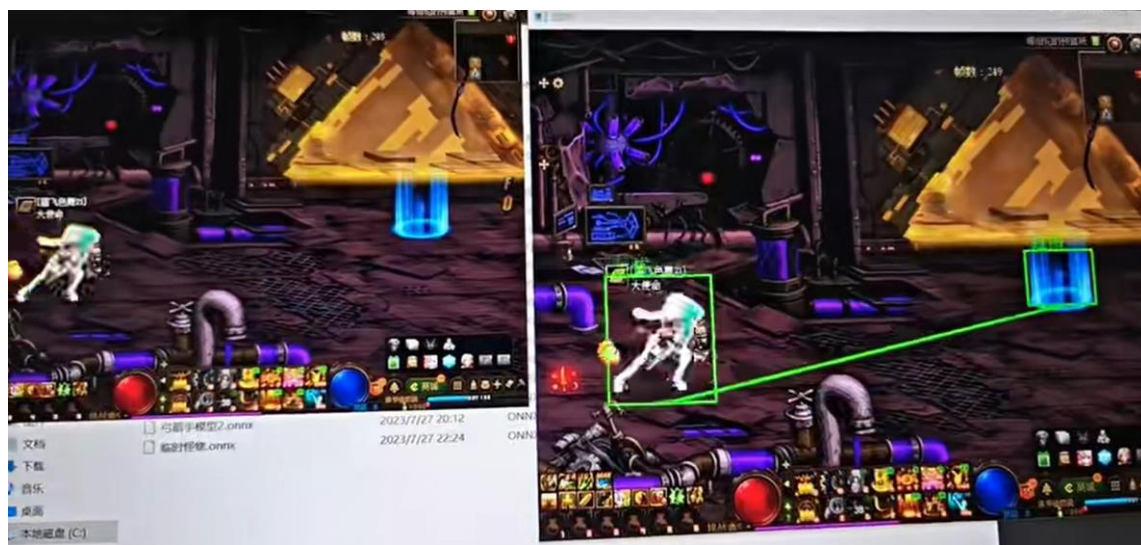


图 0-5 利用 YOLO 玩 DNF

游戏反作弊与自动化工具的博弈：由于游戏自动化工具的普及，游戏厂商也逐

渐加强了对代练和自动化工具的监管。在中国，许多热门游戏已经开始实施反作弊机制，尤其是在手游领域，采用了基于机器学习和行为分析的技术来识别和屏蔽自动化工具。国内研究者在这方面的工作主要集中在如何识别自动化工具的使用、如何对抗反作弊系统，以及如何通过技术手段实现“隐形”代练^[15]，即减少被检测到的概率。例如：使用防火墙，重新检测游戏文件，查询设备内存，扫描手机应用或者蜜罐技术等等发现作弊者并封禁；在 pvp 的游戏里匹配时检测使用的设备，根据账号玩家的强度分配同等强度的对手，使请代练的玩家受到“惩罚”；提供相应的充值服务可以一键完成所有任务等等。

代练行业的社会与经济研究：除了技术研究，国内也有部分学者关注到代练行业的社会经济问题。由于代练行业相对隐秘且缺乏正规化管理，很多研究着眼于其带来的社会影响，如代练者的劳动条件、代练市场的规范化等问题^[16]。如何平衡玩家、代练者与游戏公司之间的利益，成为了一个值得探讨的课题^[17]。

1.2.2. 国外研究现状

国外在游戏辅助工具和自动化测试方面的研究起步较早，尤其是在欧美地区，现在相关研究逐渐走向深度和广度^[18]。

国外研究很早开始关注游戏自动化测试的需求，开发各种自动化工具来进行测试，使测试人员能够高效地进行游戏测试，游戏产品的质量保障也有很大的提高，特别是在大型游戏开发过程中^[19]。常见的自动化测试技术包括基于图像识别的自动化脚本、虚拟化测试平台的搭建以及性能测试工具的开发^[20]。欧美一些游戏公司已经将自动化测试使用到游戏开发的全过程，包括性能测试、功能测试、稳定性测试。这些技术的应用大大提高测试效率，降低人力成本。例如有名的 Appium 开源的自动化测试框架和 Unity 的 utf 等等^[21]。

他们也早早将 AI 技术应用于游戏自动化领域，使用强化学习和神经网络等方法，使其在游戏中自主探索并学习最优的操作策略^[22]。这一技术应用不仅限于传统的游戏代练和测试工具，还使用到游戏开发的创作过程中^[23]。例如，使用 AI 来生成游戏内容、设计游戏情节以优化玩家体验等。这推动了游戏自动化技术的同时也给游戏产业带来了创新^[24]。例如 testers.ai, gamedriver 等等。就在 2024 年也就是今年，在 2 月时，Konami 公司下的一个游戏“游戏王 md”进行二周年活动，在直播时提到为了庆祝二周年而做了一个 md-ai 系统^[25]。这是因为游戏公司发现很多玩家喜欢挂脚本赚取资源，所以宣布上线官方脚本暗示玩家购买。



图 0-6 MD-AI 系统

国外学者对代练行业的研究大多集中于其经济效益和市场需求方面，游戏代练从一开始单纯的“玩家帮助”发展成产业链的一部分，代练为玩家提供了便利的同时也给年轻人提供了就业机会，尤其是在东南亚地区。随着代练行业的迅速发展和其经济效益的不断膨胀，代练行业的可持续性、行业规范化及其对传统劳动力市场的冲击被研究者探讨。

游戏厂商在代练行业兴起的同时间，越来越重视反作弊技术的研究。在电子竞技和在线多人游戏中，欧美地区的研究主要集中在如何通过技术手段来限制不公平竞争。此外关于游戏代练的道德问题也被广泛讨论。部分学者觉得游戏代练会破坏游戏的公平性和影响玩家的成就感，而另一些学者则认为存在即合理，代练应当得到一定程度的认可和规范。

1.2.3. 研究趋势与挑战

当前，游戏辅助工具和自动化技术领域研究在国内外还面临着一些挑战。自动化工具的应用虽说提升了代练效率，可如何确保相关工具不违反游戏规则且不破坏游戏生态系统仍是个有待解决的问题^[26]。更有的是不断在发展的 AI 和深度学习技术，避免 AI 失控，确保它不会过度干预游戏体验且不会操控玩家行为也是未来研究的重点。

总之，游戏自动化工具、代练行业和相关技术方面的研究正处于快速发展的阶段且还在不断演进。未来游戏相关产业发展会随着进步不断涌现的更多创新解决方案而更进一步。

1.3 本文的主要研究工作

本文的主要研究工作是开发一款基于 Python 和 OpenCV 的自动化辅玩工具，主要研究提高《火影***手游》玩家的游戏体验，优化日常任务等重复性任务。随着《火影***手游》自 2016 年上线以来全球范围内的玩家基数不断增加，游戏不光成为了玩家的娱乐方式，也吸引了大量代练和辅助工具的需求。本文会对这一市场趋势进行探讨并为给它提供一种技术解决方案。

1.3.1 市场背景分析：

《火影***手游》作为一款在国内外都极具影响力的格斗类手游，深受玩家喜爱。公开统计显示到 2022 年为止这游戏的日活跃用户数已达千万之多且特定时期玩家在线人数常突破峰值，玩家对游戏体验的要求一天天提高。许多人被游戏中的重复性任务困扰，这些任务占用大量游戏时间且影响玩家的整体体验。



图 0-7 2024 年 10 月 火影***手游 iPhone 设备端下载量



图 0-8 2024 年 10 月 火影***手游 iphone 设备端收入

对于一些职业玩家或代练来说，完成这些重复任务是他们获得收入的主要方式，同时游戏里部分任务对游戏技巧和时间投入要求比较高，使得一些时间不充裕的玩家没办法高效完成任务。这种情况下自动化辅玩工具的开发具有广阔的市场前景。



图 0-9 淘宝火影手游代肝

1.3.2 研究目标和任务：

本研究以设计并实现一款基于 Python 和 OpenCV 的自动化辅玩工具为核心目标，通过模拟用户操作来给玩家提供帮助。在《火影***手游》里日常任务、周常任务等重复内容由系统完成从而减少玩家手动操作的时间。具体研究任务如下：

游戏任务自动化：将 OpenCV 图像识别技术加以结合以达成对游戏里任务界面、操作按钮等的精确识别进而模拟玩家执行任务操作从而提高了任务完成效率，让玩家放开双手，给玩家节省大量时间。

优化操作效率：在不违背游戏规则且确保公平性的情况下，尽量最低限度地使用玩家电脑资源来执行自动化脚本进行任务，以使玩家能在最短时间内完成指定内容。不干扰玩家的日常生活，减少玩家在游戏中的重复性操作负担。

UI 设计与用户体验：为了让工具更方便使用，用户界面将会被设计的简洁直观，以便玩家能够轻松设置任务类型、时间安排等参数。

第二章 系统开发环境介绍

本章主要介绍实现《火影***手游》的自动化辅玩工具里涉及到的技术和工具，包括 Python 编程语言、OpenCV 计算机视觉库、ADB（Android Debug Bridge）工具、雷电模拟器、scrcpy 以及 NiceGUI。下面详细介绍这些技术在开发中的应用。

2.1 Python 编程语言

Python 是一种高级编程语言，以其简洁、易读和强大的库支持而闻名^[27]。特别适用于快速开发原型和实现自动化任务。在游戏辅助工具开发中，Python 的优势突出，主要体现在以下几个方面：

丰富的第三方库：Python 具有丰富的第三方库，如 OpenCV、multiprocessing、ADB 等，可以快速构建自动化工具。

轻脚本：Python 语法简单，其本身具备强大的脚本编写能力，可以使用比 C 或 java 更少的文本量编写出相同功能的程序。

开源：Python 解释器和模块是开源的，语言本身自由，模块的功能可以随意修改。

2.2 OpenCV 计算机视觉库

OpenCV（Open Source Computer Vision Library）是一个开源计算机视觉和机器学习软件库，广泛用于图像处理、视频分析和物体识别等领域。在游戏自动化中，OpenCV 可以用于图像识别和界面元素的检测，这是实现自动化交互的关键技术之一^[28]。

图像识别与模板匹配：OpenCV 具有强大的图像处理能力，可以根据提供的模板准确识别到对应图片所在位置，这样就可以使工具准确识别按钮，弹窗，任务图标等^[29]。

图像预处理：OpenCV 可以对图像进行灰度化、二值化、边缘检测等处理以便提高图像识别的准确性，减少误操作和提高任务完成效率。

高效的性能：OpenCV 内部优化使用多线程和 SIMD（单指令多数据流）技术，对于实时和大规模的计算任务有很好的性能表现。

2.3 ADB（Android Debug Bridge）

ADB 是一个命令行工具，它使开发人员能够与 Android 设备进行交互^[30]。而 Python 可以通过编写脚本使用 cmd、管道或者其他命令行的方式使用 ADB 能够与 Android 设备进行直接的通信并执行模拟点击、滑动、截屏等操作^[31]，是自动化工

具中的一个关键组件。

设备控制：使用 ADB 与 Android 设备进行交互，实现按键事件、触摸事件和滑动操作等。例如，模拟点击游戏中的按钮或滑动屏幕以进行任务操作。

截图与屏幕内容获取：ADB 可以截取设备屏幕的内容并保存到电脑之中，结合 OpenCV 获取截图信息并进行操作。

调试与日志功能：ADB 本身可以对 Android 设备进行调试，可以查看设备的日志信息，这对开发自动化工具时的执行跟踪问题，调试代码和优化性能起到重要作用。

2.4 雷电模拟器

雷电模拟器是一款流行的 Android 模拟器，可以在 PC 上运行 Android 应用，一款专门给手游使用的模拟器平台。对于《火影***手游》这类移动游戏，雷电模拟器提供了一个稳定的开发和测试环境，而用户也会选择这一类模拟器在电脑游玩手游，所以在开发和调试的过程中，对雷电模拟器的使用是必不可少的

模拟 Android 设备环境：雷电模拟器提供了一个虚拟的 Android 设备环境，支持多开、设备模拟等功能，开发者可以在 PC 上方便地进行游戏的自动化操作。

性能优化：与物理设备相比，模拟器通常在性能调优上更加灵活，可以针对不同硬件配置进行优化，提高自动化工具的运行效率。

调试与截图功能：雷电模拟器本身提供了根据 adb 修改而来 ld 命令行工具，使得模拟器本身更加方便调试，加上内置的屏幕录制和截图功能，可以方便地捕捉游戏过程中的关键图像，供 OpenCV 进行后续分析。

2.5 scrcpy

scrcpy 是一个开源的 Android 屏幕镜像工具，它允许通过 USB 连接或无线网络将 Android 设备的屏幕显示在 PC 上并控制^[32]。

实时控制与屏幕镜像：通过 scrcpy 可以将游戏画面实时投射到 PC 屏幕通过脚本来实现对游戏界面的实时控制。

高效的设备交互：scrcpy 具有低延迟和高分辨率的特点，可以进行更精准的点击、滑动等操作，同时 scrcpy 本身兼容 adb 调试工具，通过 adb 来固定 scrcpy 窗口的分辨率，使得识别更加稳定。两者可以交叉并行使用。

多设备支持：scrcpy 支持多个设备的连接与管理，能同时操作多个虚拟设备，可以进行批量任务处理和大规模的自动化测试。

2.6 NiceGUI

NiceGUI 是一个基于 Python 的现代化 Web UI 框架，适用于桌面应用和 Web 界面^[33]，相比于传统的 Qt 或 Tkinter 更加自由，并且可以通过 Python 直接控制 UI 组件，灵活易用^[34]。轻量级的特性和快速开发能力，让它成为构建游戏自动化 UI 的理想选择。

2.7 json

JSON（JavaScript Object Notation）是一种轻量级的数据交换格式，广泛用于数据存储和传输。本项目中，JSON 用于存储游戏自动化工具的配置信息、任务执行参数和日志数据。其结构清晰、可读性强，适用于配置文件和数据交换。JSON 独立于语言设计，很多编程语言都支持 JSON 格式的数据交换，这其中就有 python。python 提供了 json 标准库，方便进行解析，让系统能够灵活调整参数，提高可扩展性。

2.8 YOLO

YOLO（You Only Look Once）是一种基于深度学习的目标检测算法，以其高速、端到端的检测特性而广泛应用于计算机视觉领域^[35]。与传统的候选框方法不同，YOLO 将目标检测视为一个回归问题，直接从图像中预测出边界框的位置和类别，大幅提升了检测速度。YOLO 系列自发布以来不断迭代升级，从最初的 YOLOv1 到最新版本，精度和效率均有显著提升。该算法适用于实时检测任务，如视频监控、自动驾驶、工业检测等场景。由于其开源和社区活跃，YOLO 也成为研究与应用中广受欢迎的目标检测框架之一。

第三章 需求分析与系统设计

3.1 可行性分析

3.1.1 技术可行性分析

项目使用 Python 语言开发，利用 OpenCV 进行图像识别和图像处理，ADB 对安卓模拟器的控制。Python 拥有丰富的第三方库和模块，例如 multiprocessing 用于多进程管理使任务高效执行。

用户界面采用 NiceGUI 框架搭建。界面组件丰富，事件处理便捷。用户可直观配置任务，实时监控任务状态。模块化设计，任务管理、界面交互、后台运行相互独立。系统可维护性大幅提升。

配置管理采用 JSON 文件保存，用户可根据需求快速切换不同配置。任务执行期间，支持图像识别后的自动点击、滑动等多种操作场景。异常处理和日志记录功能齐全，保障执行稳定性和可靠性。

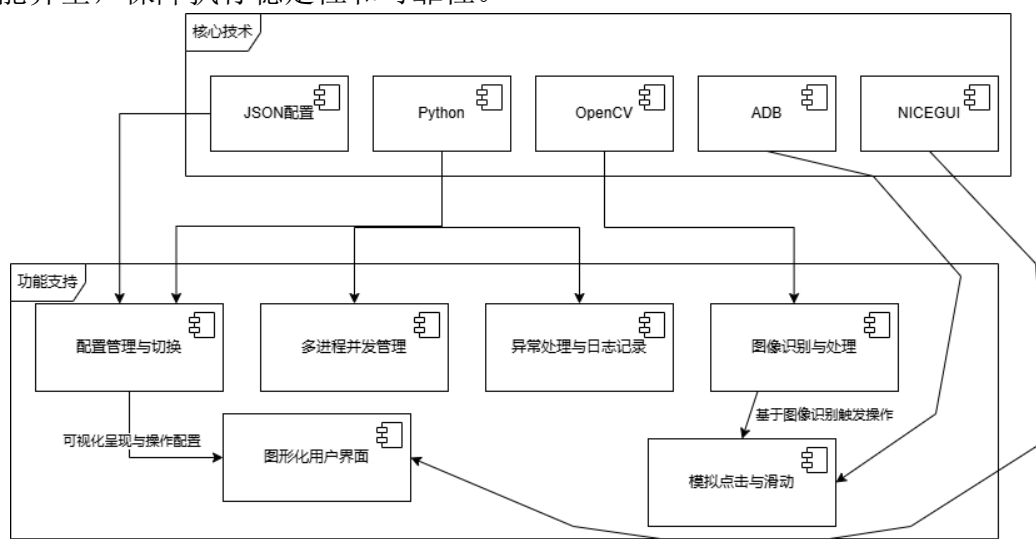


图 0-1 技术可行性分析

综上，本项目工具链完善，具备技术可行性，能够有效支持游戏自动化辅玩工具的开发与实现。

3.1.2 经济可行性分析

本项目在经济方面具备较强可行性。

开发方面，使用 Python 语言。利用 OpenCV、ADB 等开源免费工具，无需支付授权费用。Python 生态完善，社区资源丰富，技术支持便捷，二次开发门槛较低。开发环境使用普通个人电脑开发工具，硬件要求不高。

维护方面，采取模块化设计。代码结构清晰，升级和修复高效。配合自动化测

试，系统稳定性有保障，后续维护工作量可控。

收益方面，工具可大幅减少玩家重复操作时间。替代人工，降低代练行业用人成本。潜在需求广泛。可通过售卖工具、订阅服务、代练任务等模式实现收入。未来亦可拓展定制功能、增值服务，进一步提升收益水平。

整体来看，开发投入低，维护压力小，市场潜力大，经济回报预期良好。

3.1.3 操作可行性分析

本项目的操作可行性分析主要围绕系统的使用便捷性、兼容性以及用户适应性进行评估。如表 3-1 所示。

使用便捷性	使用图形化用户界面（NiceGUI），用户无需编写代码即可使用页面配置任务，如选择任务顺序、设置某任务内容等。执行时，日志与监控模块可实时反馈任务进度，用户能随时掌握执行情况。
兼容性	本系统适用于 Windows 环境，支持包括雷电模拟器等各个模拟器运行的安卓游戏，并通过 ADB 实现对游戏的直接控制。此外，使用 OpenCV 进行图像识别，不依赖游戏内部 API。
稳定性	项目采用模块化设计，支持快速配置与功能拓展，无需额外投入大量人力物力进行重新开发。同时用 JSON 文件作为配置文件，用户可以使用简单的文本编辑器对任务流程进行个性化定制。配置提供了任务顺序设置、任务状态显示和异常提示等功能，使操作过程更加透明和可控。
用户适应性	本工具主要面向有自动化需求的普通玩家和游戏代练群体，界面设计简洁，操作逻辑清晰，即使是非技术用户也能快速上手。同时，支持个性化参数设置，满足不同用户的需求。

表 0-1 操作可行性分析

综上所述，本系统在操作上具有较高的可行性，能够为目标用户提供高效、便捷的自动化游戏体验。

3.2 系统主要功能模块

3.2.1 用户需求分析

在游戏里，大量玩家每天都要重复完成如签到、日常任务、活动任务等重复任

务。这些操作不但浪费自己休闲时间，也在磨损自己的心情。针对这一情况，这个实现自动化工具的想法就产生了。用自动化手段帮助玩家完成这些重复任务，提升游戏体验，同时保证玩家在休闲时能充分娱乐。

目标用户群体

1. 普通玩家：普通玩家希望减少日常游戏中的重复操作，将更多时间投入到游戏的核心玩法中。使用这个软件使玩家自动完成签到、日常任务、领取奖励等低价值操作。

2. 游戏代练人员：游戏代练人员需要高效完成客户委托的游戏任务。项目支持批量任务执行和配置切换，提升代练效率，减少人为操作风险。

3. 游戏测试人员：项目的自动化操作能力也可用于游戏测试，通过重复执行任务验证游戏功能的稳定性和可靠性。

功能需求

1. 自动化任务执行。软件需覆盖每日任务、周常任务、活动任务等常见项目。执行时限需符合预设要求。

2. 任务配置管理。使用 JSON 文件进行任务定制。用户自由选择任务种类、执行顺序、具体参数。

3. 实时监控与日志管理。提供实时任务状态展示。输出日志，方便快速定位异常情况。

4. 异常处理与恢复。具备异常检测机制。遇到错误自动重启任务，必要时通知用户。

非功能需求

1. 易用性。构建图形界面。用户操作简单，无需编程基础。

2. 兼容性。适配主流安卓模拟器。保持图像识别准确。

3. 安全性。软件不传输敏感信息。仅通过 ADB 进行本地模拟，数据留存本地。

项目聚焦用户真实需求。提供高度定制化自动化工具。目标明确，节省操作时间，提升整体效率，优化使用体验。

3.2.2 辅助需求分析

在满足基本的自动化任务执行需求之后还需要提供一些辅助功能来提升用户体验、增强系统稳定性，使用户能够高效地管理和监控任务。辅助需求主要围绕任务管理、日志监控、配置维护和异常处理。

1. 任务管理辅助需求

任务优先级设置：用户可自己所想来设置执行优先级，确保哪些关键任务优先完成。项目应提供任务顺序调整功能来满足不同场景下的需求。

任务暂停与恢复：在任务执行的过程中，用户可能需要中途暂停任务自己来操作。软件需要提供暂停或终止任务执行的按钮，方便用户灵活管理任务执行。

特定任务执行：对于特定的任务，例如限时任务，软件需要提供单任务执行功能让特定任务单独快速执行。

2. 配置管理辅助需求

JSON 配置管理：用户通过 UI 自定义 JSON 文件里任务的执行逻辑和参数，项目需要提供友好的配置管理功能，允许用户导入、导出和编辑配置文件。

配置校验与提示：在用户提交配置时，系统应提供自动校验功能，检测可能的配置错误并给出提示，避免因配置错误导致任务失败。

3. 日志与监控辅助需求

实时日志显示：在用户界面提供日志窗口，实时展示任务执行的详细信息，可以让用户知道软件是否真的在执行任务，再执行什么任务或者任务是否执行错误。

日志存储与分析：支持将日志以时间命名保存到电脑中，方便用户或开发者后续分析和排查问题。

错误提示与告警：在任务执行过程中，如果遇到异常或错误情况，软件应及时在 GUI 界面里提示警告并记录错误信息，方便用户和开发者定位问题。

4. 异常处理辅助需求

自动重试与恢复：任务执行失败时，软件要有重试的功能并在多次重试后依然失败时提供异常报告且对用户进行通知。

资源占用检测：执行任务的时候，软件需要监控并控制 CPU 和内存占用，防止因资源不足导致程序崩溃异常。

异常退出处理：软件因为任务异常退出时需要自动记录相关信息并及时释放程序不占用电脑资源，保证后续任务能够正常执行。

综上，项目的辅助需求是实时监控和处理异常情况保障任务的顺利执行，保证并提升用户操作的便捷性和任务管理的效率。这些功能使系统在各种场景下都具备一定的稳定性和可靠性。

3.3 系统主要功能模块

3.3.1 系统架构设计

该项目系统架构采用模块化设计，包括用户界面模块、任务管理模块、任务执行模块、日志管理模块和配置管理模块。各模块之间通过配置文件和进程通信机制进行交互来确保任务执行的高效性和灵活性。

1. 用户界面模块

用户界面模块基于 NiceGUI 框架实现，提供直观简洁的任务配置与执行界面。用户打开 GUI.exe 选择任务、配置任务参数，执行任务时可查看实时日志。界面模块与任务管理模块用任务指令、多进程调度和日志接口进行交互。

2. 任务管理模块

任务管理模块负责管理任务的生命周期，包括任务启动、执行、暂停和终止等功能。运行时创建一个任务池，同时保证同一任务配置不会重复启动。多个任务则会使用多进程任务池管理多个进程任务并使用消息队列进行日志和状态信息的传递。

3. 任务执行模块

任务执行模块是系统的核心，有核心任务调度和单任务进程两个模块。前者根据用户配置执行完整的任务流程，后者用于执行单任务的特定功能模块。利用多进程机制确保任务的稳定性和独立性。

4. 日志管理模块

日志管理模块采用进程间队列传递任务执行过程中的日志信息。用户界面模块通过监听任务的日志队列实时展示日志内容，便于用户监控任务进度。

5. 配置管理模块

配置管理模块则是对 JSON 文件进行解析，程序对任务参数的存储和读取。项目支持通过 exe 形式直接执行 JSON 配置文件，无需图形界面。用户可以在快捷方式路径后加上 相应的 JSON 配置文件名，以便快速启动特定任务。而且多个实例的自动化可以同时运行不同的配置文件任务，实现多任务并行^[36]。

系统交互流程

- 1.用户通过 GUI 或直接运行 exe 指定 JSON 配置文件。
- 2.任务管理模块解析 JSON 文件并启动任务执行模块。
- 3.任务执行模块按照配置执行任务，同时向日志管理模块发送执行信息。

4. 日志管理模块将任务日志展示在用户界面，方便用户实时监控。
5. 任务执行完成后，任务管理模块清理相关资源，更新任务状态。
6. 通过上述架构设计，项目实现了任务的高效管理和执行，同时保证了用户友好的交互体验和任务的可追溯性。

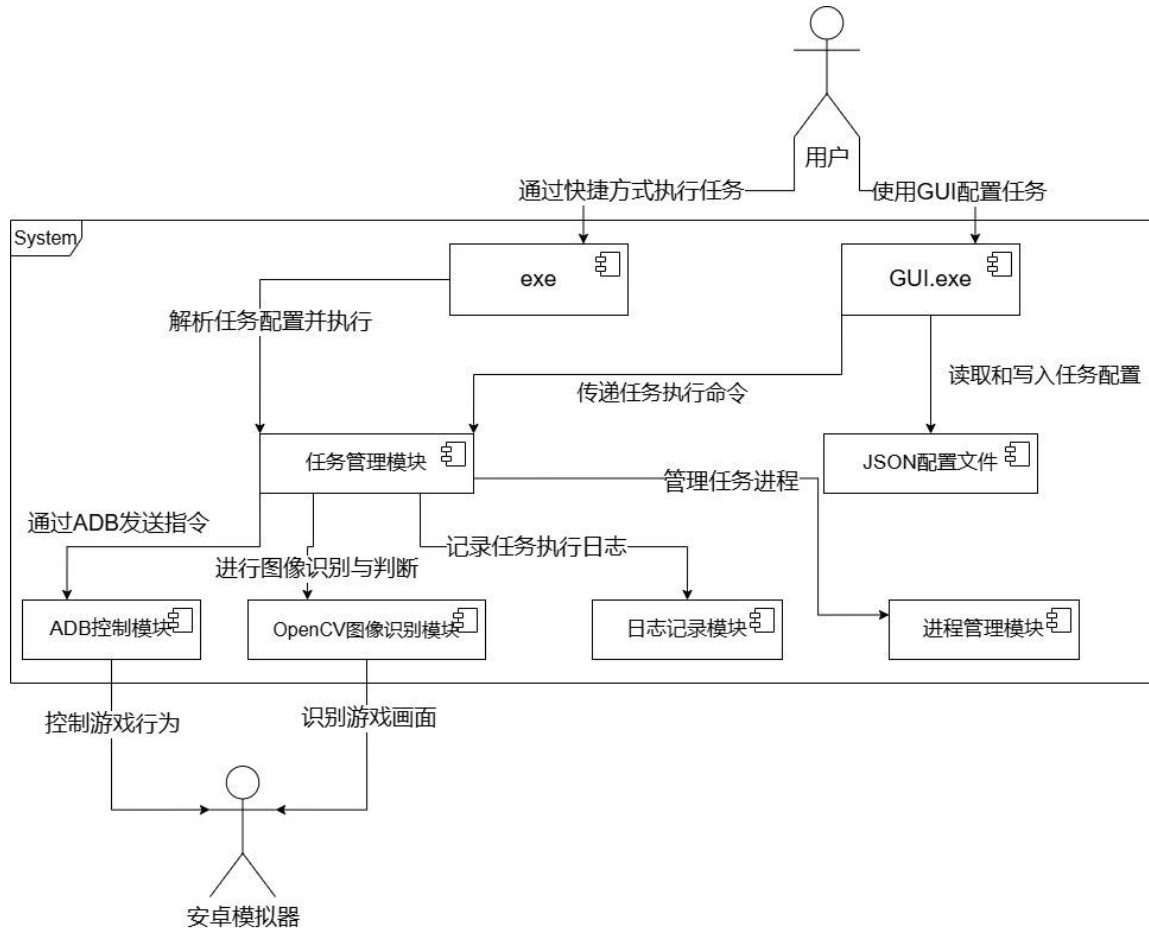


图 0-2 辅玩工具流程图

3.3.2 主要功能模块设计

软件项目采用模块化设计，以便于功能的扩展和维护。系统主要分为以下功能模块：

1. 配置管理模块

配置管理模块负责解析、读取和存储 JSON 配置文件。用户在用户界面创建配置文件。配置管理模块的功能包括：

配置解析：解析配置文件中的任务执行顺序、参数设置和任务启用状态。

配置验证：检查配置文件格式是否正确，参数一定要完整。

多任务支持：支持加载和切换多个配置文件同时运行不同的任务。

2. 任务管理模块

任务管理模块是项目的核心模块，负责任务的创建、调度和执行。功能包括：

任务解析与调度：解析并根据配置文件中设定的任务顺序依次执行任务。

任务状态管理：记录任务执行状态，出现异常执行相应异常处理。

单任务与多任务支持：创建多进程实现任务的执行。

3. GUI 模块

GUI 模块通过 NiceGUI 实现用户友好的界面并提供任务配置、启动、停止和日志监控等功能。功能包括：

任务配置界面：提供直观的界面编辑任务执行顺序和参数。

任务状态监控：显示任务执行状态和详细日志。

快速执行功能：通过相应任务按钮快速执行任务。

4. 日志管理模块

日志管理模块用于记录任务执行过程，包括：

实时日志显示：在 GUI 中显示实时任务执行中的实时日志。

日志存储：将日志保存到本地。

多任务日志管理：不同任务分配日志窗口。

5. 设备交互模块

设备交互模块通过 ADB 与 Android 模拟器进行交互，主要功能包括：

截图与图像识别：调用 OpenCV 进行图像识别和匹配。

模拟点击与滑动：使用 ADB 发送触摸和滑动事件。

任务自动化：根据识别结果执行对应任务。

6. 进程管理模块

进程管理模块使用 Python 第三方库的 multiprocessing 实现任务的独立运行。

主要功能包括：

多进程调度：创建和管理不同的任务进程。

消息队列通信：任务进程通过消息队列进行信息交互。

状态管理：监控进程状态和任务执行进度。

将这些模块的协调配合，整个软件系统实现高效的任务管理、用户友好的交互界面以及可靠的任务执行机制。而且模块化设计让系统具备良好的扩展性和维护性。

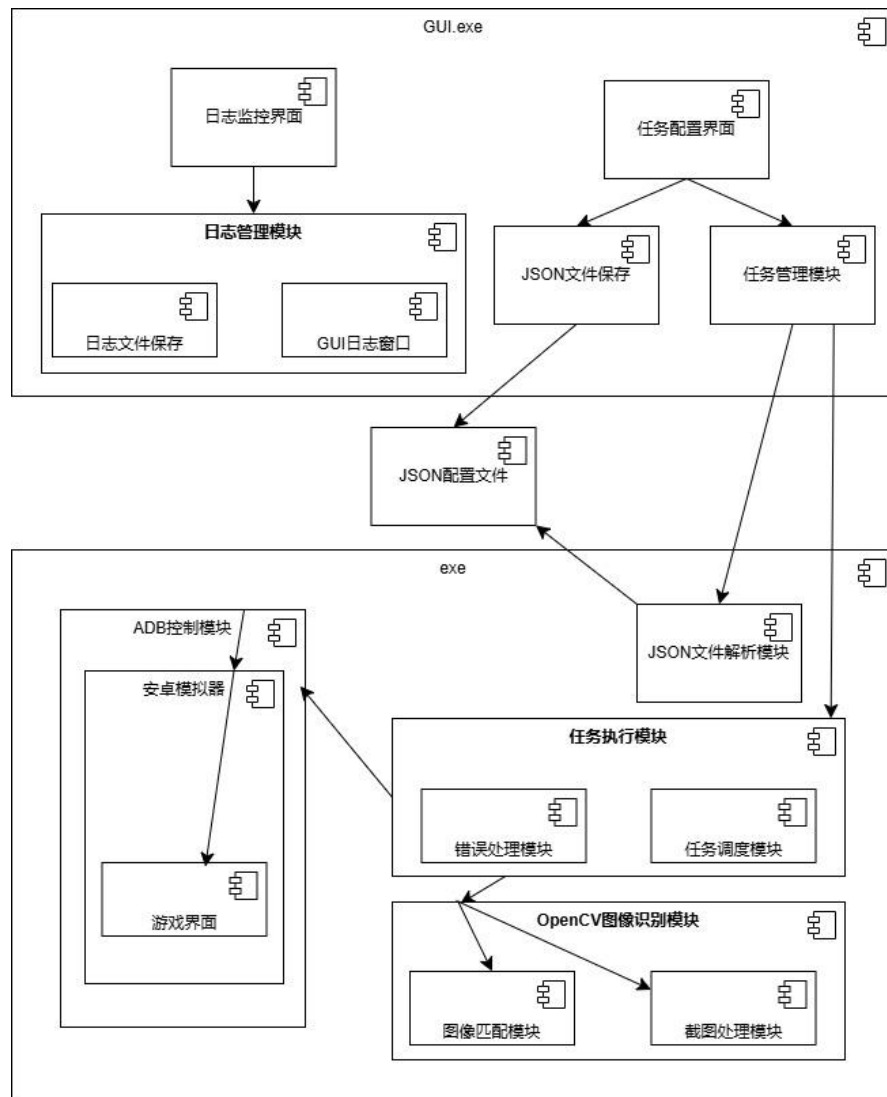


图 0-3 辅玩工具组件图

3.3.3 性能优化设计

在系统的设计和开发过程中，为了确保任务执行的高效性和稳定性，进行了多方面的性能优化。主要的优化方向包括多进程管理、图像处理加速、任务执行优化和日志管理优化。

1. 多进程管理优化

使用 `multiprocessing` 模块实现多进程并行执行，创建多进程池管理任务进程。多进程架构有效避免 Python 的 GIL（全局解释器锁）限制使 CPU 密集型和 I/O 密集型任务能同时执行并通过 `Manager.Queue` 在进程间传递任务状态和日志信息减少了阻塞等待的时间来提升整体运行效率。

2. 图像处理加速

使用 `OpenCV` 进行模板匹配、颜色检测和 OCR 识别。针对游戏场景中常见的静态 UI 元素，使用以下措施来进行优化：

模板缓存：针对频繁识别的按钮和界面元素，用模板缓存机制来减少重复计算。

多尺度匹配：缩放图像后进行匹配，提升了识别速度

ROI 限定：部分场景里使用检测特定区域（`Region of Interest`），避免全屏扫描来缩短识别时间。

3. 任务执行优化

任务执行模块在 `InTraining.py` 等文件中使用 `run_until` 方法实现了高效的执行逻辑。

条件判断优化：短路判断和快速退出机制保证在条件满足时立即结束任务减少无效点击。

动态调整执行频率：根据任务复杂度和页面反馈速度来动态调整点击和检测的间隔时间，降低 CPU 占用率。

异常检测与恢复：针对可能出现的界面卡顿或识别失败情况提供了异常检测与自动恢复机制提升任务的成功率。

4. 日志管理优化

在任务池管理中，使用 `Queue` 实现任务日志的异步写入和传输。日志系统含以下特点：

分层日志：按照任务名称和进程 ID 分配独立的日志通道快速定位问题。

实时推送：通过非阻塞的 `get_nowait()` 方法获取日志信息使界面及时更新。

异常捕获：任务执行过程时所有异常信息都会通过日志记录。方便后续分析和优化。

以上优化使系统能在执行复杂任务时具备良好的响应速度和稳定性，保证了较低的资源消耗。

3.3.4 用户界面与交互设计

项目采用 NiceGUI 作为主要的前端框架，结合 Python 后端进行任务管理和用户交互，提供直观且功能丰富的图形用户界面（GUI）。整体界面设计注重简洁性、易用性、实时性和轻量化，使用户能够高效地完成任务配置、任务执行和结果查看，同时减少系统资源占用。

1. 界面布局设计

软件界面采用多页面布局。

任务配置页面。通过 `Setting_task_order.py` 模块，添加、删除、调整任务顺序。支持全选、全不选操作。任务可单独启用或禁用。

快速任务执行页面。`show_fast_run_task_buttons` 模块提供一键启动常用任务功能。简化操作流程。

日志监控页面。任务执行期间，`logArea` 区域实时输出日志。用户可直接查看执行状态，快速排查错误。

2. 交互逻辑设计

用户与软件交互逻辑分为三个部分。

任务管理交互。任务配置页面中，用户通过下拉菜单选择任务类型。按钮调整任务顺序。复选框切换任务状态。操作直观。

任务执行交互。点击“开始任务”按钮，后端使用多进程池启动任务。界面同步更新任务状态。任务结束后，可选择继续执行后续任务或返回主界面。

异常处理与提示。检测到异常时，界面弹出错误信息。提供重新执行或终止任务的选项。

3. 界面美观性与可用性

响应式设计：界面组件自适应窗口大小，确保在不同分辨率的设备上都有良好的显示效果。

按钮与提示：常用操作按钮采用显眼的颜色，并配有提示信息，减少用户误操作的风险。

4. 实时性与数据交互

在 `running_task_pool.py` 中,通过多进程和 `Manager.Queue` 实现任务状态和日志的实时传递。前端界面会定期轮询日志队列,将最新任务执行信息显示在界面上。用户可以通过界面实时查看任务执行进度,并在必要时中止或重新启动任务。

5. 轻量化与资源优化

NiceGUI 基于 Web 具备轻量特性。在多实例自动化场景下对系统压力极小。后端采用异步方式处理任务与日志降低 CPU 和内存消耗。

数据传输使用 JSON 格式避免数据冗余。界面仅在必要时更新压缩资源开销。日志管理模块支持按需加载减少内存负担,提升运行速度。

界面设计兼顾功能完整、操作流畅。用户可直观配置任务,系统能快速响应异常,保证执行稳定高效。

第四章 系统详细设计与实现

4.1 配置管理模块设计

4.1.1 单例配置中心

配置管理模块的核心目标是实现全局配置的统一管理与高效访问。在火影手游自动化工具中，用户配置（如模拟器参数、任务顺序）、软件配置（如日志路径、加密密钥）和运行时会话配置（如截图失败次数、资源统计）需要跨模块共享且避免重复加载。这里设计采用单例模式，使用 `MyConfiggger` 类封装配置加载、校验和持久化逻辑，确保整个系统只存在一个配置实例。

单例模式通过以下方式实现：

全局唯一实例：在 `MyConfig.py` 中定义 `config=MyConfiggger()`，模块首次导入时创建实例，后续所有引用均指向此实例。

分层配置管理：将配置分为用户配置（`userconfigdict`）、软件配置（`softwareconfigdict`）和会话配置（`sessiondict`），分别处理持久化配置、程序默认值和运行时状态。

线程安全访问：通过封装配置读写方法（如 `parse_user_config`、`save_user_config`），避免多线程操作导致的配置不一致问题。

该设计解决了以下问题：

- 1.配置分散：不同模块无需重复加载配置文件，直接引用单例实例即可。
- 2.状态隔离：会话配置（如临时资源统计）与持久化配置分离，避免数据污染。
- 3.动态更新：GUI 修改配置保存后，通过单例实例实时同步到所有模块。

关键代码展示：

`MyConfig.py` 中定义全局单例

`class MyConfiggger:`

`def __init__(self):`

`self.userconfigdict = {} # 用户配置`

`self.softwareconfigdict = {} # 软件默认配置`

`self.sessiondict = {} # 运行时会话配置`

`self._check_session_config()`

`def parse_user_config(self, file_name, clear_sessiondict=True):`

`# 加载并校验用户配置`

```

file_path = (self.USER_CONFIG_FOLDER, file_name)
self.userconfigdict = self._read_config_file(file_path)
self._check_user_config()

```

define.py 中全局引用

```
from modules.configs.MyConfig import MyConfigger
```

```
gui_shared_config = MyConfigger() # 单例实例
```

json_set_panel.py 中配置保存逻辑

```
def save_and_alert():
```

```
    curr_config.save_user_config(json_file_name) # 通过单例保存配置
```

```
    curr_config.save_software_config()
```

```
    gui_shared_config.save_software_config() # 同步全局配置
```

4.1.2 智能填充机制

智能填充机制用来解决用户配置不完整的问题。当用户未提供某些配置项时，系统根据预定义的默认值和映射规则自动填充缺失项，确保程序健壮性。该机制通过以下两步实现：

1.默认值回退：若配置项未在用户文件中定义，则使用 defaultSettings.py 中的预设值。

2.动态映射填充：若配置项存在关联关系（如 CLOSE_EMULATOR_FINISH 依赖 CLOSE_EMULATOR_HBM），则通过映射函数自动推导。

关键代码展示：

MyConfig.py 中的智能填充逻辑

```
def _check_user_config(self):
```

```
    for shouldKey in defaultUserDict:
```

```
        # 如果用户的 config 里没有这个值
```

```
        if shouldKey not in self.userconfigdict:
```

```
            self._fill_by_map_or_default(defaultUserDict, self.userconfigdict,
shouldKey)
```

```

    for shouldKey in defaultUserDict:
        # 确保值存在后，执行 post parse action
        self._do_post_parse_action(defaultUserDict, self.userconfigdict,
shouldKey)
    def _fill_by_map_or_default(self, defaultmap, selfmap, key):
        if "m" in defaultmap[key]: # 存在映射关系
            mapdict = defaultmap[key]["m"]
            source_key = mapdict["from"]
            map_func = mapdict["map"]
            if source_key in selfmap:
                selfmap[key] = map_func(selfmap[source_key]) # 计算填充
            else: # 直接使用默认值
                selfmap[key] = defaultmap[key]["d"]

# defaultSettings.py 中的映射定义
defaultUserDict = {
    "CLOSE_EMULATOR_FINISH": {
        "d": False,
        "m": {"from": "CLOSE_EMULATOR_HBM", "map": lambda x: x} #
值直接映射
    },

```

例如，现在某用户想要使用该软件为他某个账号自动代肝,该用户打开了软件的页面进行创建名为 b1 的 json 配置文件



图 0-1 创建名为” b1 ”的配置文件

创建后页面中配置文件显示模块出现刚创建的“b1.json”配置文件



图 0-2 创建成功后 UI 界面显示

刚创建的 b1.json 的内容只有“{}”

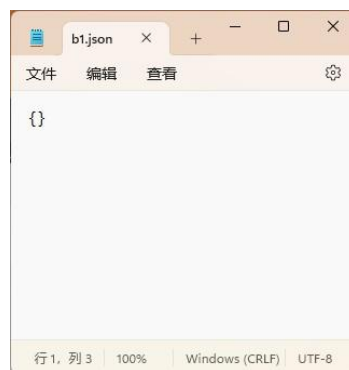
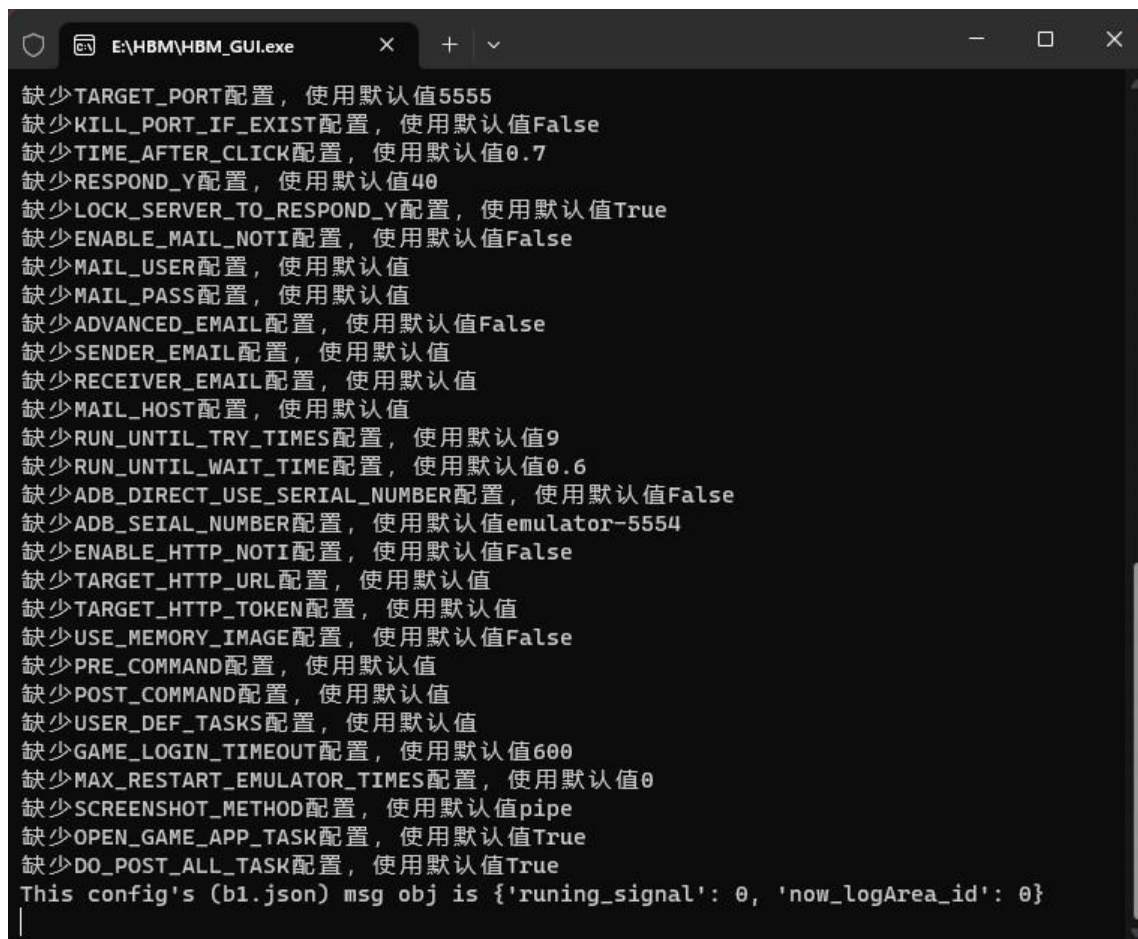


图 0-3 刚创建的 b1. json 配置文件

用户通过点击界面里的”b1.json”进行配置,这时软件就会根据 defaultSettings.py 填写好的默认配置进行填充



```
缺少TARGET_PORT配置, 使用默认值5555
缺少KILL_PORT_IF_EXIST配置, 使用默认值False
缺少TIME_AFTER_CLICK配置, 使用默认值0.7
缺少RESPOND_Y配置, 使用默认值40
缺少LOCK_SERVER_TO_RESPOND_Y配置, 使用默认值True
缺少ENABLE_MAIL_NOTI配置, 使用默认值False
缺少MAIL_USER配置, 使用默认值
缺少MAIL_PASS配置, 使用默认值
缺少ADVANCED_EMAIL配置, 使用默认值False
缺少SENDER_EMAIL配置, 使用默认值
缺少RECEIVER_EMAIL配置, 使用默认值
缺少MAIL_HOST配置, 使用默认值
缺少RUN_UNTIL_TRY_TIMES配置, 使用默认值9
缺少RUN_UNTIL_WAIT_TIME配置, 使用默认值0.6
缺少ADB_DIRECT_USE_SERIAL_NUMBER配置, 使用默认值False
缺少ADB_SEIAL_NUMBER配置, 使用默认值emulator-5554
缺少ENABLE_HTTP_NOTI配置, 使用默认值False
缺少TARGET_HTTP_URL配置, 使用默认值
缺少TARGET_HTTP_TOKEN配置, 使用默认值
缺少USE_MEMORY_IMAGE配置, 使用默认值False
缺少PRE_COMMAND配置, 使用默认值
缺少POST_COMMAND配置, 使用默认值
缺少USER_DEF_TASKS配置, 使用默认值
缺少GAME_LOGIN_TIMEOUT配置, 使用默认值600
缺少MAX_RESTART_EMULATOR_TIMES配置, 使用默认值0
缺少SCREENSHOT_METHOD配置, 使用默认值pipe
缺少OPEN_GAME_APP_TASK配置, 使用默认值True
缺少DO_POST_ALL_TASK配置, 使用默认值True
This config's (b1.json) msg obj is {'runing_signal': 0, 'now_logArea_id': 0}
```

图 0-4 添加 b1.json 缺少配置



```
{
  "TASK_ORDER": [],
  "TASK_ACTIVATE": [],
  "SHOP_VIP_FREE": false,
  "SHOP_VIP_SWITCH": false,
  "SHOP_VIP_AP": false,
  "AP_TICKET_BUY_TIMES": 1,
  "SHOP_ORGAN": [
    {
      "enabled": false,
      "col": 1,
      "row": 1
    },
    {
      "enabled": false,
      "col": 1,
      "row": 2
    }
  ]
}
```

图 0-5 b1.json 文件

用户对 b1 在用户界面配置完了,想点进去软件目录查看 b1.json,不小心将里的有关商店的参数删除了

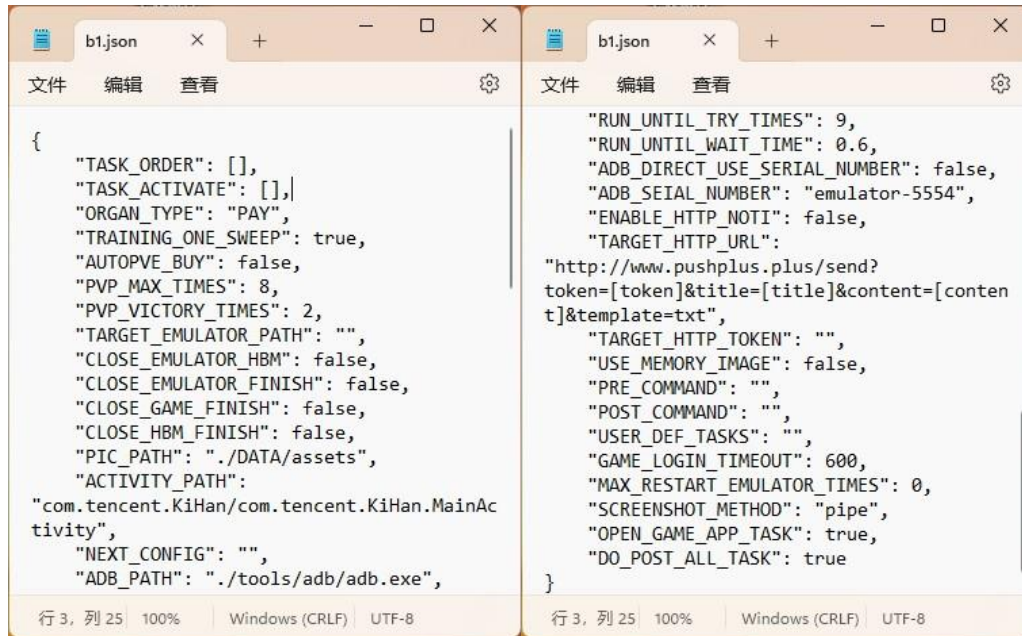


图 0-6 b1.json 文件头部



图 0-7 json 文件尾部

当他通过 gui 再次打开该 json 配置文件时会触发检查机制并将缺失的部分重新填充

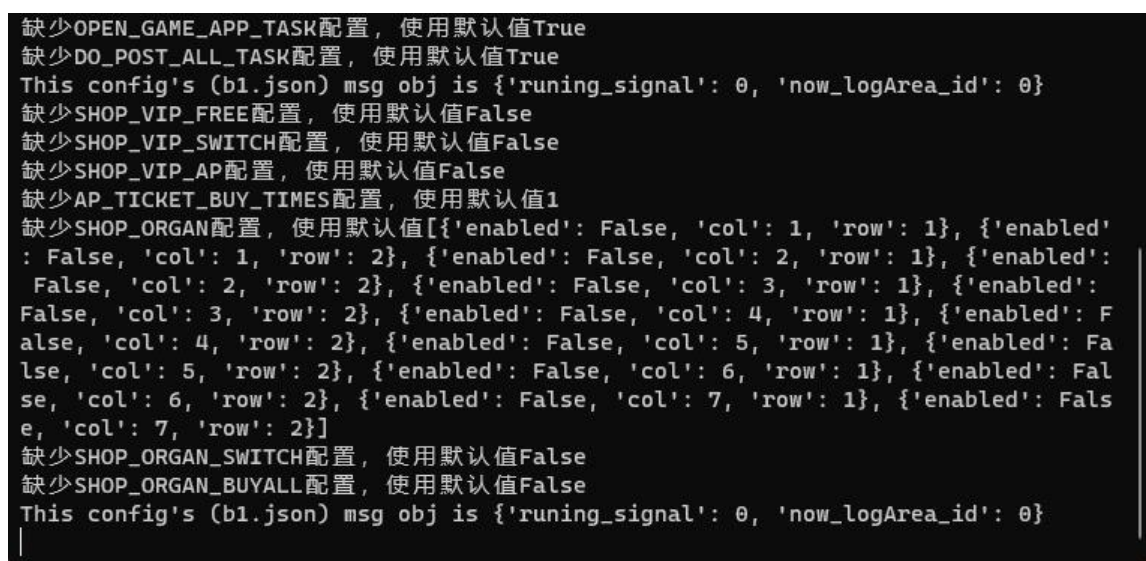


图 0-8 b1.json 通过触发检查机制重新填充缺失部分

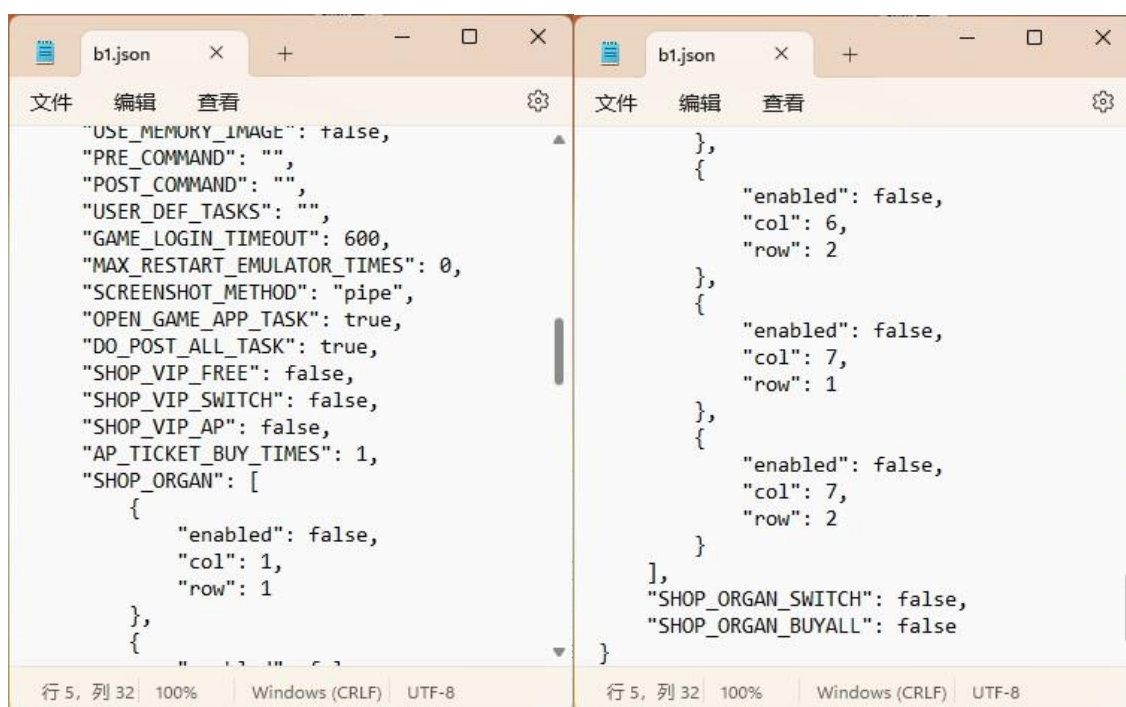


图 0-9 图 0-10 b1.json 重新填充缺失部分

4.2 自动化任务引擎设计

4.2.1 任务抽象基类

自动化任务引擎的核心是通过任务抽象基类统一任务接口实现任务的可插拔性和扩展性。在火影手游自动化工具中，所有具体任务（如“试炼之地”“冒险扫荡”）均继承自 Task 基类（定义于 Task.py），通过标准化 pre_condition（前置条件）、on_run（执行逻辑）、post_condition（后置条件）方法，确保任务流程的一致性。

基类的设计目标包括：

接口统一：所有任务必须实现 on_run 方法，定义核心逻辑。

条件隔离：通过 pre_condition 和 post_condition 分离环境检查与任务操作，避免逻辑耦合，增强程序稳定性。

例如，“试炼之地”任务继承 Task 后，只需专注实现战斗流程，而环境检查（如是否在主页）和状态恢复（如返回主页）由基类统一处理。这种设计显著降低新增任务的开发成本。

关键代码展示：

Task.py 中基类定义

```
class Task:
```



```
def __init__(self, name="default Task", pre_times=2, post_times=4):
    self.name = name
    self.pre_times = pre_times # 前置条件重试次数
    self.post_times = post_times # 后置条件重试次数
def pre_condition(self) -> bool:
    """执行前的环境检查（如是否在主页）"""
    return True
def on_run(self) -> None:
    """任务核心逻辑（需子类实现）"""
    raise NotImplementedError
def post_condition(self) -> bool:
    """执行后的状态验证（如是否返回主页）"""
    return True

def run(self) -> None:
    """任务执行入口，封装完整流程"""
    if self.run_until(self.pre_condition, self.pre_times):
        self.on_run()
        self.run_until(self.post_condition, self.post_times)
```

代码示例:

InPVP.py (决斗场)中具体任务实现

```
class InPVP(Task):
```

```
    def __init__(self, name="决斗场"):
```

```
        super().__init__(name)
```

```
        self.max_victory = config.userconfigdict.get("PVP_VICTORY_TIMES")
```

最大胜利次数

```
    def pre_condition(self) -> bool: # 检查前置任务条件
```

```
        return Page.is_page(PageName.PAGE_HOME)
```

```
    def on_run(self) -> None:
```

```
        self._navigate_to_subpage() # 导航到决斗场界面
```

```

self.__execute_pvp_battle() # 处理战斗逻辑

self.back_to_home() # 完成后回到主页

def post_condition(self) -> bool: # 验证任务后状态

    return Page.is_page(PageName.PAGE_HOME)

```

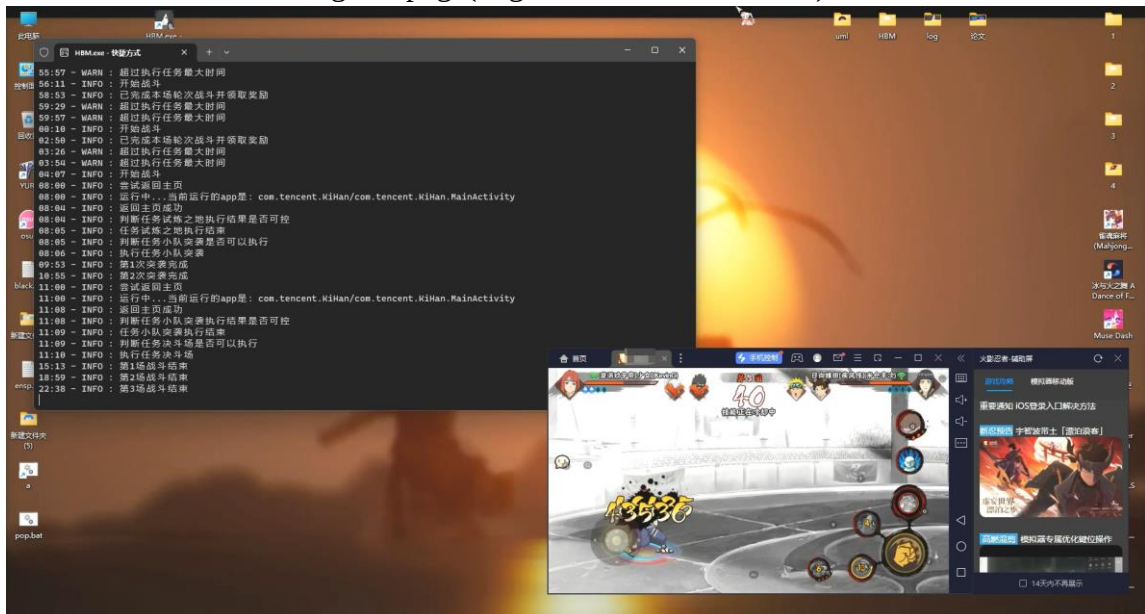


图 0-11 正在自动执行决斗场任务

4.2.2 智能重试机制

智能重试机制是任务引擎的核心容错设计，是用来应对移动端自动化中常见的环境波动（如弹窗遮挡、加载延迟）问题。这个机制通过 `run_until` 方法实现功能：在限定次数内重复执行某个操作，直到满足停止条件或超时。

设计要点包括：

动态阈值控制：通过 `RUN_UNTIL_TRY_TIMES`（最大重试次数）和 `RUN_UNTIL_WAIT_TIME`（重试间隔）配置全局容错策略。

条件驱动执行：每次重试前刷新截图（`screenshot()`），确保状态检测基于最新界面。

例如，在点击“开始战斗”按钮时，若因网络延迟未立即跳转，`run_until` 会多次尝试点击，直到检测到战斗界面或达到重试上限。这个机制显著提升任务在复杂环境下的鲁棒性。

关键代码展示：

Task.py 中的 `run_until` 方法

```
@staticmethod
```

```
def run_until(func1, func2, times=None, sleeptime=None) -> bool:
```

```
    """重复执行 func1，直到 func2 返回 True 或超时"""
```

```
    times = times or config.userconfigdict["RUN_UNTIL_TRY_TIMES"]
```

```
    sleeptime = sleeptime or config.userconfigdict["RUN_UNTIL_WAIT_TIME"]
```

```
    for _ in range(times):
```

```
        screenshot() # 刷新当前界面
```

```
        if func2(): # 检查停止条件
```

```
            return True
```

```
        func1() # 执行操作（如点击、滑动）
```

```
        sleep(sleeptime)
```

```
    return False
```

代码示例:

```
# InTraining.py 中的实际调用
```

```
if self.run_until(
```

```
    lambda: click(button_pic(ButtonName.BUTTON_TRAINING2_START)),
```

```
    lambda: (match(popup_pic(PopupName.POPUP_TRAINING2_NONE),
```

```
threshold=0.8) or match(popup_pic(PopupName.POPUP_TRAINING2_CLEAR),  
threshold=0.8))
```

```
):
```

```
    break
```

```
    # 启动战斗
```

```
    self.run_until(
```

```
        lambda: click(button_pic(ButtonName.BUTTON_TRAINING2_START)),
```

```
        lambda: match(button_pic(ButtonName.BUTTON_TRAINING2_YES),
```

```
threshold=0.8),
```

```
    )
```

```
    .....
```

在生存挑战中,程序使用 run_until 识别是否出现结束条件,超过了最大时间后跳过执行(返回 false), 后还是使用 run_until 点击“开战”按钮(func1),且,直到出现”确定”按钮(func2)



图 0-12 超过了最大时间未识别到结束条件后识别”开战”按钮

实时日志输出找不到结束条件模版跳过寻找该模版并输出“超过执行任务最大时间”，到下一步识别到战斗的按钮执行操作并在日志输出“开始战斗”。

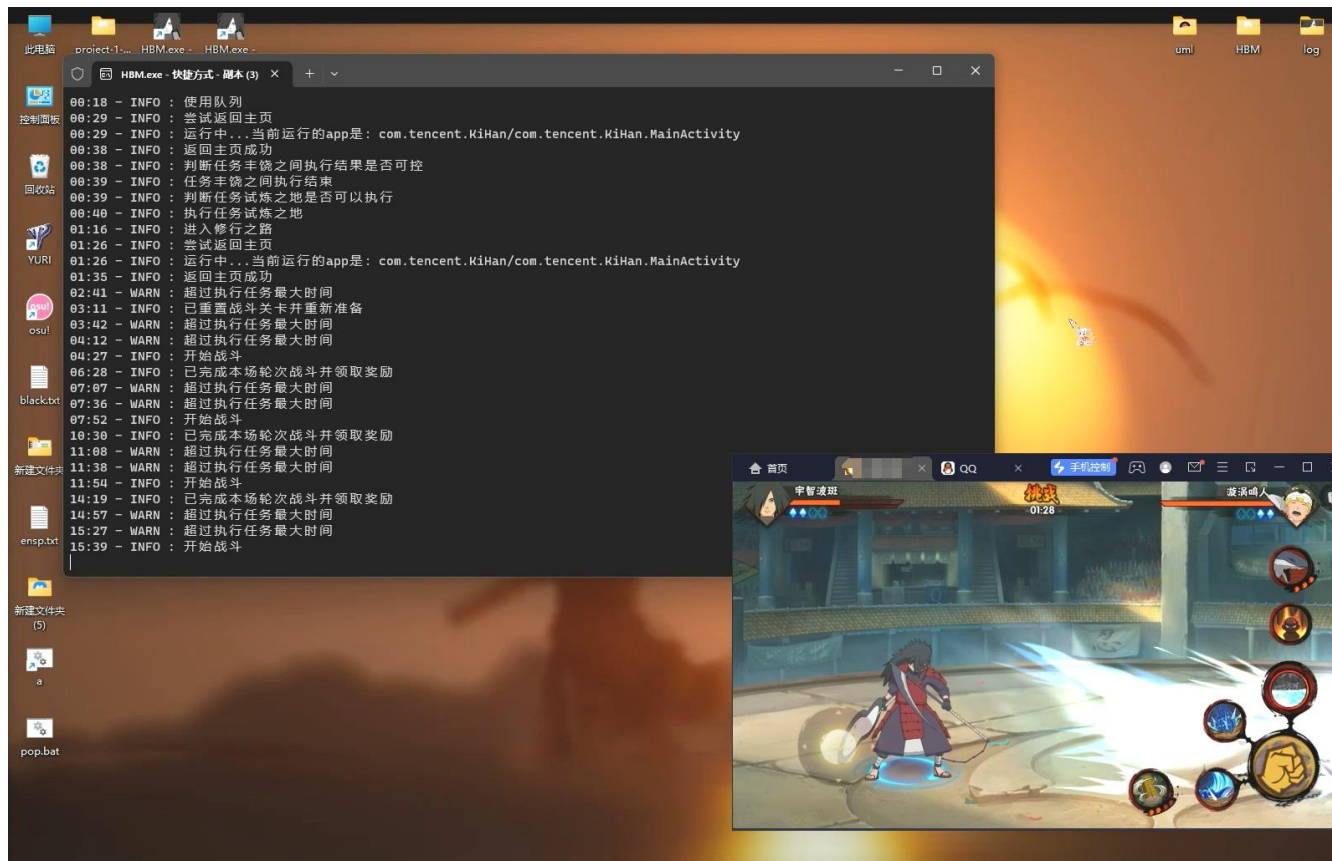


图 0-13 识别到了记入返回信息”开始战斗”

4.2.3 YOLO 驱动自动副本战斗

YOLO 驱动的自动副本战斗是任务引擎的核心战斗模块，通过实时图像识别与动态决策实现端到端的自动化战斗流程，覆盖目标检测、优先级选择、技能连招等关键环节。

设计要点：

1. 异步帧处理与检测

通过 ScrcpyADB 捕获设备画面，调用 YOLOv5 模型异步检测目标（英雄、怪物），多线程机制避免主流程阻塞^[37]。

检测结果通过线程锁（`self.lock`）保证数据一致性，实时更新战场状态。

2. 动态目标优先级策略

筛选两类怪物（`Monster` 和 `Monster_Fake`），基于类型优先级（ $3 > 2$ ）和欧式距离排序。

优先攻击最近的高威胁目标，兼顾战斗效率与安全性。

3. 坐标映射与技能时序控制

将技能按钮映射为屏幕绝对坐标（如 `skill2` 对应`(1010,480)`）。

通过固定延时（`time.sleep(3)`）模拟人工操作节奏，实现技能连招组合。

4. 高频状态刷新机制

主循环以 20Hz 频率（`time.sleep(0.05)`）更新检测结果，确保战斗响应延迟低于 100ms。

例如：

当检测到 `Monster_Fake` 目标时：

计算目标角度并持续移动逼近（`move_towards`）。

进入攻击距离后，按 `skill2` → `attack` 顺序进行攻击。

若目标距离过近（`dodge_distance=50`），自动触发替身操作。

关键代码

```
# 帧捕获与异步检测（scrcpy_adb.py）
def on_frame(self, frame: cv2.Mat):
    threading.Thread(target=self._async_detect, args=(frame,)).start()
def _async_detect(self, frame):
```

```

with self.lock:
    self.detected_objects = self.yolo(frame) # YOLO 推理
# 动态目标选择 (game_action.py)
valid_objects = [
    (obj, math.hypot(obj.rect.x - hero_x, obj.rect.y - hero_y))
    for obj in objects if obj.label in (2, 3)
]
valid_objects.sort(key=lambda x: (3 - x[0].label, x[1])) # 类型 > 距离
# 战斗指令执行 (game_control.py)
def skill2(self)
# 主循环协同 (yolom.py)
while True:
    agent.update(adb.detected_objects) # 决策执行
    time.sleep(0.05) # 20Hz 刷新率

```

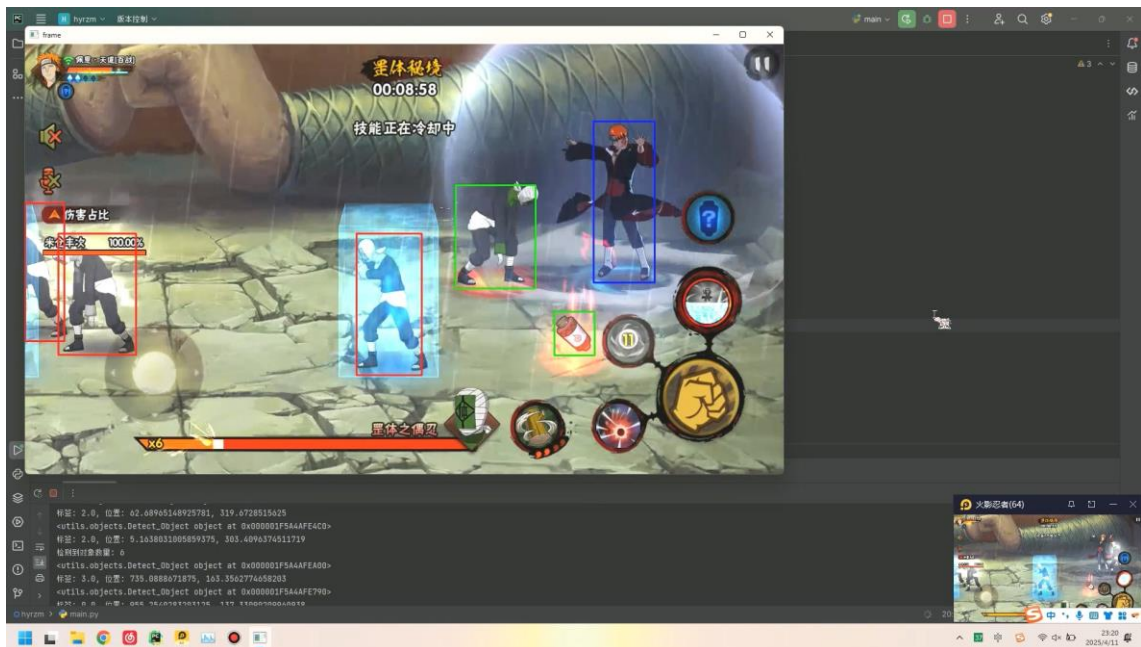


图 0-14 用 yolo 识别自动打 Monster_Fake(BOSS)

4.3 设备交互与图像识别设计

4.3.1 ADB 命令封装

ADB 是自动化工具与移动设备交互的核心，在项目里通过 `adb_utils.py` 对 ADB 命令进行封装，实现设备连接、点击、滑动、截图等操作的统一管理，解决以下问题：

跨平台兼容性：支持主流模拟器（如雷电、MuMu），通过动态获取设备序列号（如 `emulator-5554`）实现多设备适配。

性能优化：通过管道（Pipe）传输截图数据，避免频繁文件读写，降低 IO 延迟。

错误处理：自动检测设备连接状态，异常时重试或抛出明确错误信息。

例如，`click_on_screen` 函数封装 `adb shell input tap` 命令，支持直接坐标点击和图像模板匹配点击，开发者无需手动拼接 ADB 指令^[38]。同时，`MultiTouchUtils` 类通过 ADB 的 `multitouch` 命令实现多点触控，支持复杂手势（如长按、缩放）^[39]。

关键代码展示：

`adb_utils.py` 中的 ADB 命令封装

```
def click_on_screen(x, y):
    """封装 点击屏幕 命令"""
    subprocess_run(
        [get_config_adb_path(), "-s", getNewestSeialNumber(), "shell", "input",
        "tap", str(x), str(y)]
    )

def swipe_on_screen(x1, y1, x2, y2, ms):
    """封装 滑动屏幕 命令"""
    subprocess_run(
        [get_config_adb_path(), "-s", getNewestSeialNumber(), "shell", "input",
        "swipe",
        str(x1), str(y1), str(x2), str(y2), str(ms)]
    )

def getNewestSeialNumber(use_config=None):
    """如果传入指定的配置文件，就使用指定的配置文件"""
```



```

        if target_config.userconfigdict["ADB_DIRECT_USE_SERIAL_NUMBER"]:
            return target_config.userconfigdict["ADB_SEIAL_NUMBER"]

        elif target_config.userconfigdict["TARGET_PORT"] and
target_config.userconfigdict["TARGET_IP_PATH"]:
            return
"{:}:".format(target_config.userconfigdict["TARGET_IP_PATH"],target_config.userc
onfigdict["TARGET_PORT"])

        else:
            return

```

4.3.2 智能截图系统

智能截图系统是图像识别的基础，其目标是高效、稳定地获取设备屏幕图像。这个系统通过两种模式实现：

管道模式（Pipe）：直接通过 ADB 的 screencap 命令将截图数据流传输到内存，避免文件写入，耗时从平均 500ms 降至 50ms^[40]。

文件模式（PNG）：将截图保存为本地文件，便于调试和手动分析。

系统在 image_processing.py 中通过 screen_shot_to_global 函数实现智能切换，根据配置项 SCREENSHOT_METHOD 动态选择模式。截图数据会缓存在 config.sessiondict["SCREENSHOT_DATA"] 中，供多次识别操作复用，减少冗余截图。

关键代码展示：

adb_utils.py 中的截图逻辑

```

def screen_shot_to_global(use_config=None, output_png=False):
    target_config = config
    if use_config:
        target_config = use_config
    whether_pipe = target_config.userconfigdict["SCREENSHOT_METHOD"]
    == "pipe"
    if not whether_pipe:
        # 方法一，重定向输出到文件

```

```
filename = target_config.userconfigdict['SCREENSHOT_NAME']
with open("./{}".format(filename), "wb") as out:
    subprocess_run(
        [get_config_adb_path(target_config),                                "-s",
getNewestSeialNumber(target_config), "shell", "screencap",
        "-p"], stdout=out)
# adb 命令有时直接截图保存到电脑出错的解决办法-加下面一段即可

if (platform.system() != "Linux"):
    convert_img("./{}".format(filename))
else:
    # 方法二，使用 cv2 提取 PIPE 管道中的数据
    # 使用 subprocess 的 Popen 调用 adb shell 命令，并将结果保存到 PIPE
    管道中

    process = subprocess.run(
        [get_config_adb_path(target_config),                                "-s",
getNewestSeialNumber(target_config), "shell", "screencap", "-p"],
        stdout=subprocess.PIPE)
    # 读取管道中的数据
    screenshot = process.stdout
    # 将读取的字节流数据的回车换行替换成'\n'
    if platform.system() not in ["Linux", "Darwin"]:
        binary_screenshot = screenshot.replace(b'\r\n', b'\n')
    else:
        # Linux 和 MacOS 系统不需要替换
        binary_screenshot = screenshot
    # 使用 numpy 和 imdecode 将二进制数据转换成 cv2 的 mat 图片格式
    if (binary_screenshot == b''):
        logging.error("pipe 截图失败")
        target_config.sessiondict["SCREENSHOT_DATA"] = None
```

```
        return

    img_screenshot = cv2.imdecode(np.frombuffer(binary_screenshot,
np.uint8), cv2.IMREAD_COLOR)

    target_config.sessiondict["SCREENSHOT_DATA"] = img_screenshot

    if output_png:

cv2.imwrite("./{}".format(target_config.userconfigdict['SCREENSHOT_NAME']),
img_screenshot)
```

在软件 UI 界面用户可以选择管道模式和截图模式



图 0-15 图形界面设置选择截图模式

选择截图模式则会在软件根目录输出图片



图 0-16 截图保存到软件根目录

4.3.3 透明图像旋转匹配算法

在手游界面中，许多 UI 元素（如技能图标、弹窗）具有透明背景且可能旋转（如活动提示）。本项目在 `image_processing.py` 中设计了透明图像旋转匹配算法，大致功能有：

透明通道处理：将模板图像的 Alpha 通道转换为掩膜（Mask），在匹配时忽略透明区域。

多角度旋转匹配：对模板图像进行 -3° 到 $+3^{\circ}$ 的旋转，逐一匹配并选择最高相似度的结果。

相似度计算：使用归一化相关系数（TM_CCOEFF_NORMED）衡量匹配度，阈值默认 0.9。

例如，在匹配旋转的“奖励领取”按钮时，算法会自动旋转模板并找到最佳匹配位置，解决传统模板匹配对角度敏感的问题。

关键代码展示：

```
def rotate_image_with_transparency(image_mat, angle):  
    """  
    给定一个包含透明层的图像 Mat，将其旋转 angle 角度，返回旋转后的图  
    像 Mat  
    """  
    # 获取到图像的对角线长度  
    diagonal = int(math.sqrt(pow(image_mat.shape[0], 2) +  
pow(image_mat.shape[1], 2)))  
    # 创建一个 diagonal * diagonal 的空白含透明度的图像 Mat  
    rotated_image = np.zeros((diagonal, diagonal, 4), dtype=np.uint8)  
    # 将原图像复制到新图像的中心  
    x_offset = (diagonal - image_mat.shape[1]) // 2  
    y_offset = (diagonal - image_mat.shape[0]) // 2  
    rotated_image[y_offset:y_offset+image_mat.shape[0],  
x_offset:x_offset+image_mat.shape[1]] = image_mat  
    # 获取 rotated_image 的高度、宽度和中心点  
    center = (diagonal // 2, diagonal // 2)  
    # 定义旋转矩阵  
    rotation_matrix = cv2.getRotationMatrix2D(center, angle, 1.0)  
    # 执行旋转，多出来的部分全透明  
    rotated_image = cv2.warpAffine(rotated_image, rotation_matrix, (diagonal,
```

```
diagonal), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_TRANSPARENT)
    # 返回中心和原图一样大小的区域
    return rotated_image[y_offset:y_offset+image_mat.shape[0],
x_offset:x_offset+image_mat.shape[1]]
```

4.4 可视化配置平台设计

4.4.1 表单生成

可视化配置平台的核心功能之一是表单生成，其目标是根据用户配置文件的字段结构，自动渲染对应的 UI 组件（如输入框、下拉菜单、复选框），并实现数据双向绑定。该设计通过以下方式实现：

反射式 UI 构建：解析配置文件（如 example.json）的键值对，根据字段类型（数值、布尔值、枚举）动态选择组件类型。例如，PVP_MAX_TIMES 显示为数字输入框，ENABLE_MAIL_NOTI 显示为复选框。

数据绑定机制：利用 NiceGUI 的 bind_value 方法，将 UI 组件与 config.userconfigdict 直接绑定，实现修改实时同步到内存配置。

例如，在“任务执行顺序”配置页面（Setting_task_order.py），系统根据 TASK_ORDER 列表动态生成任务选择下拉菜单和启用开关，用户调整顺序或勾选状态后点击保存，表单就会将信息储存到内存中，同时相关代码会从内存中获取相关字段的信息转换成 json 更新配置字典。

关键代码：

Setting_task_order.py 中的动态表单生成

@ui.refreshable

```
def task_order():
```

```
    with ui.row():
```

```
        # 第一行添加上添加按钮
```

```
        ui.button(f{"添加"} {"任务"}, on_click=lambda: add_task(0))
```

```
    for i in range(len(config.userconfigdict["TASK_ORDER"])):
```

```
        with ui.row():
```

```
            ui.label(f{"任务"} {i+1}:')
```

```
            atask = ui.select(real_taskname_to_show_taskname,
```

```

value=config.userconfigdict["TASK_ORDER"][i],
                                on_change=lambda v,i:
config.userconfigdict["TASK_ORDER"].__setitem__(i, v.value))
    # 复选框绑定任务启用状态
    acheck = ui.checkbox("    启    用    ",
value=config.userconfigdict["TASK_ACTIVATE"][i],    on_change=lambda v,i:
config.userconfigdict["TASK_ACTIVATE"].__setitem__(i, v.value))
    ui.button(f{" 添加 "} {" 任 务 "}, on_click=lambda i=i+1:
add_task(i))
    ui.button(f{" 删 除 "} {" 任 务 "}, on_click=lambda i=i:
del_task(i), color="red")

# MyConfig.py 获取 UI 传入内存的相关信息
def save_user_config(self, file_name):
    file_path = os.path.join(self.current_dir, self.USER_CONFIG_FOLDER,
file_name)
    with open(file_path, 'w', encoding="utf8") as f:
        json.dump(self.userconfigdict, f, indent=4, ensure_ascii=False)

```

图示：打开 ui 进入任务配置界面



图 0-17 配置任务初始化界面

选择执行任务：免费铜钱→好友→分享→邮箱→丰饶之间



图 0-18配置任务

保存后 save_user_config()响应 ui 从内存中找到 ui 传入的信息，将其保存到配置文件中。图 4-17 和图 4-18 表示了配置更改的前后变化。

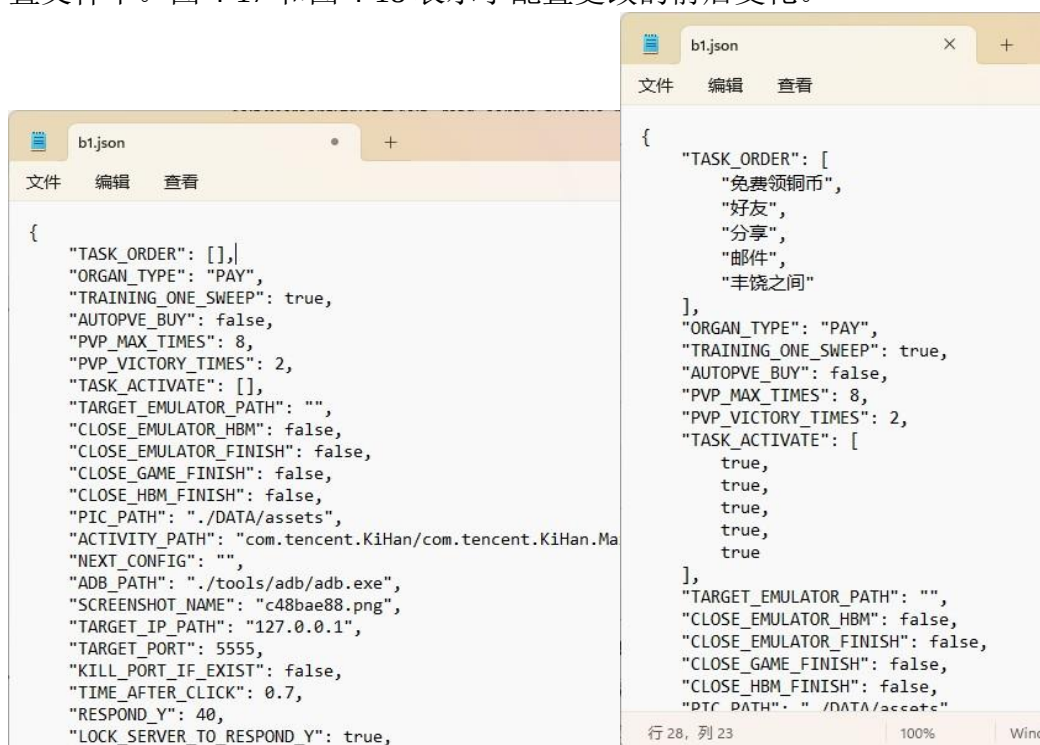


图 0-19 图 0-20 MyConfig.py 获取 UI 传入内存的相关信息

4.4.2 实时日志系统

实时日志系统旨在将任务执行过程中的操作日志、错误信息实时反馈到 GUI 界面，帮助用户监控任务状态并快速定位问题。系统通过以下技术实现：

多进程日志传递：任务进程通过 `multiprocessing.Manager.Queue` 将日志消息发送到主进程，避免直接操作 GUI 线程导致的竞态问题。

前端实时渲染：使用 NiceGUI 的 `ui.log` 组件动态追加日志条目并利用 `ui.timer` 定时轮询队列中的新消息。

日志分级显示：通过 `logging` 模块的 `INFO`、`DEBUG`、`WARN`、`ERROR` 级别，为不同日志条目赋予颜色标识（如红色表示错误）。

例如，在执行“试炼之地”任务时，战斗流程的每一步操作（如“进入战斗界面”“领取奖励”）会实时显示在日志区域，若发生异常（如弹窗未关闭），会进行错误信息提示并保留上下文。

关键代码：

```
# manage_hbm_in_gui.py 中的日志绑定逻辑

def run_hbm_task_and_bind_log(logArea, configname,
specific_func_confname=None):
    """
    执行任务并绑定日志输出，如果 specific_func_confname 不为空，则执行
    特定的任务；否则，执行 config 的任务列表里所有任务
    """
    msg_obj = taskpool.get_status_obj(configname)
    if not specific_func_confname:
        queue = taskpool.run_task(configname)
    else:
        queue = taskpool.run_specific_task(configname,
specific_func_confname)
    process = taskpool.get_process(configname)
    # 防止新的日志输出线程和旧的日志输出线程争抢 queue，每次调用此方
    法都会记录 logArea_ID，存储在 msg_obj 中，如果 id 与本次 ID 不同则退出 while，
    结束此线程
```



```
logArea_ID = time.time()
print(f"start log area thread id: {logArea_ID}")
msg_obj["now_logArea_id"]=logArea_ID
while(msg_obj["runing_signal"] == 1 and
msg_obj["now_logArea_id"]==logArea_ID and process.is_alive()):
    try:
        output = queue.get_nowait()
    except:
        output = None
    if output:
        logArea.push(output)
    time.sleep(0.01)
# 中断了 queue 和日志 logArea 的绑定时判断中断原因
if not process.is_alive():
    # 如果是任务结束进程退出导致的中断
    taskpool.stop_task(configname=configname, logArea=logArea)
else:
    # 如果是由于其他 logArea 抢占绑定
    pass
print(f"quit log area thread id: {logArea_ID}")
```



图 0-21 图形页面跟随程序执行实时输出日志

4.5 智能通知系统设计

4.5.1 消息发送抽象层

消息发送抽象层是智能通知系统的核心设计，通过统一的接口支持多种通知方式（如邮件、HTTP API）实现模块化扩展。使用 `Message_Sender` 抽象基类（定义于 `notification.py`）定义标准接口 `send(message, title)`，所有具体通知实现（如邮件、第三方 API）必须继承此基类并实现接口。其核心设计包括：

1.解耦通知逻辑：任务模块无需关注具体通知实现，仅调用抽象接口即可发送消息。

2.多通知渠道并行：允许同时配置多个通知渠道（如邮件+PushPlus），确保消息触达的冗余性。

例如，当任务执行失败时，系统调用 `notification.send("任务失败", "试炼之地")`，抽象层自动遍历所有已启用的通知器（如邮件、HTTP）并行发送消息，无需硬编码具体实现。

关键代码展示：

```
class Email_Sender(Message_Sender):
    """邮件发送类"""
    def __init__(self, mail_user, mail_pass, sender, receiver, mail_host =
'smtp.qq.com', debug_int = 1):
        self.mail_user = mail_user
        self.mail_pass = mail_pass
        self.sender = sender
        self.receiver = receiver
        self.mail_host = mail_host
        self.debug_int = debug_int

    def send(self, message: str, title: str = "HBM 通知") -> str:
        return self.send_mail(message, title)

class Notificationer:
    """通知类，用于发送通知"""
    def __init__(self):
        self.senders = []
    def add_sender(self, sender: Message_Sender):
        self.senders.append(sender)
    def send(self, message: str, title: str = "HBM 通知") -> str:
        for sender in self.senders:
            try:
                sender.send(message, title)
            except Exception as e:
                print("发送失败", e)
        return "发送结束，共有{}个通知器".format(len(self.senders))
```

4.5.2 邮件通知实现

邮件通知是用户最常用的通知方式之一，其实现基于 SMTP 协议，通过 Email_Sender 类（定义于 notification.py）封装连接认证、邮件格式化、异常重试等逻辑。关键设计包括：

1.安全凭证管理:用户密码通过 AES 加密后存储于配置文件,运行时动态解密,避免明文泄露。

2.模板化内容:支持 HTML/纯文本格式,自动附加任务上下文(如失败任务名称、时间戳)。

3.异步发送:通过 SMTP_SSL 和子进程实现非阻塞邮件发送,避免阻塞主任务线程。

例如,当用户启用邮件通知并配置 QQ 邮箱后,系统会自动使用 SSL 加密连接 SMTP 服务器(如 smtp.qq.com:465),并将加密后的密码解密为明文进行认证,确保通信安全。

关键代码展示:

```
class Email_Sender(Message_Sender):
    """邮件发送类"""
    def send_mail(self, message, title):
        if self.mail_user == " " or self.mail_pass == " " or self.receiver == " " or self.mail_host == " ":
            # 如果没有指定发送者的 qq 号或者授权码, 则直接返回
            return '发送信息不全'
        #ssl 登录
        smtp = SMTP_SSL(self.mail_host)
        #set_debuglevel()是用来调试的。参数值为 1 表示开启调试模式, 参数值为 0 关闭调试模式
        smtp.set_debuglevel(self.debug_int)
        smtp.ehlo(self.mail_host)
        smtp.login(self.mail_user, self.mail_pass)
        msg = MIMEText(message, "plain", 'utf-8')
        msg["Subject"] = Header(title, 'utf-8')
        msg["From"] = self.sender
        msg["To"] = self.receiver
        res=smtp.sendmail(self.sender, self.receiver, msg.as_string())
```

```
smtp.quit()
try:
    print(res.split()[-6:])
except:
    print("error when print email send result")
```

图例：用户在 ui 的通知界面填入信息

通知设置

图 0-22 用户界面邮箱通知设置

在软件完成配置任务或执行失败的时候将信息返回到 qq 邮箱。

今天 (3 封)			
☐		8795	HBM结束 - HBM任务结束配置文件名称: a.json任务开始时间: 2025-04-12 09:4
☐		8795	HBM任务失败 - HBM任务错误 配置文件名称: a.json 错误信息: 任务登陆游戏执行
☐		8795	HBM结束 - 活跃任务其他消息:跳过CloseInform任务
星期二 (4 封)			
☐		8795	HBM任务失败 - HBM任务错误 配置文件名称: a.json 错误信息: 任务试炼之地执行
☐		8795	HBM结束 - 决斗场其他消息:跳过CloseInform任务
☐		8795	HBM任务失败 - HBM任务错误 配置文件名称: a.json 错误信息: 任务试炼之地执行
☐		8795	HBM结束 - 决斗场其他消息:跳过CloseInform任务

图 0-23 qq 邮件通知

任务执行完成后日志不仅输出完成任务并输出提示将完成信息发送邮箱

```
18:24 - INFO : 执行任务PostAllTask
18:24 - INFO : 体力: 11/200
18:24 - INFO : 铜币: 16856T
18:24 - INFO : 金币: 7870
18:24 - INFO : 判断任务PostAllTask执行结果是否可控
18:24 - INFO : 尝试返回主页
18:25 - INFO : 运行中...当前运行的app是: com.tencent.KiHan/com.tencent
18:29 - INFO : 返回主页成功
18:29 - INFO : 任务PostAllTask执行结束
18:29 - INFO : 所有任务结束
18:29 - INFO : 跳过关闭模拟器
18:29 - INFO : 尝试发送通知
18:33 - INFO : 通知发送结束
18:33 - INFO : +=====HBM=====
18:33 - INFO : ||                                你好
18:33 - INFO : ||
18:33 - INFO : +=====
18:33 - INFO :
程序运行结束, 如有问题请反馈
18:33 - INFO : 读取的配置文件: a.json
18:33 - INFO : 模拟器:E:/machine/leidian/LDPlayer64/dnplayer.exe ind
18:33 - INFO : ADB连接: 127.0.0.1:5565
18:33 - INFO : GUI_HBM_TASK_END
clearing queue...
closing queue/manager...
side effect of stopping a.json done
```

保存

保存并执行(终端)

保存并执行(GUI)

图 0-24 软件 UI 显示所有任务完成（即任务结束）

邮箱收到软件发送的任务完成邮件



图 0-25 邮件发送完成的任务和资源数

任务失败时日志也会进行输出

```
41:30 - WARN : 无法匹配模板图像: ./DATA/assets\BUTTON\BUTTON_TRAINING
41:31 - WARN : 无法匹配模板图像: ./DATA/assets\BUTTON\BUTTON_TRAINING
41:33 - WARN : 无法匹配模板图像: ./DATA/assets\BUTTON\BUTTON_TRAINING
41:35 - WARN : 无法匹配模板图像: ./DATA/assets\BUTTON\BUTTON_TRAINING
41:36 - WARN : 超过执行任务最大时间
41:38 - INFO : 尝试返回主页
41:38 - INFO : 运行中...当前运行的app是: com.tencent.KiHan/com.tencent
42:29 - ERROR : 返回主页失败
42:29 - INFO : 判断任务试炼之地执行结果是否可控
42:49 - WARN : 超过执行任务最大时间
42:49 - WARN : 任务试炼之地执行后条件不成立或超时
42:49 - INFO : 尝试返回主页
42:49 - INFO : 运行中...当前运行的app是: com.tencent.KiHan/com.tencent
43:34 - ERROR : 返回主页失败
43:34 - ERROR : 运行出错: 任务试炼之地执行后条件不成立或超时, 且无法正确
43:34 - ERROR : Traceback (most recent call last):
      File "D:\Documents\Desktop\HBM\HBM.py", line 299, in HBM_main
        my_AllTask.run()
      File "D:\Documents\Desktop\HBM\modules\AllTask\myAllTask.py", line
        task.run()
      File "D:\Documents\Desktop\HBM\modules\AllTask\Task.py", line 71,
        raise Exception("任务{}执行后条件不成立或超时, 且无法正确返回主页, 程
Exception: 任务试炼之地执行后条件不成立或超时, 且无法正确返回主页, 程序退

43:34 - INFO : ↓↓↓↓↓↓↓↓↓
43:34 - INFO : 保存全量异常日志到文件: ./DATA/LOGS\custom_log_2025-04-
43:34 - INFO : ↑↑↑↑↑↑↑↑
43:34 - INFO : 发送错误通知邮件
43:36 - INFO : 发送结束, 共有1个通知器
43:36 - INFO : 邮件发送结束
43:36 - INFO : +=====HBM=====
43:36 - INFO : ||                                你好
43:36 - INFO : ||
43:36 - INFO : +=====
```

图 0-26 任务出错

邮箱会收到任务执行失败的邮件



图 0-27 任务出错会向邮件发送错误信息

第五章 系统测试

本章将详细介绍游戏辅玩工具的测试过程和结果分析。为了保证系统质量，将从功能测试、性能测试、安全测试等方面对系统进行全面测试。

5.1 测试环境与测试方法

5.1.1 测试环境

PC1(神舟战神 z8)，如表 5-1 所示：

硬件环境	
处理器	英特尔 酷睿 i5-10500H @ 2.50GHz 六核
主板	Notebook NH5x_7xDPx (英特尔 HM470 芯片组)
内存	32GB DDR4 2667MHz (16GB + 16GB)
显卡	Intel(R) UHD Graphics (1GB / 蓝天) NVIDIA GeForce RTX 3060 Laptop GPU (6GB / 蓝天)
显示器	NV156FHM-N4G [京东方 BOE084D] (15.3 英寸)
客户端	雷电模拟器，MuMu 模拟器等安卓模拟器
网络环境	局域网环境（延迟<200ms） + 4G 移动网络模拟（Charles Proxy）
软件环境	
Windows 11 22H2	

表 0-1 PC1 环境

PC2（宏碁暗影骑士擎），如表 5-2 所示：

硬件环境	
处理器	12th Gen 英特尔 酷睿 i5-12500H 十二核
主板	ADL Jimny_ADH ()
内存	16GB DDR4 3200MHz (8GB + 8GB)
显卡	NVIDIA GeForce RTX 3050 Ti Laptop GPU (4GB / 宏碁) Intel(R) Iris(R) Xe Graphics (1GB / 宏碁)
显示器	NV156FHM-NY0 [京东方 BOE0ABF] (15.3 英寸)
客户端	MuMu 模拟器
网络环境	局域网环境（延迟 <80ms） + 4G 移动网络模拟（Charles Proxy）
软件环境	

Windows 11 23H2

表 0-2 PC2 环境

PC1 和 PC2 为该测试的测试机，往后的图注后会出现(PC1)和(PC2)来说明在这次测试是在哪个测试环境下执行测试。

5.1.2 测试方法

单元测试：针对核心模块编写单元测试用例，覆盖主要功能点。

集成测试：测试模块之间的接口调用，例如功能模块与界面模块的交互。

系统测试：对整个系统进行端到端的测试，确保所有功能按预期工作。

测试类型如表 5-3 所示：

测试类型	覆盖范围	测试策略
黑盒测试	功能测试、用户界面交互、任务执行流程	- 模拟用户操作（如配置文件加载、任务执行） - 验证输入输出是否符合预期（如日志、通知）
白盒测试	核心模块逻辑（HBM.py、Task.py、adb_utils.py）	- 单元测试覆盖率 >80% - 边界条件测试（如无效配置文件、ADB 超时）
灰盒测试	模块间交互（GUI ↔ 核心逻辑、ADB ↔ 图像处理）	- 接口参数验证 - 多进程资源竞争测试（如并发任务执行）

表 0-3 测试类型

5.2 功能测试

5.2.1 核心功能测试

测 试 项	测试用例	预期结果	实际结果	是 否 通过
1.配置文 件加载	1.加载 example.json 配置文件 2.修改配置后保存并重新加载	配 置 项 （ 如 TASK_ORDER）正 确解析,任务顺序与 启用状态一致	配 置 项 （ 如 TASK_ORDER）正 确解析,任务顺序与 启用状态一致	通过
2.ADB 连 接 与 截 图	1.启动模拟器 2.执行 check_connect()	ADB 连接成功,截 图分辨率 1280x720	ADB 连接成功,截 图 分 辨 率 1280x720	通过
3.任务 执 行 流 程	验证任务链（决斗场 → 商店 → 排行榜 →好友）	任务按配置顺序执 行,日志输出正常, 结束时发送邮件通 知	任务按配置顺序执 行,日志输出正常, 结束时发送邮件通 知	通过
4.图像 匹 配 与 OCR	1.使用 match_pattern 匹配按钮 2.OCR 识别金币数 量	模 板 匹 配 准 确 率 >90%, OCR 识别误 差率 <10%	模 板 匹 配 准 确 率 >90%, OCR 识别误 差率 <10%	通过
5.异常 处 理	1.模拟 ADB 断开 2.提供无效模板路径	日志输出明确错误 信息,任务自动终止 或 重 试 （ 如 HBM_send_err_mail 触发）	日志输出明确错误 信息,任务自动终止 或 重 试 （ 如 HBM_send_err_mail 触发）	通过
6.多任 务 并 发	同时运行两个配置文 件（如 config1.json 和 config2.json）	进程隔离,日志独立 输出,资源竞争无冲 突	进程隔离,日志独立 输出,资源竞争无冲 突	通过

表 0-4 核心功能测试

1. 配置文件加载

打开 GUI.exe，看是否自动加载已有的配置文件

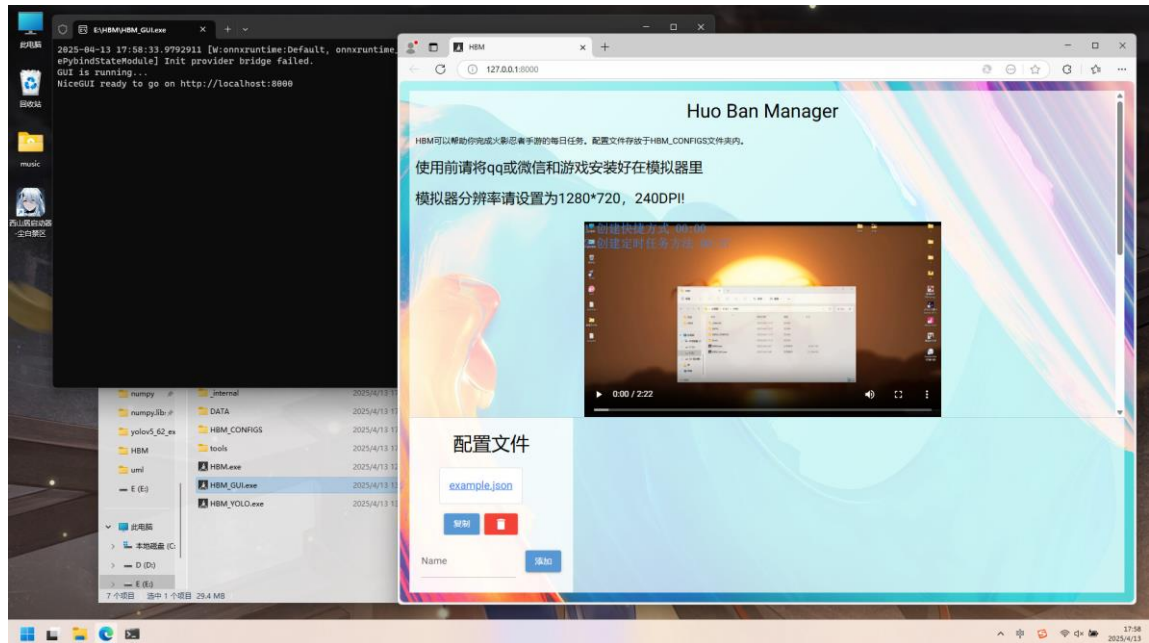


图 0-1 打开 GUI.exe 后自动配置已有的配置文件(example.json) (PC2)

点击 example.json 配置文件进入任务配置页查看状态

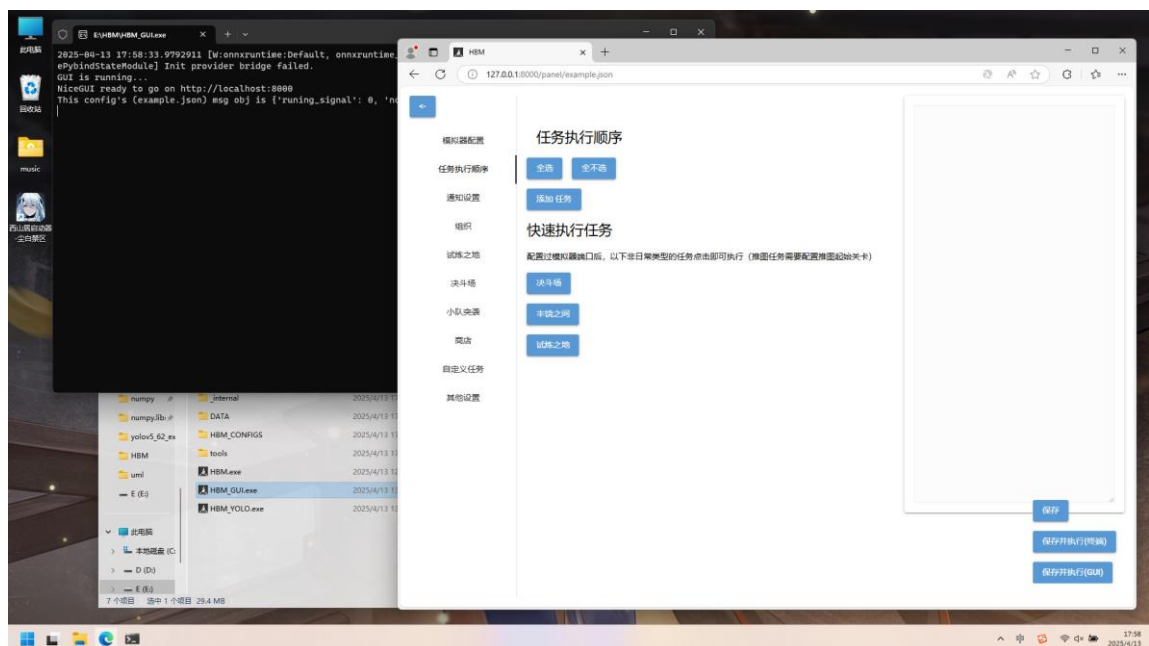


图 0-2 example.json 任务配置页面(初始化状态) (PC2)

在任务页配置任务，设置成：邮箱 → 决斗场 → 商店 → 任务集会所，其中，商店不启用，点击保存

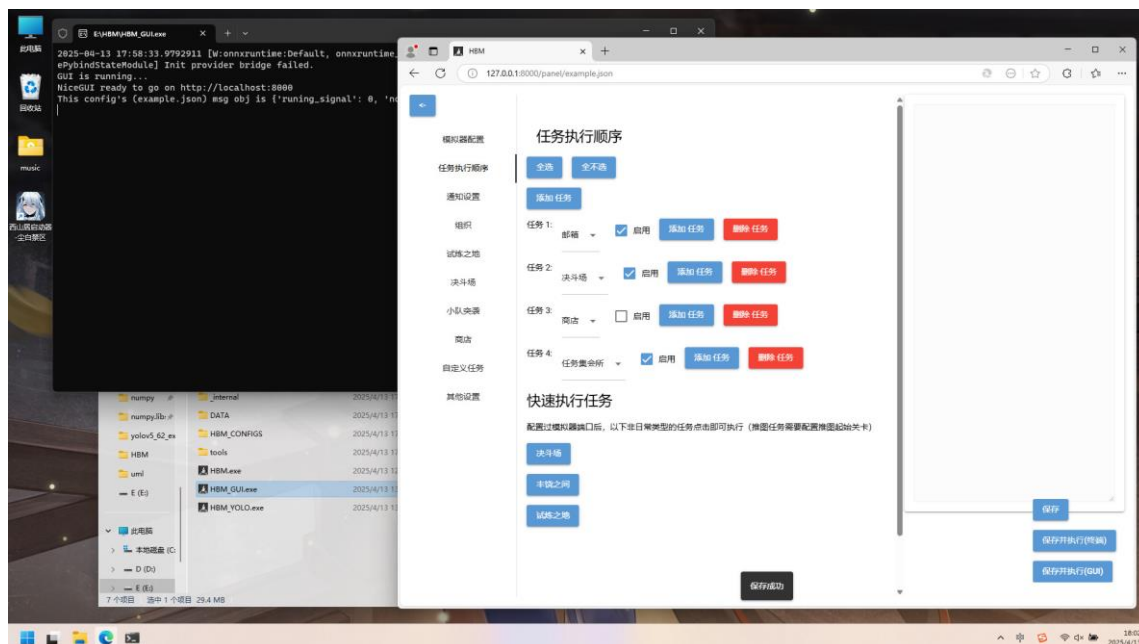


图 0-3 任务配置项配置内容并保存(PC2)

返回 UI 首页再次点击进入配置文件时，UI 的终端会显示对该配置文件进行读取

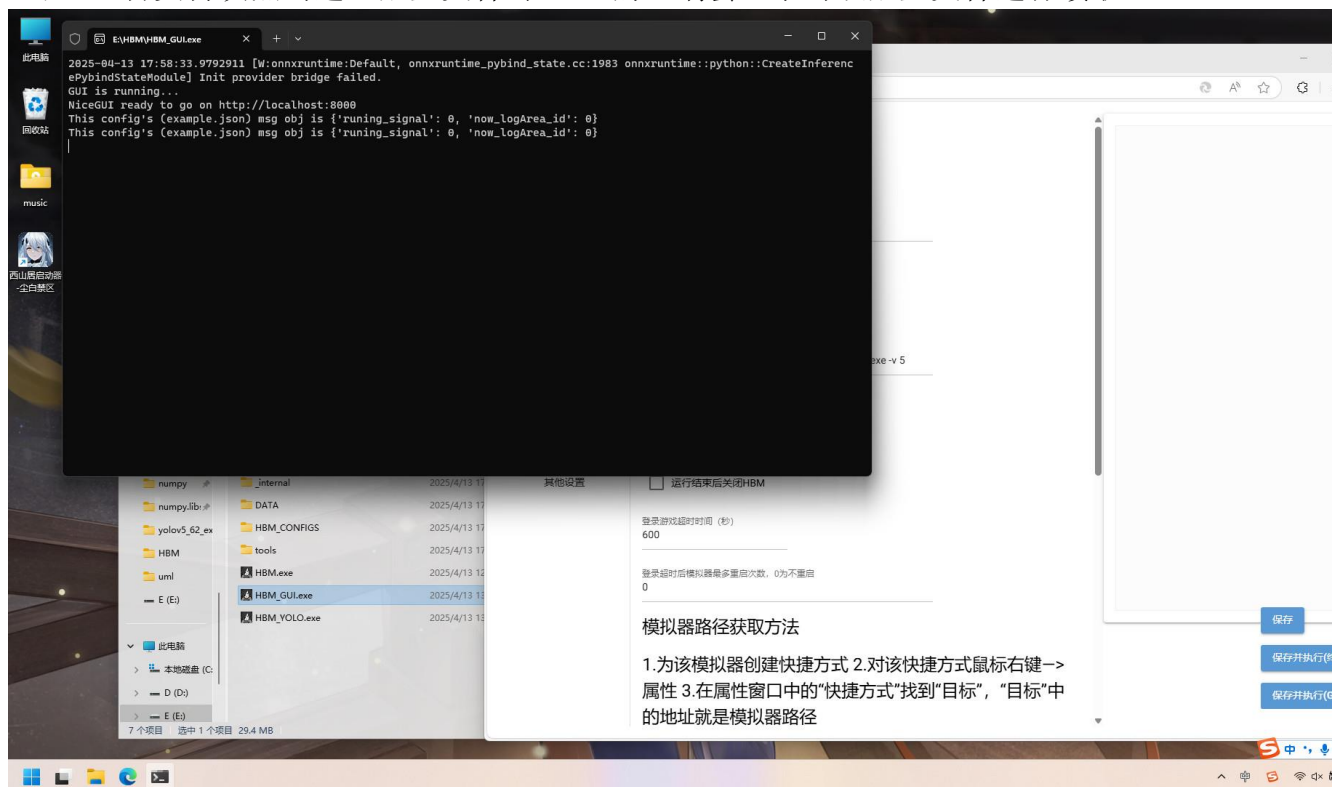


图 0-4 重新打开配置文件时,GUI 终端也会重新载入配置文件内容(PC2)

查看配置文件的任务配置页，发现和第一次进入该配置文件的任务配置页保存状态的配置一致，测试 1 符合预期

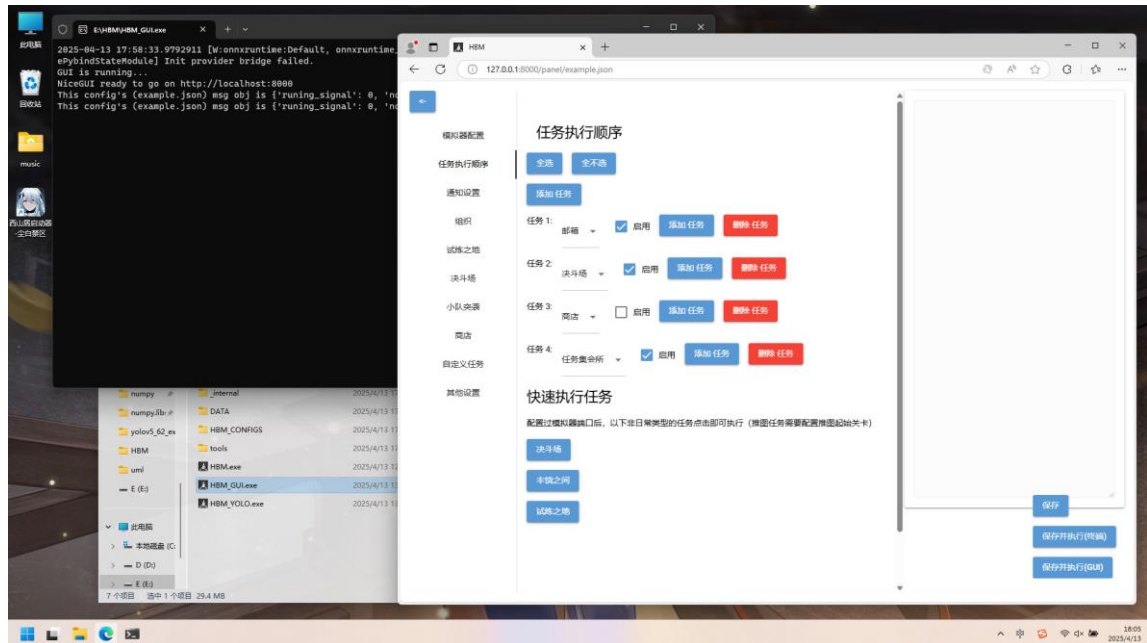


图 0-5 任务配置状态与上一次保存一致(时间对应) (PC2)

2.ADB 连接与截图

执行测试 2 前将配置文件的截图模式选择为 png，即将截取到的图片保存到软件根目录

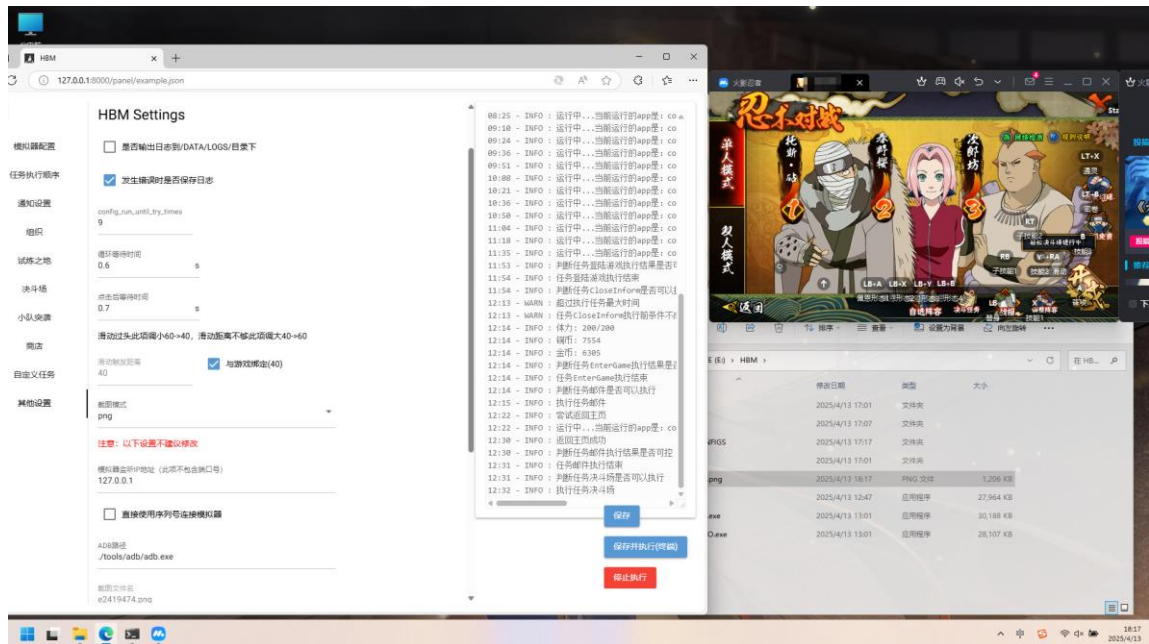


图 0-6 0、开启截图模式以便查看是否存在截图(PC2)

这里实时日志输出了“adb 连接成功”，并且软件根目录出现截图，表示 check_connect()成功执行，adb 连接成功

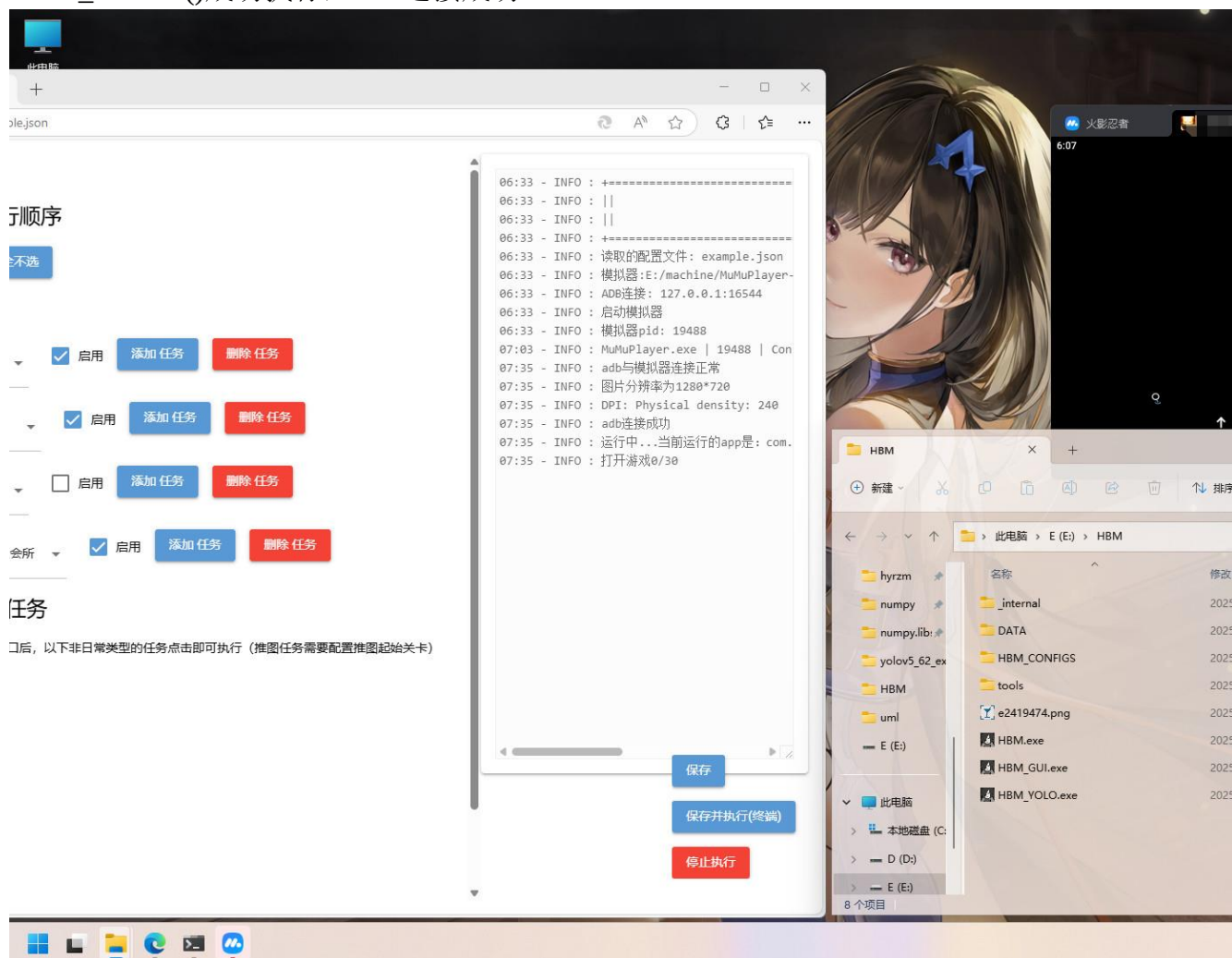


图 0-7 check_connect() 确保成功执行(出现”adb 连接成功”日志和截图) (PC2)

截图的图片分辨率也是 1280 * 720 测试 2 符合预期



图 0-8 截图大小 (PC2)

3.任务执行流程&4.图像匹配与 OCR

测试执行前：1.执行功能的 exe(非 GUI)创建快捷方式，并在快捷方式属性里的目标添加要执行的配置文件；

2.进入 UI 界面配置该配置文件的任务为：决斗场→商店→排行榜→好友，且都为启用状态



图 0-9exe 启动文件使用 example.json 配置文件(PC2)



图 0-10 验证任务链(PC2)

软件使用图片识别进行自动化，使用 match_pattern 进行匹配模版成功

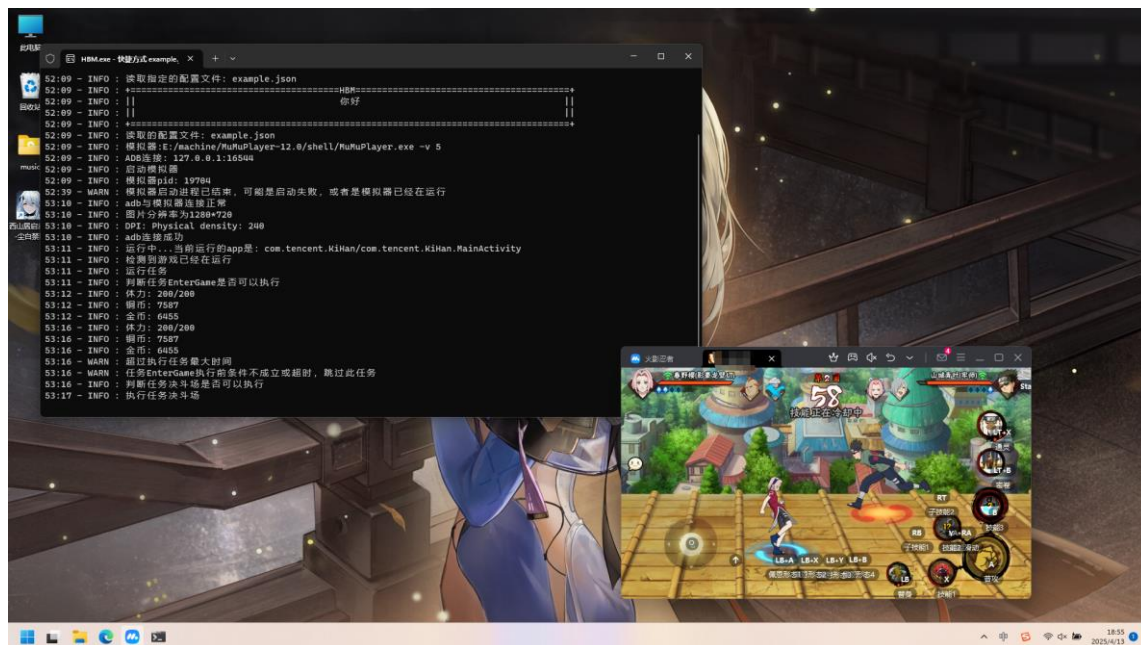


图 0-11 正在识别并点击(match_pattern) (PC2)

任务链执行完成后，软件终端结束任务最后输出 OCR 识别的体力数，铜币数，金币数，数量无误，并输出任务结束。日志输出保持正常。



图 0-12 任务链执行成功,orc 检测并输出体力、铜币、金币数(PC2)

邮箱返回 OCR 识别内容和执行的任务内容，测试 3 和测试 4 符合预期

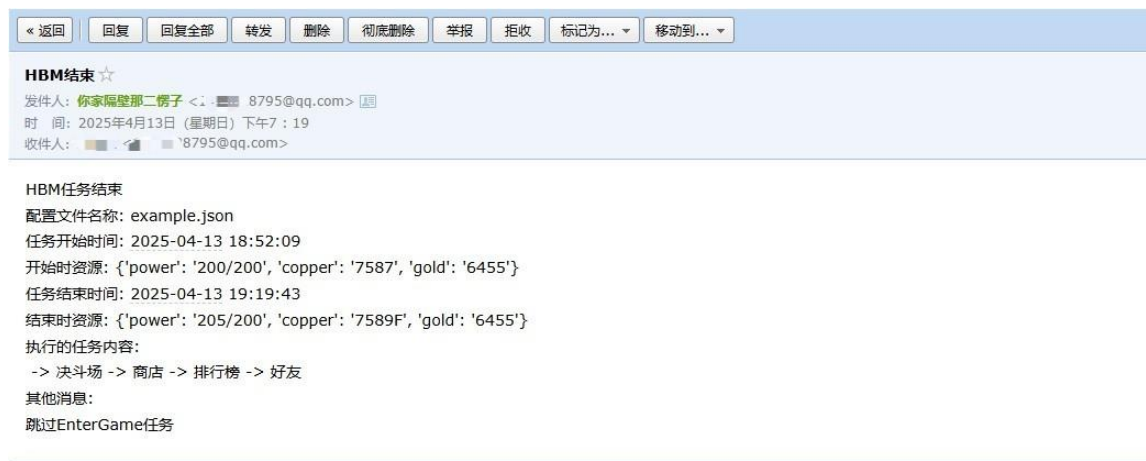


图 0-13 任务成功结束邮件返回任务流程和 ocr 检测内容

5.异常处理

这里使用 cmd 在软件执行任务的时候输入 adb disconnect 127:0:0:1:5565(adb 断开连接命令, 指定 127:0:0:1:5565), UI 终端显示出现错误。

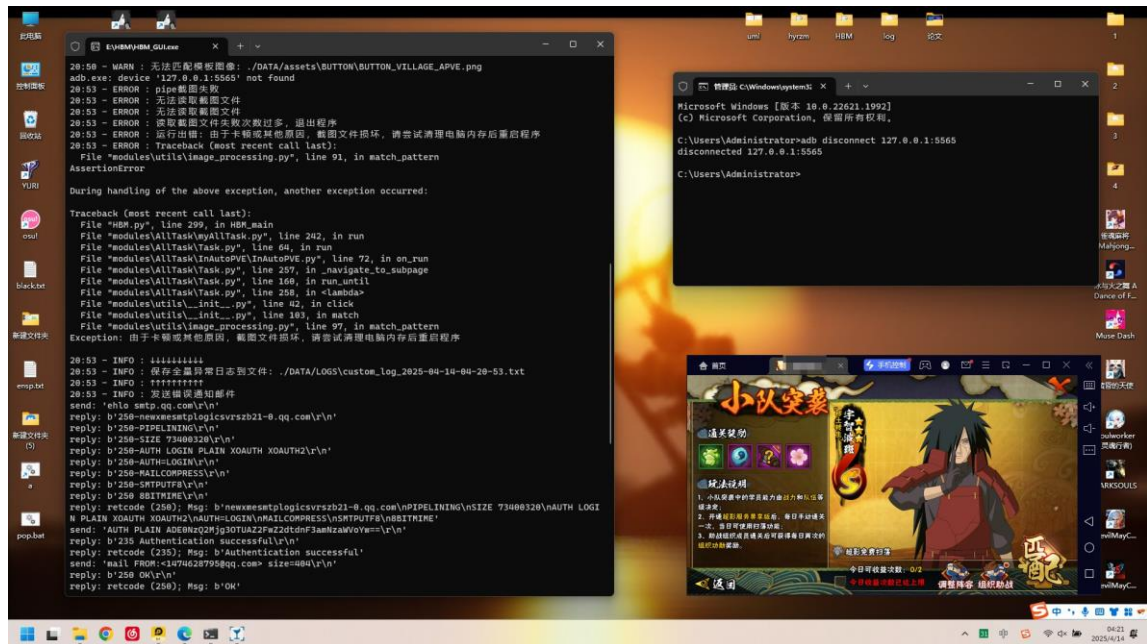


图 0-14 模拟 ADB 断开(PC1)

UI 界面实时日志输出任务错误

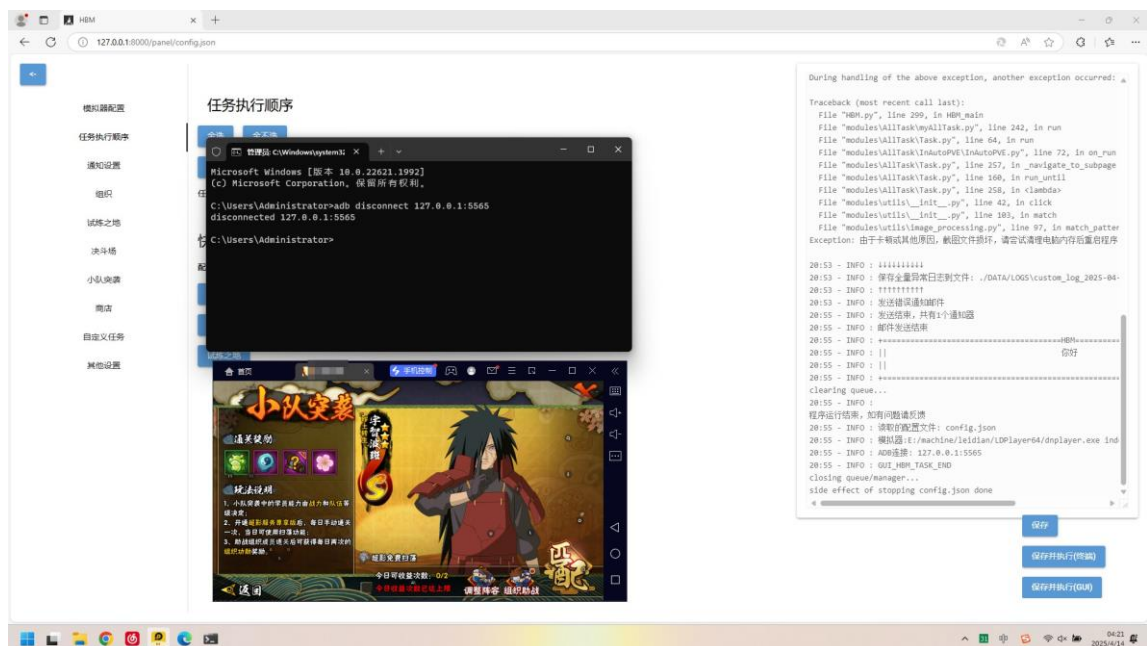


图 0-15 模拟 ADB 断开, 发送邮件(PC1)

邮箱返回任务执行失败并给出错误信息，测试任务 5.1 符合预期



图 0-16 收到失败邮件

提供错误的模版日志输出明确的错误，重试达最长时间后中止。



图 0-17 放入无法匹配的错误模版(PC1)

邮箱返回任务执行失败并给出错误信息 测试任务 5.2 符合预期



图 0-18 收到失败邮件

6.多任务并发

同时运行两个配置文件不会相互干扰, 测试 6 符合预期

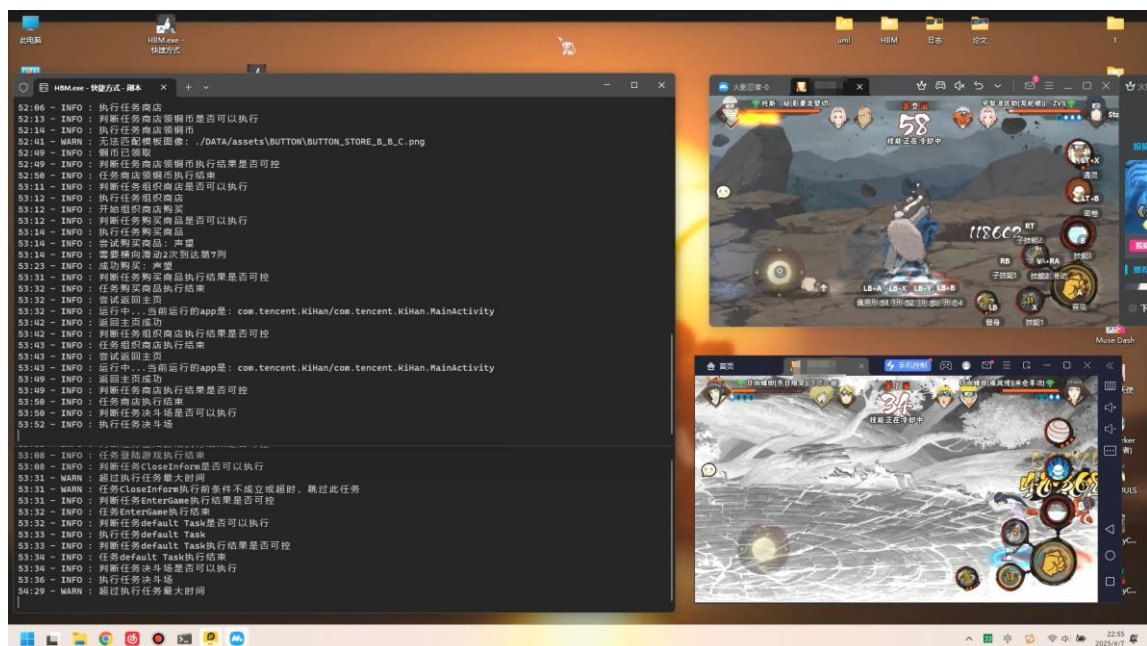


图 0-19 启动两个 exe 执行两个配置文件(PC1)

5.2.2 GUI 功能测试

测试项	测试用例	预期结果	实际结果	是否通过
1. 配置文件管理	1.创建/复制/删除配置文件 2.修改任务顺序并保存	配置文件操作成功，任务顺序实时更新	配置文件操作成功，任务顺序实时更新	通过
2. 实时日志绑定	1.启动任务并绑定日志窗口 2.中断日志线程	日志实时输出，中断后无残留进程	日志实时输出，中断后无残留进程	通过
3. 任务执行流程	1. 执行 HBM_core_process() 2. 验证任务链(邮箱 → 决斗场→任务集会所)	任务按配置顺序执行，日志输出正常，结束时发送邮件通知	任务按配置顺序执行，日志输出正常，结束时发送邮件通知	通过
4. 多任务并发	同时运行两个配置文件（如 config1.json 和 config2.json）	进程隔离，日志独立输出，资源竞争无冲突	进程隔离，日志独立输出，资源竞争无冲突	通过

表 0-5 GUI 功能测试

1. 配置文件管理

输入名称后点击添加按钮

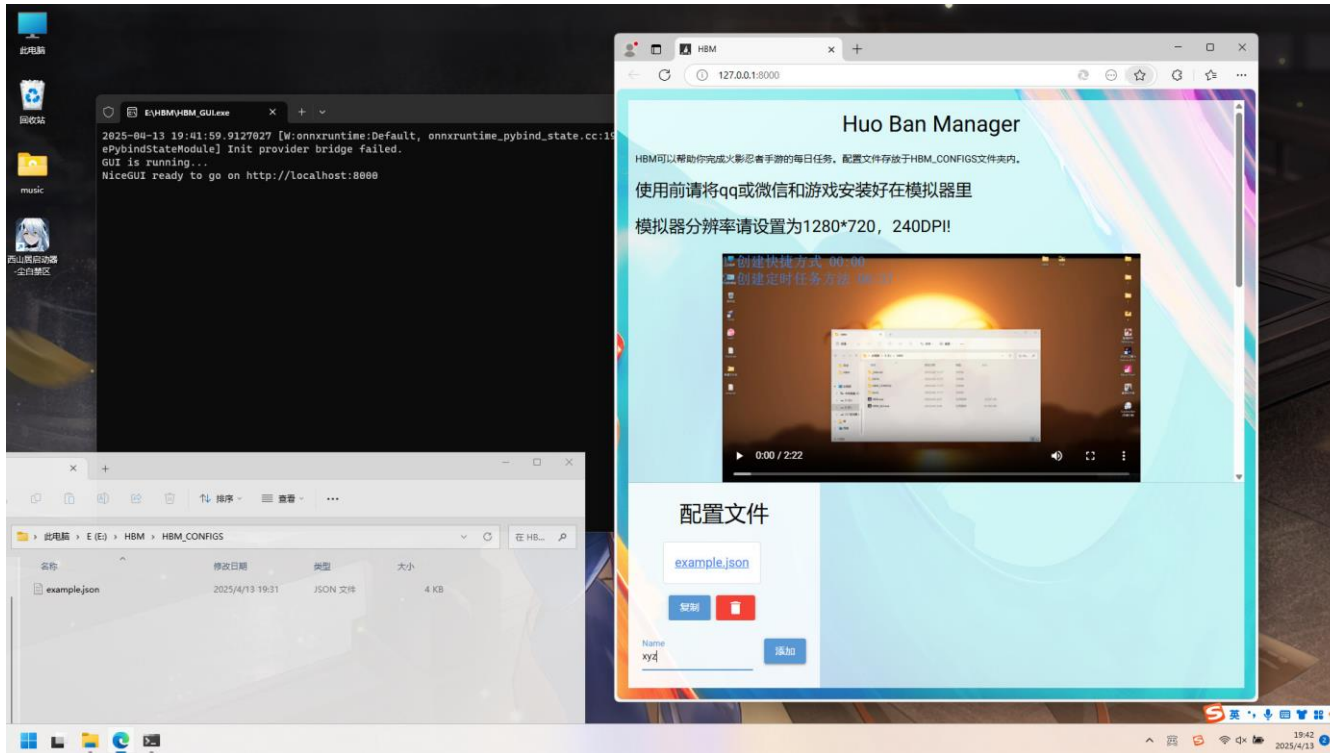


图 0-20 输入名称添加配置文件 (PC2)

添加成功，发现软件的配置文件目录出现新的文件，这里通过 ui 和记事本查看到该文件处于初始化配置状态，创建文件成功。

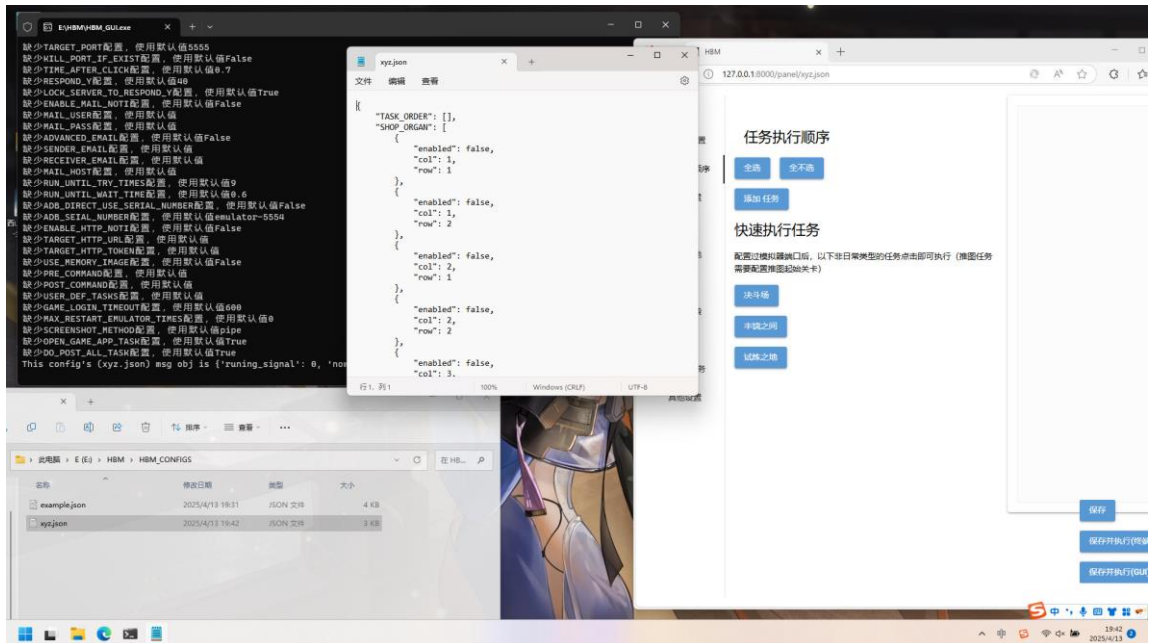


图 0-21 新的配置文件状态 (PC2)

这里对 example.json 进行复制操作复制的文件名为 example1.json

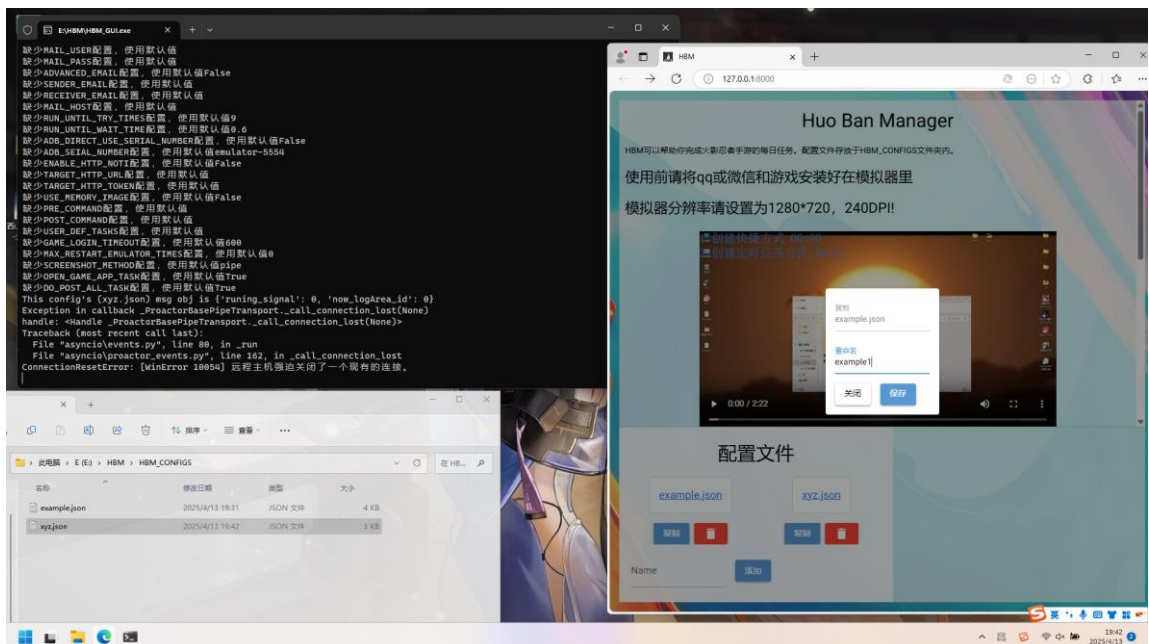


图 0-22 复制 example.json 文件 (PC2)

UI 首页显示刚复制的配置文件，配置文件夹也出现 example1.json 文件。

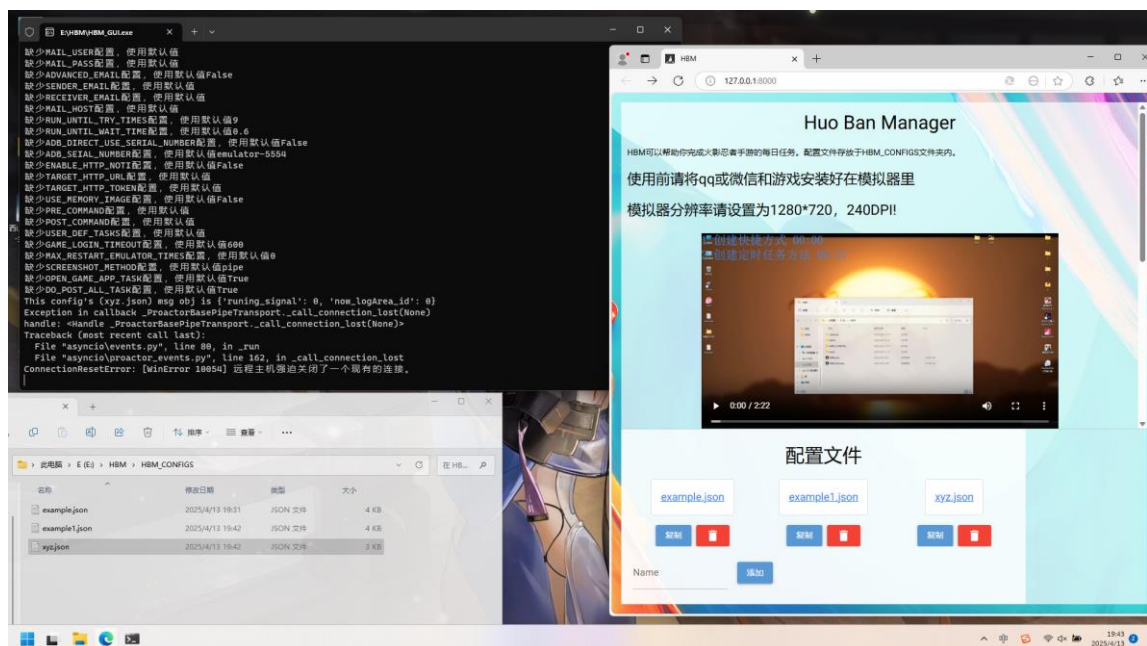


图 0-23 复制配置文件为 example1.json(PC2)

这里是 example.json 的任务配置是决斗场→商店→排行榜→好友,且都在启动状态,对其进行记录。

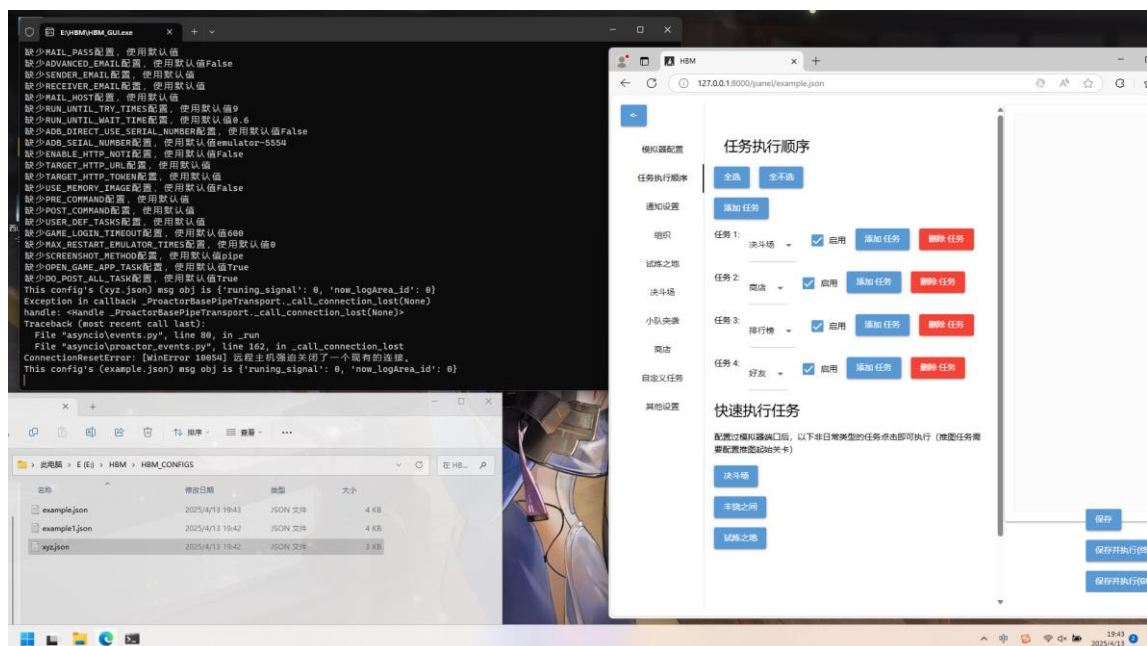


图 0-24 example.json 任务配置(PC2)

这里是 example1.json 的文件配置，发现和 example.json 的任务配置一样，都是决斗场→商店→排行榜→好友，且都在启动状态，复制配置文件成功

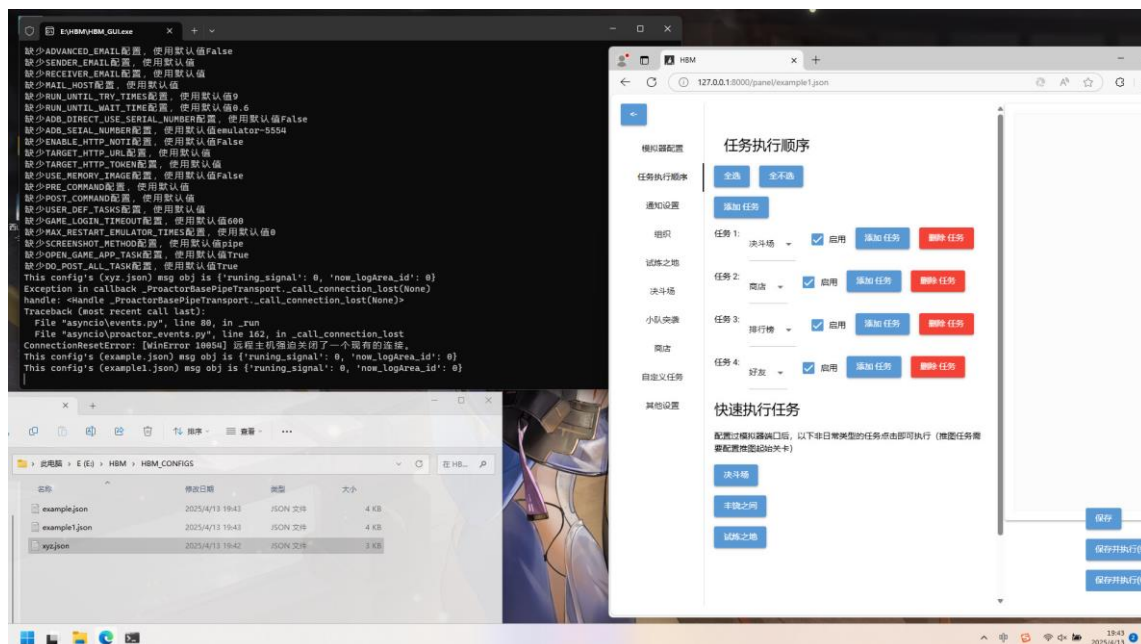


图 0-25 example1.json 任务配置(PC2)

现在对 example1.json 进行删除

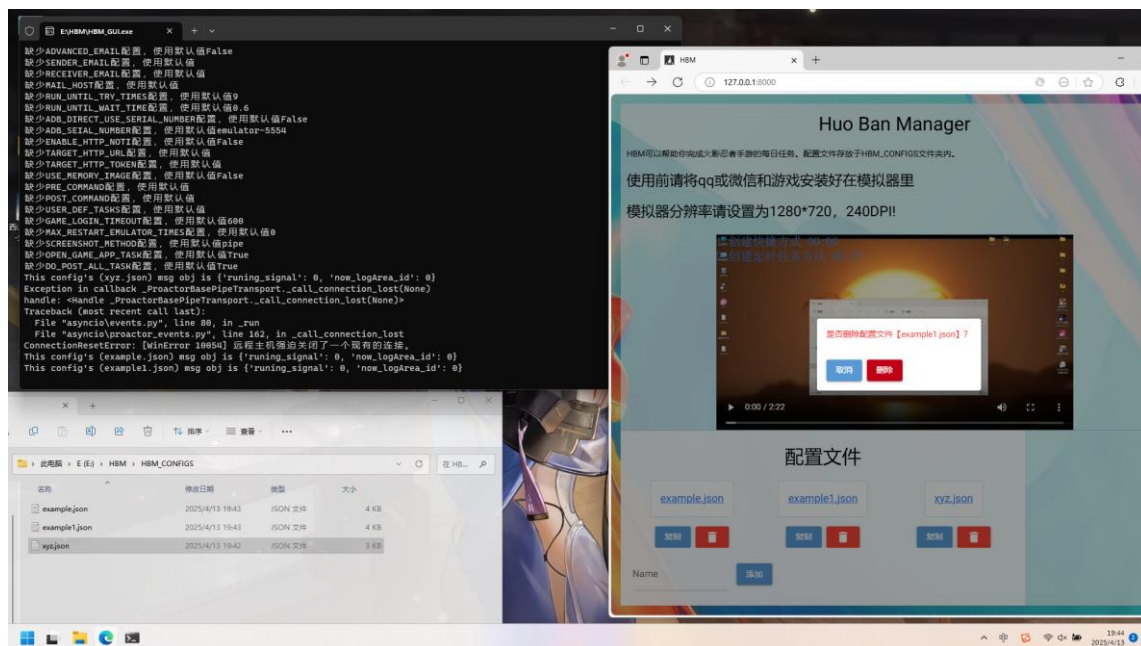


图 0-26删除配置文件(PC2)

结果配置文件夹和 UI 界面的 example1.json 都消失了，测试任务 1.1 符合预期。

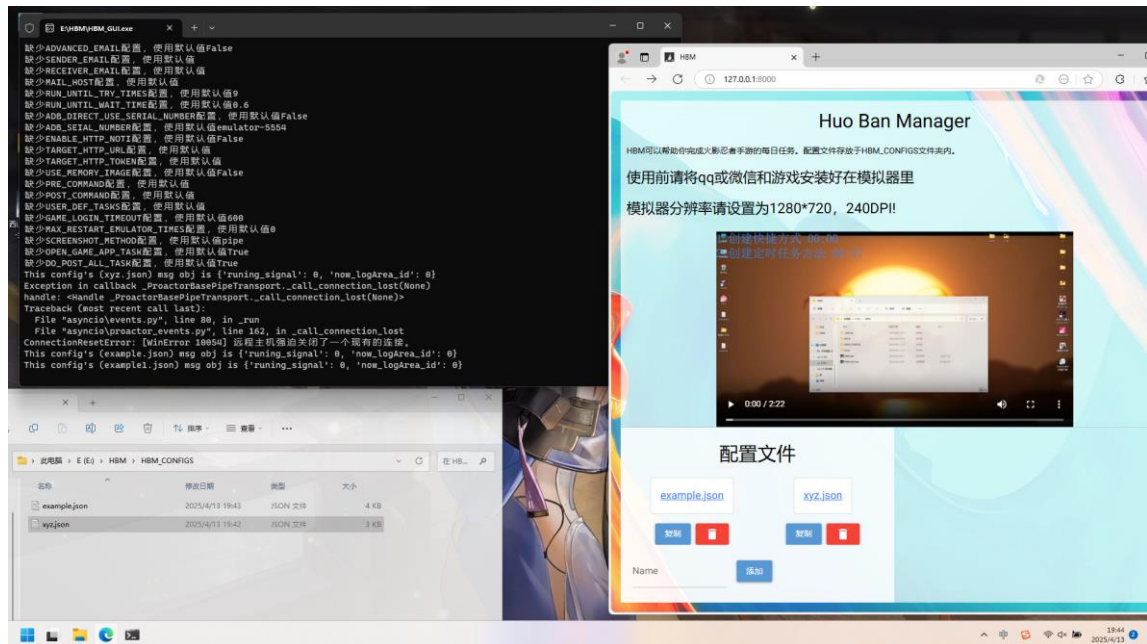


图 0-27成功删除配置文件(PC2)

更改任务配置，点击 example.json 可见现在的任务配置是邮箱 → 决斗场→商店→任务集会所，且商店未启用



图 0-28 example.json 原配置(PC2)

将任务配置更改成决斗场→商店→排行榜→好友并保存



图 0-29 example.json 更改后的配置 (PC2)

退出后再次查看任务配置发现是更改后保存的配置，测试 1.2 符合预期



图 0-30再次进入查看配置 (PC2)

2. 实时日志绑定

点击启动后，执行任务中，UI 的日志也在实时输出

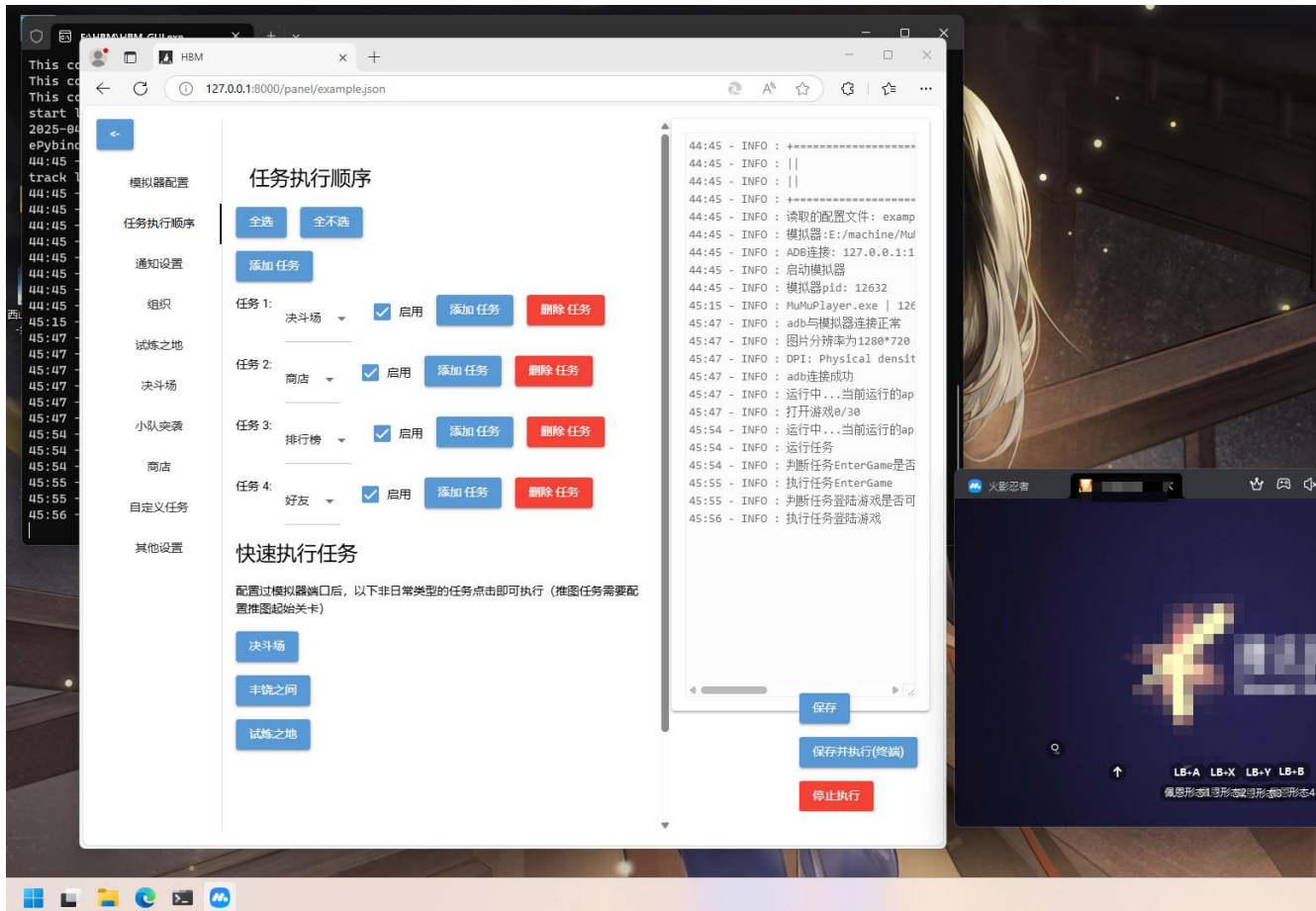


图 0-31 日志跟随程序打印 (PC2)

点击停止按钮后，日志停止输出，测试 2 符合预期

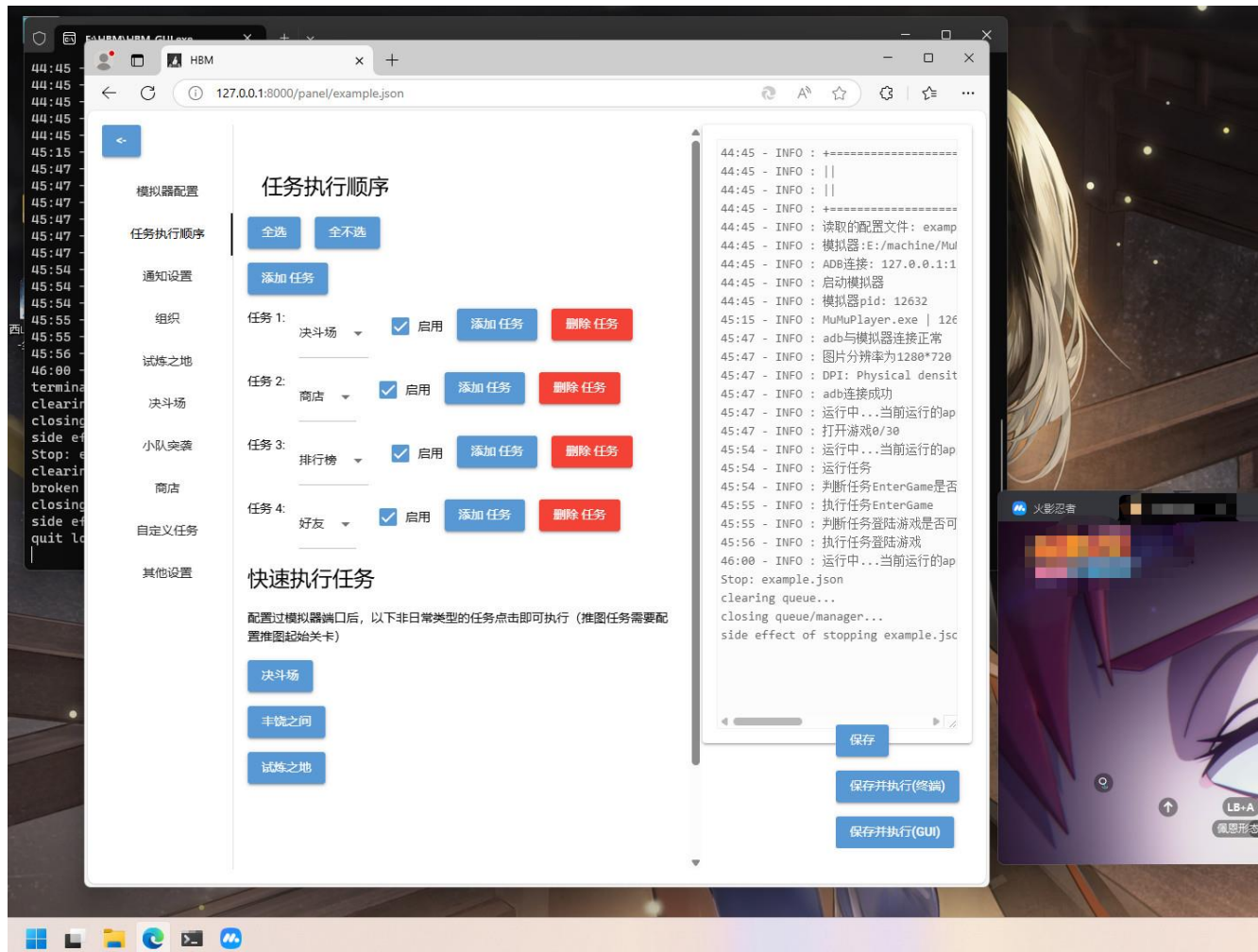


图 0-32 停止程序, 日志一并停止 (PC2)

3. 任务执行流程

使用 UI 来启动任务进程并完成

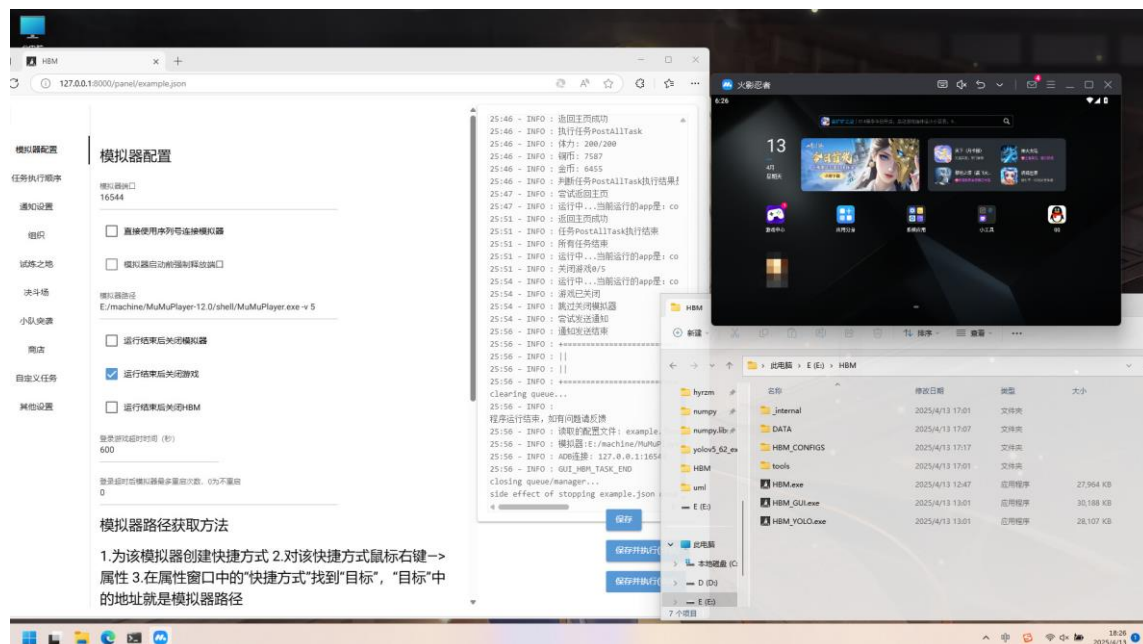


图 0-33 完成任务日志(衔接图 5-7) (PC2)

邮箱返回任务执行成功邮件，测试 3 符合预期



图 0-34 收到任务完成邮件

4. 多任务并发

通过 UI 来启动两个配置文件并同时进行自动化任务，并没有相互干扰，测试 4 符合预期

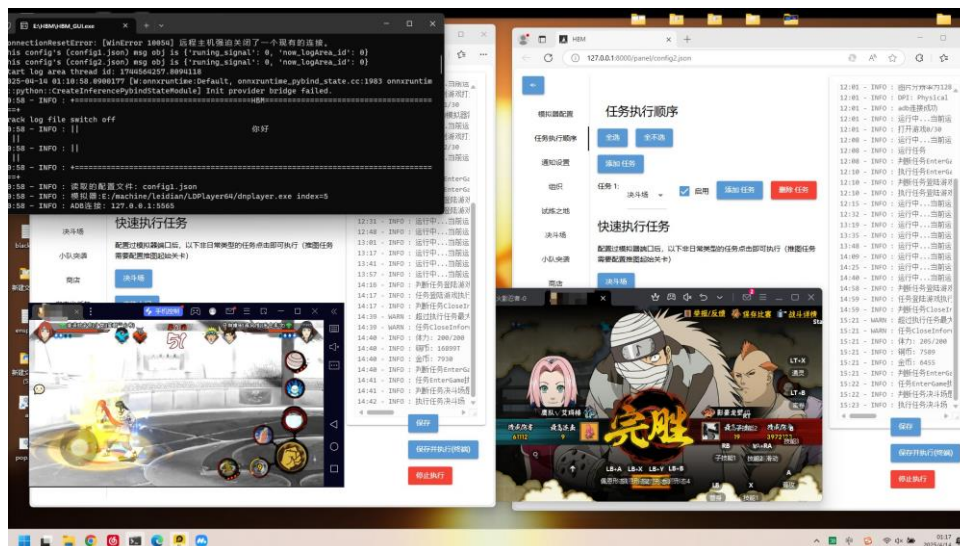


图 0-35 使用 GUI 同时运行两个配置文件(PC1)

5.3. 性能测试

5.3.1 核心性能指标

测试项	测试方法	预期指标	实际结果	是否满足
1.ADB 操作延迟	测量 click_on_screen 和 swipe_on_screen 的平均响应时间	单次点击 <500ms，滑动 <1s	单次点击 <500ms，滑动 <1s	满足
2. 图像处理效率	统计 match_pattern 处理 100 张截图的耗时	平均 <600ms/次	平均 <600ms/次	满足
3. 多任务并发能力	1.使用 exe 同时运行 3 个配置文件，监控 CPU/内存占用 2.使用 gui 同时运行 3 个配置文件，监控 CPU/内存占用	CPU <80%，内存 <400MB CPU <180%，内存 <800MB	CPU <80%，内存 <400MB CPU <180%，内存 <800MB	满足 满足

表 0-6 核心性能测试

1. ADB 操作延迟&2. 图像处理效率

编写 Python 文件进行计算

```
D:\apl\c\SE\python39\python39.exe D:\Documents\Desktop\HBM\test_demo.py
读取默认config文件: config.json
点击平均延迟: 0.337s
滑动平均延迟: 0.591s
匹配耗时统计:
- 总次数: 100
- 总耗时: 29.257 秒
- 平均耗时: 292.57 毫秒/次

进程已结束, 退出代码为 0
```

图 0-36 某次操作测试结果(PC1)

测试 1、2 符合预期

3.多任务并发能力

编写 cmd bat 文件获取 cpu 使用和内存占用,并计算平均值,同时执行三个配置文件

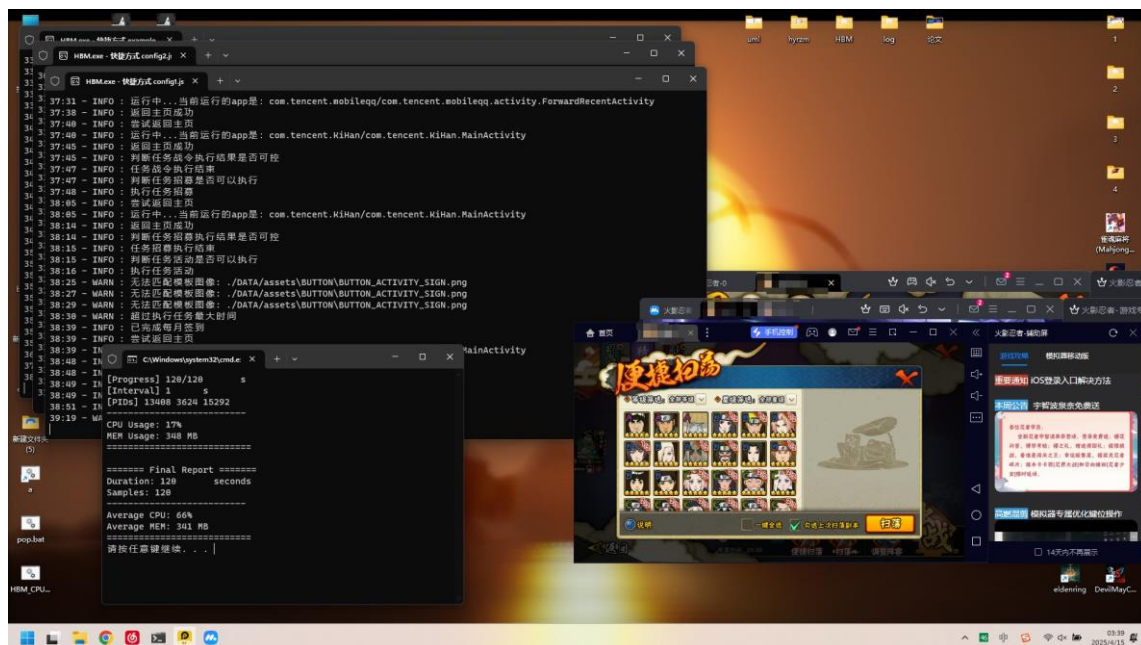


图 0-37 某次测试截图(使用 exe 同时运行 3 个任务不同的配置文件) (PC1)

多次测试结果：在 120s 内，CPU <80%，内存 <400MB，测试 3.1 符合预期

```
C:\Windows\system32\cmd.e: X +

[Progress] 120/120      s
[Interval] 1          s
[PIDs] 13408 3624 15292

-----
CPU Usage: 17%
MEM Usage: 348 MB
=====

===== Final Report =====
Duration: 120          seconds
Samples: 120
-----

Average CPU: 66%
Average MEM: 341 MB
=====

请按任意键继续. . . |
```

图 0-38 某次 120s 内 exe 性能测试结果 (PC1)

使用用户界面点击运行来运行 3 个不同的配置文件

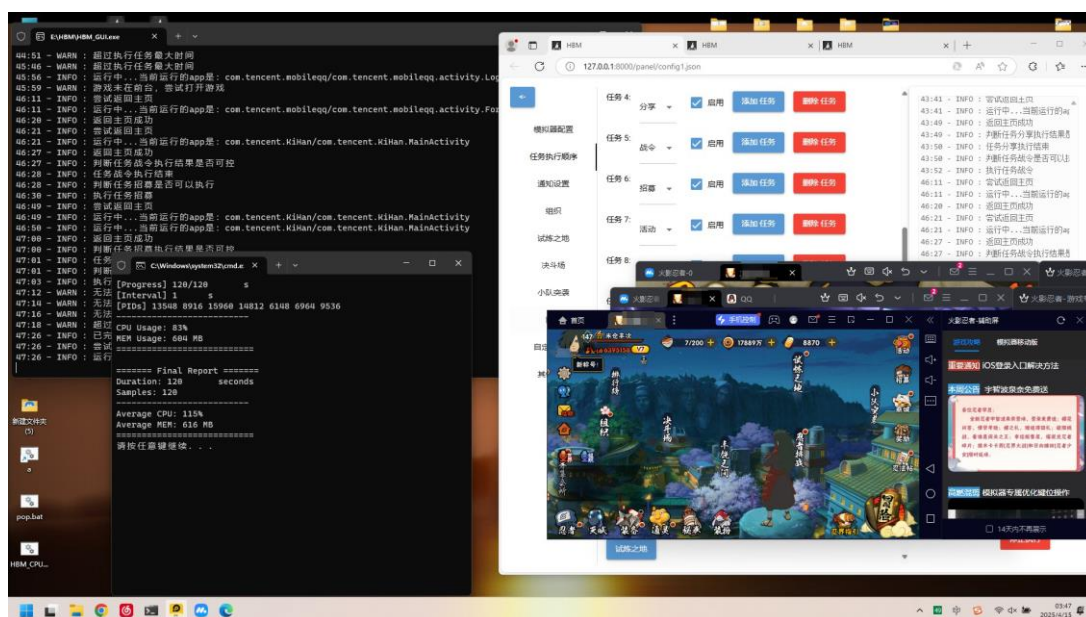


图 0-39 某次测试截图(使用 GUI 同时运行 3 个任务不同的配置文件) (PC1)

多次测试结果：在 120s 内，CPU <180%，内存 <800MB，测试 3.2 符合预期

图 0-40 某次 120s 内 GUI 性能测试结果（PC1）

5.3.2 稳定性测试

测试项	测试方法	预期结果	实际结果	是否满足
1. 长时间运行	连续执行任务 6 小时	无内存泄漏，任务成功率 >95%	无内存泄漏，任务成功率 >95%	满足
2. 高频率操作	模拟每秒 10 次点击操作（如 MultiTouchUtils）	无进程崩溃，ADB 无卡死	无进程崩溃，ADB 无卡死	满足

表 0-7 稳定性测试

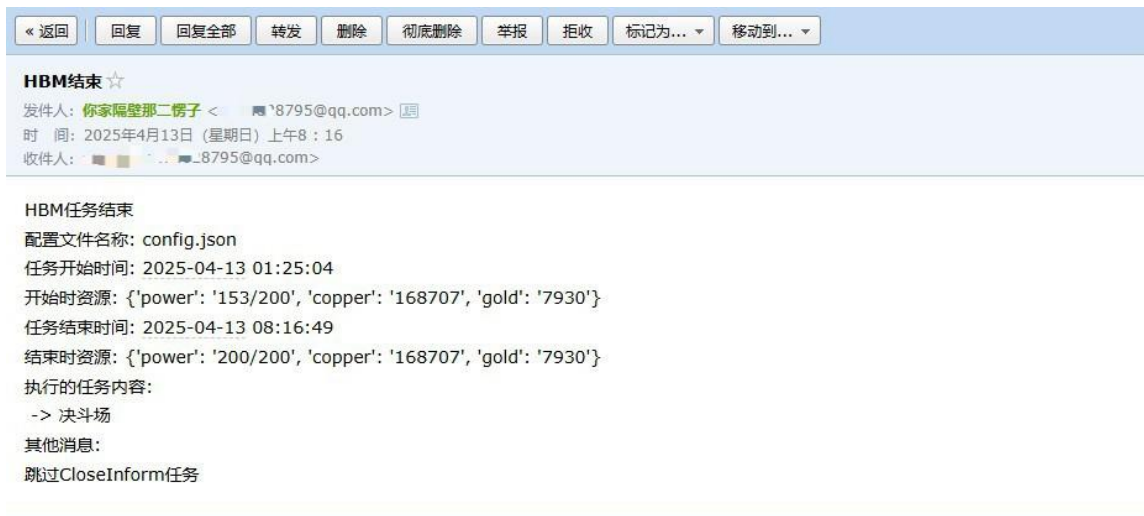


图 0-41 长时间运行

测试 1 符合预期

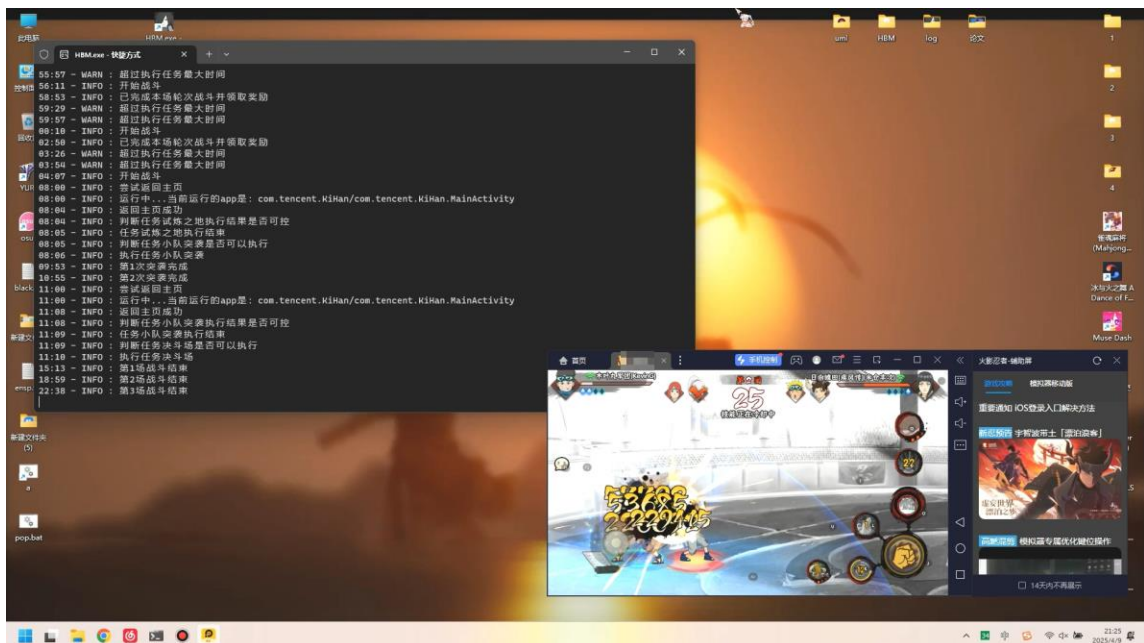


图 0-42 高频率操作 (PC1)

调整引用 MultiTouchUtils 的函数 advanced_hybrid_operation 的点击频率, 测试 2 符合预期

5.4. 安全测试

5.4.1 数据安全

测试项	测试方法	预期结果	实际结果	是否满足
配置文 件加密	检查 MAIL_PASS 是否使用 decrypt_data 解密	敏感字段加密存储， 密 钥 动 态 生 成 (ENCRYPT_KEY)	敏感字段加密存储， 密 钥 动 态 生 成 (ENCRYPT_KEY)	满足

表 0-8 数据安全测试

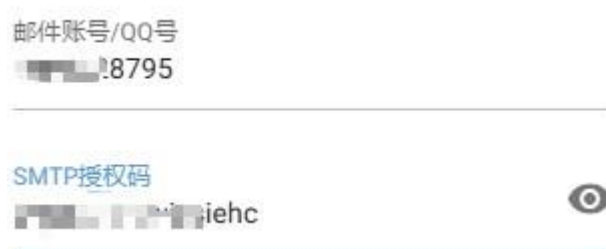


图 0-43 配置文件中输入的内容

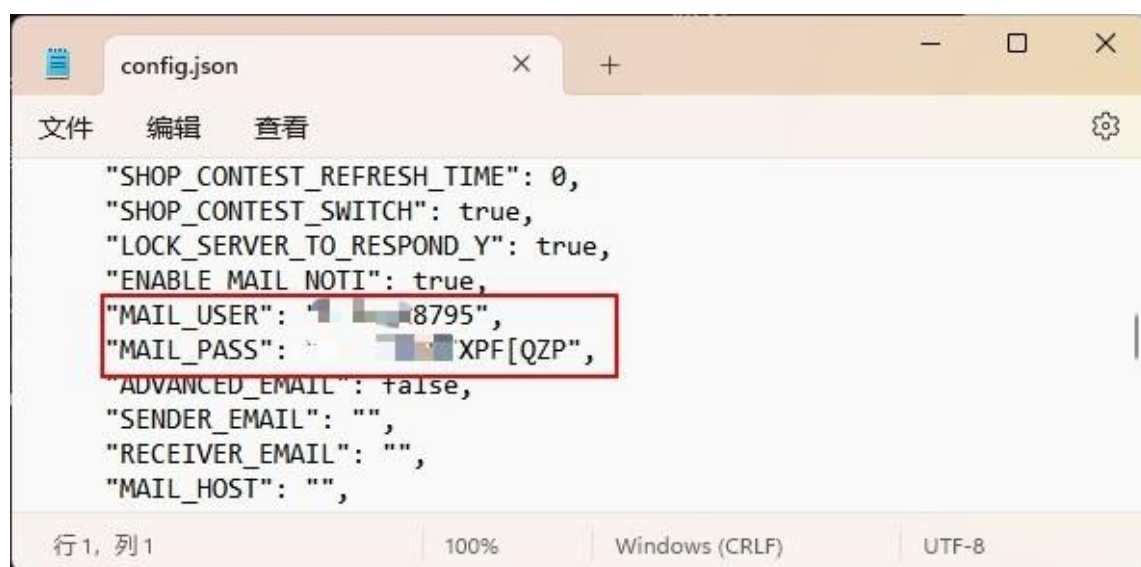
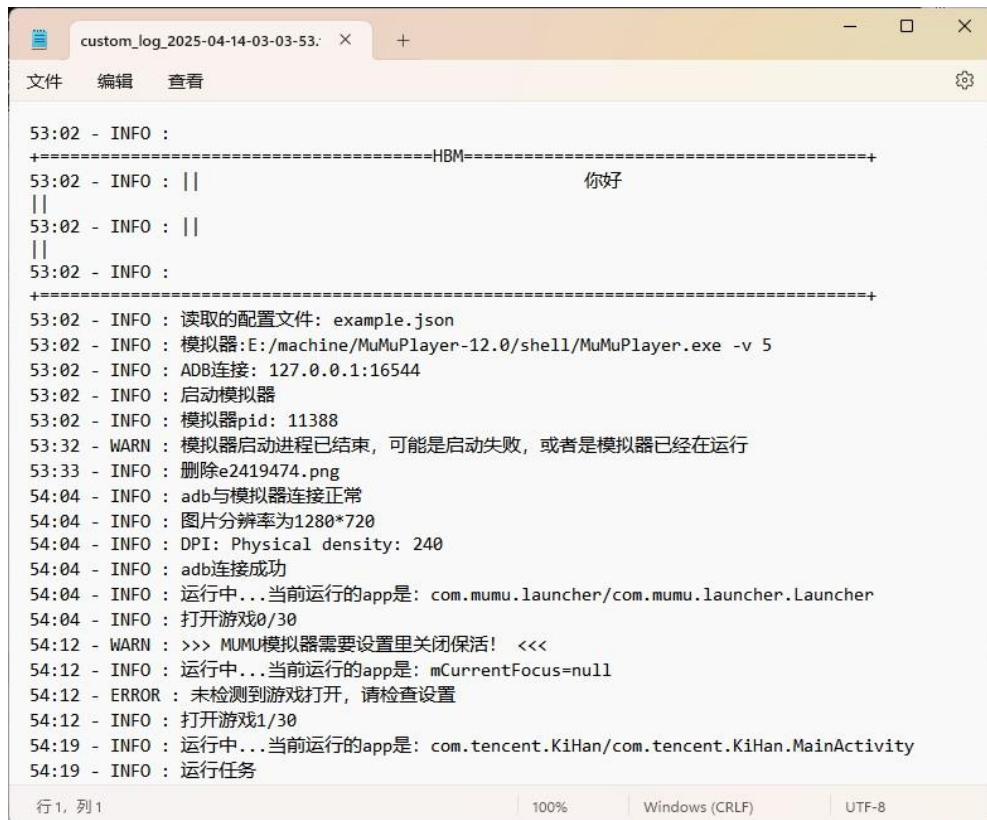


图 0-44配置文件中显示的内容

5.4.2 日志安全

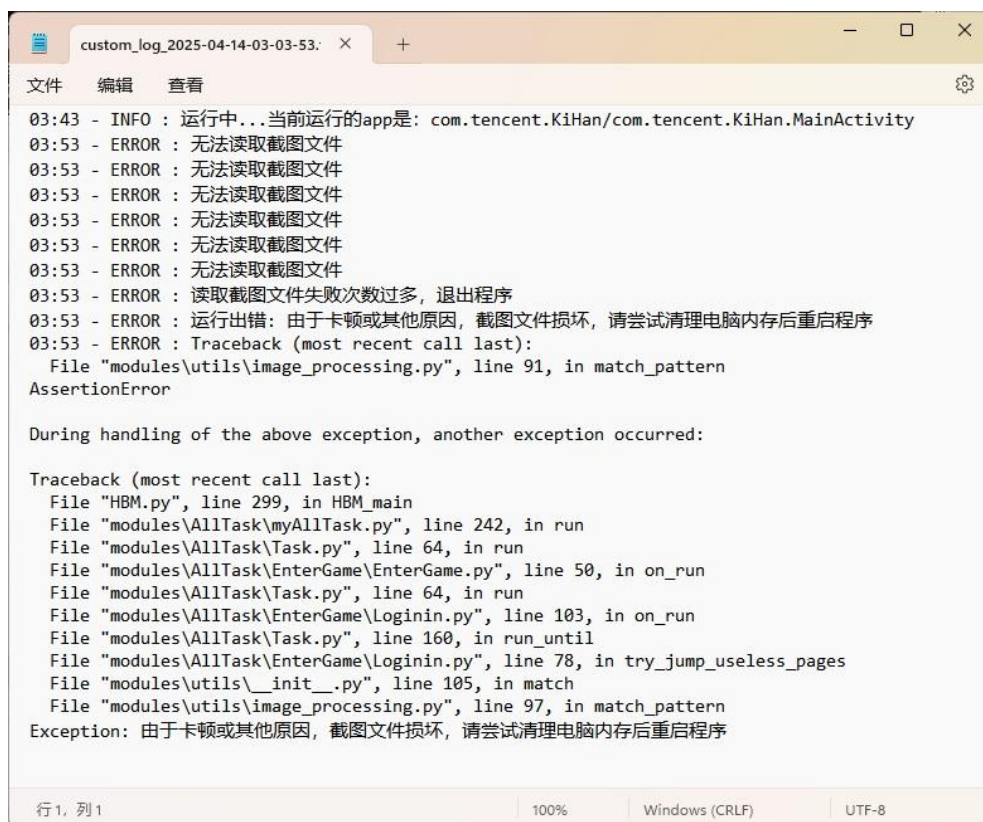
测试项	测试方法	预期结果	实际结果	是否满足
日志泄露	检查日志文件（如 DATA/LOGS）是否包含敏感信息	日志脱敏，无明文密码或密钥	日志脱敏，无明文密码或密钥	满足

表 0-9 日志安全测试



```
custom_log_2025-04-14-03-03-53: x +
文件 编辑 查看
53:02 - INFO :
+-----HBM-----+
53:02 - INFO : || 你好
||
53:02 - INFO : ||
||
53:02 - INFO :
+-----+
53:02 - INFO : 读取的配置文件: example.json
53:02 - INFO : 模拟器:E:/machine/MuMuPlayer-12.0/shell/MuMuPlayer.exe -v 5
53:02 - INFO : ADB连接: 127.0.0.1:16544
53:02 - INFO : 启动模拟器
53:02 - INFO : 模拟器pid: 11388
53:32 - WARN : 模拟器启动进程已结束, 可能是启动失败, 或者是模拟器已经在运行
53:33 - INFO : 删除e2419474.png
54:04 - INFO : adb与模拟器连接正常
54:04 - INFO : 图片分辨率为1280*720
54:04 - INFO : DPI: Physical density: 240
54:04 - INFO : adb连接成功
54:04 - INFO : 运行中...当前运行的app是: com.mumu.launcher/com.mumu.launcher.Launcher
54:04 - INFO : 打开游戏0/30
54:12 - WARN : >>> MUMU模拟器需要设置里关闭保活! <<<
54:12 - INFO : 运行中...当前运行的app是: mCurrentFocus=null
54:12 - ERROR : 未检测到游戏打开, 请检查设置
54:12 - INFO : 打开游戏1/30
54:19 - INFO : 运行中...当前运行的app是: com.tencent.KiHan/com.tencent.KiHan.MainActivity
54:19 - INFO : 运行任务
行 1, 列 1 100% Windows (CRLF) UTF-8
```

图 0-45某份日志头部



The screenshot shows a log file viewer window titled 'custom_log_2025-04-14-03-03-53'. The log contains the following entries:

```
03:43 - INFO : 运行中...当前运行的app是: com.tencent.KiHan/com.tencent.KiHan.MainActivity
03:53 - ERROR : 无法读取截图文件
03:53 - ERROR : 无法读取截图文件
03:53 - ERROR : 无法读取截图文件
03:53 - ERROR : 无法读取截图文件
03:53 - ERROR : 无法读取截图文件
03:53 - ERROR : 无法读取截图文件
03:53 - ERROR : 读取截图文件失败次数过多, 退出程序
03:53 - ERROR : 运行出错: 由于卡顿或其他原因, 截图文件损坏, 请尝试清理电脑内存后重启程序
03:53 - ERROR : Traceback (most recent call last):
  File "modules\utils\image_processing.py", line 91, in match_pattern
AssertionError

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "HBM.py", line 299, in HBM_main
  File "modules\AllTask\myAllTask.py", line 242, in run
  File "modules\AllTask\Task.py", line 64, in run
  File "modules\AllTask\EnterGame\EnterGame.py", line 50, in on_run
  File "modules\AllTask\Task.py", line 64, in run
  File "modules\AllTask\EnterGame\Loginin.py", line 103, in on_run
  File "modules\AllTask\Task.py", line 160, in run_until
  File "modules\AllTask\EnterGame\Loginin.py", line 78, in try_jump_useless_pages
  File "modules\utils\__init__.py", line 105, in match
  File "modules\utils\image_processing.py", line 97, in match_pattern
Exception: 由于卡顿或其他原因, 截图文件损坏, 请尝试清理电脑内存后重启程序
```

The status bar at the bottom indicates '行 1, 列 1', '100%', 'Windows (CRLF)', and 'UTF-8'.

图 0-46某份日志尾部

测试符合预期。

第六章 总结与展望

声明:本论文只做学术研究,不做任何商业行为。

这个项目从零到实现,大部分地方都离不开 opencv 和 adb 这两个核心工具,也离不开 python 这种适用于开发快速原型项目的语言,方便测试和调试,节省了很多时间。

实际测试下来还是有很多不尽人意的地方,比如在高延迟的情况下或者是电脑资源已经有一半被使用的情况下,程序会因为模拟器的卡顿而“自动略过”任务,导致这个游戏如果你在这两种情况下需要重复两次这个工具才能确保“重复任务”能够全部完成。还有一些细节复杂的图片在这种情况下也是给人感觉是识别不了的,但是工具实际已经识别到了。当然,这个工具还是能节省不少时间让玩家去专注别的事情而不是这些无意义且重复的事情上。

总之这个工具还是存在很多问题,但起码这次实践证明了“用代码打游戏”这种事情是靠谱的,也为了后续的优化和功能开发打下基础。希望能够在未来不断地修改中能继续提升游戏体验,发挥更多的价值。

参考文献

- [1] 鹏军创富. 8 款搬砖游戏, 每天 200 左右, 准备一台电脑, 挂机即可! [EB/OL]. 知乎, 2021-12-23. <https://zhuanlan.zhihu.com/p/449076443>
- [2] 互联网那些事. 两年五十一万, 代练比普通人更赚钱? [EB/OL]. 澎湃新闻, 2024-05-11. https://www.thepaper.cn/newsDetail_forward_27323725
- [3] 李愚. 刘韵珊. 游走在灰色边界, 游戏代练真的很赚钱? [EB/OL]. 腾讯网, 2024-06-23. <https://news.qq.com/rain/a/20240623A04SP600>
- [4] TalkingData 移动数据研究中心. TalkingData-2015 年移动游戏行业报告[EB/OL]. 移动观象台, 2016-03-03. <https://mi.talkingdata.com/report-detail.html?id=330>
- [5] Roman Kierst (小罗), Yang Tingting (杨婷婷). Grinding for Gold: Inside China's Game Boosting Economy[EB/OL]. The world of chinese, 2024-06-26. <https://www.theworldofchinese.com/2024/06/grinding-for-gold-inside-chinas-game-boosting-economy/>
- [6] 游谈舍. 2077 代肝现象: 花了钱买的 game, 我为什么还要花时间去玩? [EB/OL]. 网易, 2020-12-26. <https://www.163.com/dy/article/FUPSNN69052688NB.html>
- [7] 高原. 网络游戏代练成为新兴职业 在校生成生力军[EB/OL]. 新浪, 2005-10-10. <https://tech.sina.com.cn/i/2005-10-10/1141736061.shtml>
- [8] 周敏. 游戏代练形成产业链[J]. 沪港经济, 2017, (007):46-49.
- [9] 阿祥. 代练执照洗白灰色产业[J]. 互联网天地, 2009, (01):54-55.
- [10] 尹天. 你如何看待在游戏中代练/代打的行为? [EB/OL]. 知乎, 2021-03-17. <https://www.zhihu.com/question/449192547>
- [11] 赵若慈, 豹变. 代肝游戏月入过万, 我是《原神》的“采摘工人” [EB/OL]. 澎湃新闻, 2024-01-02. https://www.thepaper.cn/newsDetail_forward_25867670
- [12] bigfun 毕方. 代肝、代练——游戏背后的灰色产业圈[EB/OL]. 知乎, 2021-03-25. <https://zhuanlan.zhihu.com/p/359912990>
- [13] 前途工作社区. 游戏代练产业在未来的发展趋势怎么样? [EB/OL]. 知乎, 2021-08-10. <https://www.zhihu.com/question/478359108>
- [14] 阿拉德小酒馆. DNF: 第 3 种科技搬砖火了! AI 风席卷阿拉德, 有用但不要用! [EB/OL]. 17173, 2024-11-17.

- <https://news.17173.com/content/11172024/153305455.shtml>
- [15]半两财经. 起底游戏代练：收入低还很“卷” 已形成黑产业链. [EB/OL]. 2024-05-06. <https://news.qq.com/rain/a/20240506A01REN00>
- [16]Moselle. 游戏代练行业盈利模式深度分析——代练商家的养成之路[EB/OL]. 界面新闻, 2018-02-11. <https://www.jiemian.com/article/1938475.html>
- [17]新识研究所. “胖猫”背后的游戏代练：有人月入3万，有人倒贴钱. [EB/OL]. 脉脉, 2024-05-08. <https://maimai.cn/article/detail?fid=1831983375&efid=574QayWFy3CzkipmP0ZHhw>
- [18]Testers HUB. The Evolution of Game Testing: From Manual to Automated Processes [EB/OL]. medium, 2024-08-06. <https://medium.com/@testershut/the-evolution-of-game-testing-from-manual-to-automated-processes-6c2ac8aef34b>
- [19]Roy Segall. The History of Mobile – Part 2: The Evolution of Testing [EB/OL]. Testim, 2024-05-08. <https://www.testim.io/blog/the-history-of-mobile-part-2-the-evolution-of-testing/>
- [20]iXie. 5 Best Practices for Mobile Game Test Automation [EB/OL]. Ixie, 2023-11-03. <https://www.ixiegaming.com/blog/5-best-practices-mobile-game-test-automation/>
- [21]Rohith Ramesh. 10 Best Mobile Game Testing Tools in 2025 [EB/OL]. headspin, 2024-08-29. <https://www.headspin.io/blog/best-mobile-game-testing-tools>
- [22]Niharika Bajaj. AI-Driven Innovations: A Guide to Game Bot Development [EB/OL]. Searchmyexpert, 2024-01-15. <https://www.searchmyexpert.com/resources/bot-development/bot-development-gaming>
- [23]Rohith Ramesh. A Comprehensive Guide: How AI is Transforming Game Testing [EB/OL]. headspin, 2024-08-29. <https://www.headspin.io/blog/ai-integration-in-game-testing>
- [24]Einārs Netlis-Galejs. Possible Ways to Automate Games: In-game Movements Using AI & Python [EB/OL]. testdevlab, 2023-08-28. <https://www.testdevlab.com/blog/possible-ways-to-automate-in-game-movements-u>

- sing-ai
- [25] Official Yu-Gi-Oh!, JRPG ANIME BLESS GAMING (JABG). MD AI Demo at Tokyo Dome Yu-Gi-Oh[EB/OL]. Youtube. 2024-02-05
<https://www.youtube.com/watch?v=QdPqFMGs-PM>
- [26] 游戏干线. 网络时代赏金猎人 聚焦游戏中的代练行业[EB/OL]. 知乎, 2019-01-01. <https://zhuanlan.zhihu.com/p/70856977>
- [27] 张良均, 谭立云, 刘名军, 江建明. Python 数据分析与挖掘实战[M]. 北京: 机械工业出版社, 2022.
- [28] OpenCV Team. OpenCV[EB/OL]. 2018. <https://docs.opencv.org/4.x/index.html>
- [29] 蛋片鸡. Python+OpenCV 图像处理（九）—— 模板匹配[EB/OL]. 博客园, 2018-06-02. <https://www.cnblogs.com/FHC1994/p/9123393.html>
- [30] 罗剑. Android 应用开发教程 微课版[M]. 武汉: 华中科技大学出版社, 2020.08
- [31] 丁如敏, 盛娟. 腾讯 Android 自动化测试实战[M]. 北京: 机械工业出版社, 2016.
- [32] Pypi. scrapy-client[EB/OL]. 2017. <https://pypi.org/project/scrapy-client/>
- [33] Reddit. NiceGUI: Let any browser be the frontend for your Python code[EB/OL]. 2023.
https://www.reddit.com/r/Python/comments/10d6ugv/nicegui_let_any_browser_be_the_frontend_for_your/
- [34] Quasar. quasarchs[EB/OL]. 2023. <http://www.quasarchs.com/>
- [35] ultralytics. YOLOv5[EB/OL]. GitHub, 2020 <https://github.com/ultralytics/yolov5>
- [36] 白月黑羽. PyQt 教程[EB/OL]. 白月黑羽, 2023. https://www.byhy.net/py/qt/qt_04/
- [37] Tencent. ncnn[EB/OL]. GitHub, 2017.7 <https://github.com/Tencent/ncnn>
- [38] MaaAssistantArknights. MaaTouch[EB/OL]. GitHub, 2023.
<https://github.com/MaaAssistantArknights/MaaTouch>
- [39] openstf. minitouch[EB/OL]. GitHub, 2023. <https://github.com/openstf/minitouch>
- [40] taujiong. subprocess.Popen 参数介绍 [EB/OL]. 博客园, 2022-04-26.
<https://www.cnblogs.com/superbaby11/p/16195273.html>

致 谢

在本论文的完成过程中，我深感一路走来并不轻松，所幸有诸多良师益友的支持与帮助，才得以顺利完成。首先，我由衷感谢我的指导老师，在整个研究与写作过程中给予了我耐心的指导和专业的建议，而且是在最开始相遇之时就决定做该项目并不断的鼓励、激励我，使我在不断修改中逐步完善论文的结构与内容。其次，感谢其他任课老师在学习阶段为我打下的坚实基础，也为本论文的完成提供了思路启发。

此外，我也要感谢在论文过程中给予我鼓励与协助的同学与朋友，是你们在关键时刻提供的反馈与帮助，使我在遇到困难时不至于一筹莫展。

同时，对本文中引用的诸位前辈学者的研究成果表示诚挚的敬意，正是他们的学术贡献，为我的研究提供了坚实的理论支持。

还有远在千里之外的老同学，他在百忙之中抽出时间使用了他的电脑(PC2)帮助我完成一些测试。

尽管我已尽力完善本论文，但由于个人水平所限，仍难免存在疏漏与不足，诚挚欢迎各位老师和同仁提出宝贵意见。